New Applications of RSA

Yao-Ching Hsieh* Abhishek Jain† Brent Waters‡

September 23, 2026

Abstract

We present new constructions of the following primitives featuring succinct communication:

- **Batch threshold encryption**: Assuming RSA and DCR, we construct a threshold encryption scheme in which a batch of ciphertexts can be decrypted via short hints from a quorum of decryptors. Previously, this was known only from bilinear maps.
- **Aggregate signatures**: Assuming RSA, we construct a signature scheme where signatures on different messages from an arbitrary number of signers can be aggregated into one signature. Our construction achieves bounded-query security. In contrast to prior (fully secure) schemes in the standard model, ours makes black-box use of cryptography.
- **Broadcast encryption**: Assuming RSA, we construct a secret-key encryption scheme where a sender can encrypt a message to multiple recipients via a short ciphertext. Our construction achieves bounded-query security. Previously, (fully secure) broadcast encryption was known from bilinear maps or obfustopia or nonstandard lattice assumptions.

Our constructions are simple and achieve *optimal* succinctness in the key parameter (decryption batch size or number of signers or recipient set size). In this metric, they outperform or match known constructions.

Our results are obtained by applying the lens of projective pseudorandom generators (PPRGs) to these problems. Towards this, we identify new enhanced properties for PPRGs and instantiate them from the RSA assumption.

*University of Washington, ychsieh@cs.washington.edu. This work was done while the author was an intern at NTT Research.

†NTT Research and Johns Hopkins, abhishek.jain@ntt-research.com.

‡UT Austin and NTT Research, bwaters@cs.utexas.edu.

Contents

1 Introduction **3**
- 1.1 Our Results . . . 4
- 1.2 Related Work . . . 6

2 Technical overview **8**
- 2.1 Projective PRG from RSA . . . 8
- 2.2 Batched Threshold Encryption . . . 10
- 2.3 Bounded-query Aggregate Signatures . . . 15
- 2.4 Bounded-query Secret-key Broadcast Encryption . . . 17

3 Preliminaries **19**
- 3.1 Programmable Prime Sampler . . . 19
- 3.2 RSA Assumption . . . 19
- 3.3 Goldreich-Levin Hardcore Bit . . . 20
- 3.4 Cover-Free Set System . . . 20
- 3.5 Simulation-Extractable NIZKPoK . . . 21

4 Reusable Projective PRG **22**
- 4.1 Definition . . . 22
- 4.2 Multi-hop Reusable Projective PRG from RSA . . . 24
- 4.3 Additional Algebraic Properties of Construction 1 . . . 25

5 Keyless Batch Encryption and Batched Threshold Encryption **30**
- 5.1 Keyless Batch Encryption . . . 30
- 5.2 Constructing Keyless Batch Encryption from RSA . . . 31
- 5.3 Batched Threshold Encryption . . . 35
- 5.4 Constructing Batched Threshold Encryption . . . 37

6 Aggregate Signatures **39**
- 6.1 Definition . . . 39
- 6.2 Construction . . . 41

7 Bounded-query Secret-key Broadcast Encryption **46**
- 7.1 Definition . . . 46
- 7.2 Construction . . . 48

8 Homomorphic Threshold Encryption for Cyclic Groups **52**
- 8.1 Homomorphic Threshold Encryption . . . 53
- 8.2 Constructing Multiplicative HTE . . . 54

A Deferred Details of Section 5 **66**
- A.1 Security Definitions of Batched Threshold Encryption . . . 66
- A.2 Deferred Security Proof for Construction 3 . . . 67

1 Introduction

The RSA assumption [RSA78] is fundamental in cryptography due to its role in the early foundations of public-key encryption. Subsequently, RSA and related factoring-based assumptions led to the design of many cryptographic primitives, including signatures [Ita83, Oka88, GMR88], threshold cryptosystems [Sho00, FPS01] and accumulators [BdM94, BP97]. In recent years, however, advances in cryptography, in particular, the design of primitives with powerful functionality and efficiency features, have been led by assumptions with stronger algebraic properties, such as pairing- and lattice-based assumptions.

In this work, we revisit the RSA assumption for building the following cryptographic primitives featuring *succinctness*, i.e., low communication complexity.

- **Batch threshold encryption**: In a threshold encryption scheme [DF90], a ciphertext can be decrypted via ciphertext-dependent hints from any quorum of parties holding key shares. In a batch threshold encryption scheme [CGPP24], each decryptor can release a short hint for any batch of B ciphertexts such that the size of the hint is independent of B.

- **Aggregate signatures**: In an aggregate signature scheme [BGLS03], signatures from B different signers can be aggregated into a single signature of size independent of B.

- **Broadcast encryption**: In a secret-key[1] broadcast encryption scheme [FN94], a sender can encrypt a message to B different recipients, each holding a different decryption key, via a ciphertext of size independent of B.

Known constructions of these primitives in the standard model rely primarily on pairings or lattices or obfustopia assumptions ; we elaborate on prior work later.[2]

In this work, we establish the existence of these primitives from RSA and standard factoring-based assumptions. Some of our schemes achieve security only against polynomially bounded adversarial queries. Nevertheless, all of them achieve *optimal* succinctness in the key parameter B, namely, the number of ciphertexts or signers or recipients. In this metric, they match or often outperform known schemes.

Succinctness via Projective PRGs. All of our constructions achieve succinctness via a common tool, namely, projective pseudorandom generators (PPRGs) [ABI+23]. In a PPRG scheme, given a seed σ, one can generate a "projected seed" σ_S for any subset S of the output positions such that σ_S reveals the output bits for locations in S while the other bits appear pseudorandom. The key requirement is that the size of the projected seed must be independent of $|S|$. PPRGs are known from several assumptions in the common reference string (CRS) model [ABI+23]. Remarkably, the best-known scheme is from RSA, whose asymptotic efficiency (in all parameters) is matched only by obfuscation-based constructions.

In this work, we identify new enhanced properties for PPRGs such as multi-hop projection, homomorphism, public verification and seed union; we elaborate on these later. We show how to instantiate such enhanced PPRGs from RSA. We then obtain our main results by leveraging enhanced PPRGs in a black-box manner.

[1] Most prior work considers *public-key* broadcast encryption. In this work (similar to [BW13]), we consider the secret-key setting which is not believed to be easier than the public-key setting.

[2] Aggregate signatures are an exception in that they can be constructed generically from batch arguments [WW22, DGKV22], which are also known from the subexponential DDH assumption in non-pairing groups [CGJ+23].

1.1 Our Results

Batch Threshold Encryption. Batch threshold encryption (BTE) [CGPP24] extends the standard notion of threshold encryption by providing a mechanism for batched decryption of multiple ciphertexts. Specifically, given a batch of B ciphertexts, any quorum of parties in the decryption committee can release short decryption hints that can be used to decrypt all ciphertexts in the batch, and nothing else. The key nontrivial requirement is that the size of each decryption hint must be *independent* of the batch size B. BTE security is formalized similarly to chosen-ciphertext attack (CCA) security for threshold encryption. In the last two years, BTE has been extensively studied due to its application to mempool privacy [DGK+20, BO22, KLJD25, CGPP24] in the blockchain ecosystem.

We consider the notion of "indexed" BTE [BFOQ25], where each ciphertext is associated with an index that is chosen at the time of encryption, and decryption correctness is guaranteed for any batch of ciphertexts with distinct indices. We show the following result:

Theorem 1. *(Informal) Assuming RSA and DCR, there exists a batched threshold encryption scheme supporting a polynomial-size index space with CCA security. For a maximum batch size B_{max}, it achieves the following efficiency:*

Communication		***Computation***	
Public key	$O(1)$	*Key generation*	$O(B_{\mathsf{max}})$
Secret key	$O(1)$		
Ciphertext	$O(1)$	*Encryption*	$O(1)$
Hint	$O(1)$	*Hint generation*	$O(B)$
		Decryption	$O(B_{\mathsf{max}})$

where B denotes the batch size and the encryption algorithm uses an "expanded" public key that can be generated once in time $O(B_{\mathsf{max}})$. We omit the dependency on the security parameter, and ignore all $\log B$ *factors which are bounded by the security parameter.* [3]

All previously documented BTE schemes in the literature rely on pairings [CGPP24, SAS24, CGPW25, BFOQ25, AFP25, BCF+25, GWWW26, ABD+26, BNRT26, ADG+26, Pol26], and only one of them is based on standard, static assumptions [BFOQ25].[4] Theorem 1 presents a new feasibility result for BTE from standard factoring-based assumptions. Furthermore, our scheme enjoys the following advantages compared to prior work:

- **Optimal Communication**: Our scheme achieves *optimal* communication efficiency. That is, all parameters — public key size, secret key size, ciphertext size and hint size — grow at most polylogarithmically in the batch size. Prior schemes are suboptimal in one or more parameters. In particular, prior pairing-based schemes require public key size at least $O(B_{\mathsf{max}})$.
- **Quasilinear Decryption**: Ours is the first scheme based on standard assumptions that achieves quasilinear decryption time. Most prior schemes require quadratic decryption time. Recent schemes (based on nonstandard assumptions) that achieve quasilinear decryption time either

[3] In any threshold encryption scheme, the runtimes of the key generation and the decryption algorithms necessarily grow with the number of parties. We omit this dependence to keep emphasis on the dependence on the batch size.

[4] In Section 1.2, we discuss two undocumented approaches based on prior work that yield BTE with either: (i) non-succinct hints growing polynomially in decryption committee size, based on LWE, or (ii) non-succinct ciphertexts growing linearly in batch size, based on DDH. Neither of these approaches is desirable due to their inefficiency, which might explain why they have not been documented.

require large secret key size that grows linearly in B_{max} [BNRT26, ADG+26, Pol26], support only an a priori bounded number of batch decryptions [FPTX25], or rely on an idealized model [ABD+26].

On the flip side, our scheme only supports a polynomial-size index space. To achieve a stronger notion of correctness against adversarially generated ciphertexts (referred to as "censorship resilience" [BNRT26]), some recent works construct "indexless" BTE that supports an exponentially large index space [BNRT26, ADG+26, Pol26]. All these works, however, rely on nonstandard assumptions, and constructing indexless BTE from standard assumptions remains an open problem.

As an intermediate building block that may be of independent interest, we construct a homomorphic threshold encryption scheme for any cyclic group from DCR. More generally, we show that any *additively* homomorphic (threshold) encryption scheme with a sufficiently large message space can be transformed into a *multiplicatively* homomorphic one.

Aggregate Signatures. In an aggregate signature scheme [BGLS03], signatures on different messages by an arbitrary number of signers can be aggregated into a single signature.[5] Aggregate signatures have been studied in a variety of models; see Section 1.2. In the standard model, aggregate signatures are known from obfustopia assumptions [HKW15] or non-interactive batch arguments [WW22, DGKV22]. The latter can be instantiated from LWE [CJJ22], pairings [WW22], and subexponential DDH [CGJ+23]. All of these constructions, however, make *non-black-box* use of cryptography.

Assuming RSA, we present a purely algebraic *black-box* construction of aggregate signatures with *bounded-query* security. Our scheme guarantees unforgeability in the standard model when the adversary makes an a priori (polynomially) bounded number of signing queries per honest signer.

Theorem 2. *(Informal) Assuming RSA, there exists a bounded-query aggregate signature scheme. For a signature query bound Q, it achieves the following efficiency:*

Communication		***Computation***	
Public parameter	$O(1)$	*Setup*	$O(Q^2)$
Signing & verification key	$O(1)$	*Key Generation*	$O(1)$
Signature & aggregated signature	$O(1)$	*Signing*	$O(Q^2)$
		Aggregation	$O(BQ)$
		Verification	$O(BQ + Q^2)$

where B is the number of signatures aggregated.

Note that the signature query bound impacts only the runtime, but not the communication efficiency of our scheme.

Broadcast Encryption. In a broadcast encryption scheme [FN94], a sender can encrypt a message to a set of recipients, each holding a different decryption key, with a ciphertext whose size is sublinear in the number of recipients. The encryption scheme should be collusion-resistant, i.e., multiple colluding users outside a recipient set S should not be able to decrypt a ciphertext encrypted to S. In the standard model, broadcast encryption schemes are known from pairings [BGW05, BW06, GW09, Wat09, GKSW10, CGW15, GKW18, Wee21, KMW23], obfustopia assumptions [BZ14, GVW19], and

[5] This is different from multisignatures [Ita83], where we only require aggregation of signatures on the *same* message.

nonstandard lattice assumptions [BV22, Wee22, Wee24, CW24, Wee25, CHW25, AMR25, AMR26, GY26]; see Section 1.2 for a survey.

Assuming RSA, we present a secret-key broadcast encryption scheme with *bounded-query* security. Our scheme achieves adaptive security in the standard model when the adversary makes an a priori (polynomially) bounded number of encryption queries. We can further strengthen the security model to achieve *bounded-queryset* security, where the total number of encryption queries is unbounded, but the number of distinct recipient sets appearing in those queries is a priori bounded.

Theorem 3. *(Informal) Assuming RSA, there exists an adaptively secure bounded-query secret-key broadcast encryption scheme. For any number $B_{\max}$ of users and encryption query bound Q, our scheme achieves the following efficiency:*

Communication		***Computation***	
Master secret key	$O(1)$	*Key generation*	$O(B_{\max}Q)$
Party secret key	$O(Q)$		
Ciphertext	$O(1)$	*Encryption*	$O(B)$
		Decryption	$O(B)$

where B is the broadcast set size.

Our scheme achieves *optimal* succinctness in the number of users. Previously, such efficiency was only known from obfustopia assumptions [BZ14, GVW19] or nonstandard lattice assumptions [Wee25, AMR25]. Achieving optimal succinctness with full (i.e., unbounded-query) security from standard, non-obfustopia assumptions remains an open problem. Furthermore, even under recent nonstandard lattice assumptions, achieving both adaptive security and optimal succinctness remains open.

1.2 Related Work

Succinct primitives from RSA. In this work, we adopt the projective PRG abstraction from [ABI+23] as a building block for constructing new primitives with succinct aggregation. We note that most of the algebraic properties of RSA groups that underlie the [ABI+23] construction (as well as our work) have a rich history dating back to the 1990s. This includes work on RSA-based hierarchical key assignments [AT83, CT90], RSA-based accumulators [BdM94, BP97, STY01] and batch RSA [Fia90]. Nevertheless, the PPRG abstraction provides a clean view of the functionality provided by existing techniques, enabling simple constructions of new primitives.

Batched threshold encryption. Prior to our work, BTE was known only from pairing-based assumptions. Bormet et al. [BFOQ25] first proposed the notion and provided a construction supporting a polynomial-size index space from the standard Decisional Bilinear Diffie-Hellman assumption. However, the construction suffers from two main shortcomings: (i) the public key size grows quadratically with the maximal batch size, and (ii) the decryption complexity is quadratic in the batch size. Subsequent work improved upon these aspects, achieving a linear-size public key [BCF+25] and (quasi)linear time decryption [ABD+26]. Furthermore, some recent works also support an exponential-size index space [BNRT26, ADG+26, Pol26]. Nevertheless, all of these enhancements rely on *nonstandard* q-type assumptions or the use of generic or algebraic group models. Furthermore, all known schemes supporting an exponential-size index space require larger public and secret key sizes compared to constructions that support a polynomial-size index space.

Compared to these works, our work achieves the first construction of BTE with fully constant-size communication (ignoring polynomial factors in the security parameter), and (quasi)linear time decryption, from standard factoring-based assumptions (RSA and DCR). Similarly to the only prior work based on a standard assumption [BFOQ25], our scheme supports a polynomial-size index space.

If we weaken the succinctness requirement of BTE, it is possible to obtain constructions from other assumptions. Under the LWE assumption, Boneh et al. [BLT25] presented a non-threshold batch encryption scheme. Combining this scheme with the universal thresholdizer [BGG+18] from LWE yields a BTE scheme with *non-succinct decryption hints* whose size grows with the number of decrypting parties. Under the DDH assumption, Applebaum et al. [AIKW13] implicitly constructed a non-threshold batch encryption scheme in which the size of each ciphertext grows linearly with the batch size. The scheme additionally admits key homomorphism; when plugged into the BEAT-MEV recipe with threshold ElGamal, it yields a BTE scheme with *non-succinct ciphertexts*.

All of the above works fall into the category of "epochless" BTE. There has also been a large body of work studying a weaker notion of *epoched* BTE [GV22, CGPP24, SAS24, CGPW25, AFP25, GWWW26], where every ciphertext is additionally tied to an epoch/tag and has to be discarded when *one* decryption hint for the epoch is revealed.

Aggregate signatures. Early constructions of (full-fledged) aggregate signatures either relied on random oracles [BGLS03, BNN07] or on primitives without instantiations from standard assumptions, such as SNARKs [BCCT13] or multilinear maps [RS09, FHPS13, HSW13]. Later, standard model constructions from well-founded primitives, such as iO [HKW15] and batch arguments (BARGs) [DGKV22], were proposed. In particular, BARGs can be constructed from LWE [CJJ22], k-Lin [WW22], or subexponential DDH [CGJ+23]. Nevertheless, these constructions all use cryptography in a non-black-box manner, either by obfuscating cryptographic primitives or by proving cryptographic relations.

In our work, we provide a direct construction of aggregate signatures from RSA. We consider the relaxed security model of bounded-query security.[6] To the best of our knowledge, such a notion has not previously been studied in the literature. When the query bound is restricted to one, security under our notion is implied by the security notion for synchronized aggregate signatures [GR06]. Constructions of synchronized aggregate signatures in the plain model are known from bilinear maps [AGH10] and RSA [HW18].

There has also been a large body of research on aggregate signatures under different relaxations, including aggregation for only a single signer [GV22], aggregation for multiple signers on the same message (i.e., non-interactive multisignatures) [Bol03, LOS+06, BDN18], a priori bounded aggregation [WW22, HWW25a], relaxed succinctness requirements [DHSS20, AAB+24, Sak25], and signing procedures that are sequential [LMRS04, LOS+06, BGOY07, MT07, Nev08, FLS12, BGR12, BGR12, LLY13a, LLY13b, EMP16, BT23], interactive [BJ10, BK20, NRS26], or synchronized [GR06, AGH10, HW18, FSZ22].

Broadcast encryption. Public-key broadcast encryption in the plain model is known from many different types of assumptions and primitives, including

- Static pairing-based assumptions, such as subgroup decision [BW06], 2-Lin [Wat09, GKSW10], or k-Lin [CGW15, GKW18, Wee21, KMW23]. Restricting to constructions with constant-size

[6] This is incomparable with bounded-aggregation aggregate signatures [WW22, HWW25a]. Bounded-query considers full functionality with relaxed security, whereas bounded-aggregation considers full security with restricted functionality.

secret keys and ciphertexts, the best-known construction requires a master public key whose size is quadratic in the number of users.

- q-type assumptions [BGW05, GW09, KMW23, HWW25b]. Restricting to constructions with constant-size secret keys and ciphertexts, the best-known construction requires a master public key whose size is linear in the number of users.
- Nonstandard lattice-based assumptions [BV22, Wee22, Wee24, CW24, Wee25, CHW25, AMR25, AMR26, GY26]. In this setting, optimal broadcast encryption schemes with constant-size public keys, secret keys, and ciphertexts are achievable. On the other hand, constructing broadcast encryption schemes from standard lattice assumptions remains open.
- Obfuscation-based primitives, such as multilinear maps [BS03, BW13, BWZ14], witness encryption [GVW19, FWW23], and indistinguishability obfuscation [BZ14]. In this setting, optimal broadcast encryption schemes are achievable.

Additionally, optimal broadcast encryption schemes with *adaptive* security are known only from obfuscation-based primitives.

In this work, we consider relaxations in two directions. First, we consider *secret-key* broadcast encryption, which is not known to be easier than the public-key setting. To the best of our knowledge, the only existing construction that is inherently secret-key relies on bit-fixing constrained PRFs [BW13], which are known only from multilinear maps. Second, we consider *bounded-query* security. To the best of our knowledge, this relaxation has not previously been considered in the literature.

2 Technical overview

2.1 Projective PRG from RSA

The starting point of all our constructions is the projective pseudorandom generator (PPRG) construction from RSA given in [ABI+23]. As discussed in Section 1.2, this construction provides a useful abstraction that combines relevant algebraic properties of RSA developed in the literature and gives us a clean interface upon which to build.

In a PPRG scheme, one can expand a short seed σ into a pseudorandom string y of length ℓ, similar to a standard PRG. The key additional property is *projection*: for any subset $S \subseteq [\ell]$, one can "project" the seed σ to a succinct seed σ_S that can be used to compute the bits of y at positions in S, while the bits at positions outside S remain pseudorandom even given σ_S. Importantly, σ_S should be succinct in $|S|$.

The RSA-based PPRG construction is fairly intuitive. Given an element $v \in \mathbb{Z}_N^*$ and pairwise coprime exponents $e_1, \ldots, e_\ell$, each coprime to $\phi(N)$, the "projected" element corresponding to a set $S \subseteq [\ell]$ is of the form

$$u_S = v^{\prod_{i \in S} 1/e_i}.$$

It reveals RSA preimages $u_{\{i\}} = v^{1/e_i}$ for all $i \in S$, but finding RSA preimages $u_{\{j\}} = v^{1/e_j}$ for $j \notin S$ remains hard. In other words, the element u_S serves as a succinct projected seed that reveals the private information v^{1/e_i} only for indices in S.

We now sketch the RSA-based PPRG construction from [ABI+23], with minor adaptations.

- $\mathsf{Setup}(1^\lambda, 1^\ell)$: The setup of a PPRG with output length ℓ consists of an RSA modulus $N = pq$ and ℓ distinct (pseudo)random prime exponents $e_1, \ldots, e_\ell$. Furthermore, the random primes

can be described by a short random seed ρ from which they are generated.[7] The common reference string is $\mathsf{crs} = (N, \rho)$, which has size $\mathsf{poly}(\lambda)$ and is therefore succinct in the PPRG output length ℓ.

- $\mathsf{SeedGen}(\mathsf{crs})$: A master seed of the PPRG is simply a random element $u \leftarrow \mathbb{Z}_N^*$. The master seed uniquely defines the value
$$v = u^{\prod_{i\in[\ell]} e_i},$$
where each RSA preimage v^{1/e_i} can be efficiently computed from u.

- $\mathsf{Project}(\mathsf{crs}, u, S)$: To project the master seed u to a subset S, one simply computes
$$u_S = u^{\prod_{i\in[\ell]\setminus S} e_i} = v^{\prod_{i\in S} 1/e_i}.$$

- $\mathsf{Eval}(\mathsf{crs}, u_S, i \in S)$: The i-th pseudorandom bit generated by the seed u_S is defined as $\mathsf{Ext}(u_{\{i\}})$, the hardcore bit extracted from the singleton projection $u_{\{i\}} = v^{1/e_i}$. The projection is computed as
$$u_{\{i\}} = (u_S)^{\prod_{j\in S\setminus\{i\}} e_j}.$$

It was shown in [ABI+23] that the unrevealed bits are selectively pseudorandom assuming the hardness of RSA (referred to as robustness in [ABI+23]). Specifically, given multiple projected seeds $u_{S_1}, \ldots, u_{S_k}$, the unrevealed bits at indices in $[\ell] \setminus \cup_{j\in[k]} S_j$ remain pseudorandom. It was further shown in [HWW25b] that the construction achieves adaptive security, i.e., where the subsets are chosen after the CRS.

In our actual construction, we restrict the seed space to QR_N for its cyclicity properties. We omit the details here, and refer readers to Section 4 for the full description.

Enhanced Properties. In this work, we observe additional properties of the construction that are useful for our applications. We summarize them below.

- **Multi-hop projection.** While the original definition of PPRGs only requires projection from the master seed, it is easy to see that the above RSA-based construction supports multi-hop projection. Given a projected seed $u_S = u^{\prod_{i\in[\ell]\setminus S} e_i}$ that allows evaluation on the set S, one can further project it to a subset $S' \subseteq S$ by computing
$$u_{S'} = (u_S)^{\prod_{i\in S\setminus S'} e_i} = u^{\prod_{i\in[\ell]\setminus S'} e_i}.$$
We note that most existing PPRG constructions (e.g., the group-based and lattice-based constructions in [ABI+23]) do *not* support multi-hop projection.

- **Homomorphism.** In the above construction, all operations can be performed without knowledge of the factorization of N. This allows multiple seeds to be generated and projected using the same modulus N and the same set of prime exponents. Seeds under the same setup then admit homomorphic relations: the product of two seeds, $u_1 u_2$, satisfies
$$\mathsf{Project}(\mathsf{crs}, u_1 u_2, S) = \mathsf{Project}(\mathsf{crs}, u_1, S) \cdot \mathsf{Project}(\mathsf{crs}, u_2, S).$$

[7] For the security proof, one would additionally need the prime sampling procedure to be *locally programmable*. It was shown in [ABI+23] that such a sampler exists with seed length $\mathsf{poly}(\lambda) \cdot \log \ell$. We refer readers to [ABI+23] for details.

- **Verifiability.** Another useful property of the construction is the verifiability of projected seeds. The value $v = u^{\prod_{j\in[\ell]} e_j} = \mathsf{Project}(\mathsf{crs}, u, \emptyset)$ can serve as a "verification seed" without leaking the evaluation at any index. With the multi-hop projection property, one can verify any projected seed u_S by checking whether u_S projects to v, i.e.,

$$u_S^{\prod_{i\in S} e_i} \stackrel{?}{=} v.$$

- **Seed unions.** Finally, the construction also supports a seed union operation: given two projected seeds u_{S_1}, u_{S_2} from the same master seed u, one can efficiently compute a projected seed $u_{S_1\cup S_2}$ for the union $S_1 \cup S_2$. This follows from Shamir's trick [Sha83], which is widely used in the RSA literature. Given $u^{1/x}$ and $u^{1/y}$, where x, y are coprime, one can compute $u^{1/xy}$ as $u^{b/x}u^{a/y}$, where (a, b) are the Bézout coefficients [Béz79] satisfying $ax + by = 1$.

2.2 Batched Threshold Encryption

We begin with a brief review of the syntax of BTE.

- Setup: The setup algorithm takes as input the maximum batch size B and index space size ℓ, and outputs the public parameters pp.
- KeyGen: Based on pp, the key generation algorithm generates a t-out-of-n threshold key tuple and outputs the public key pk and the secret keys $\mathsf{sk}_1, \ldots, \mathsf{sk}_n$ for the n parties.
- Enc: The encryption algorithm encrypts a message m under the public key pk and an index $j \in [\ell]$.
- $\mathsf{PartialDec}$: The partial decryption algorithm takes in a threshold secret key sk_i and a batch of ciphertexts $\{\mathsf{ct}_j\}_{j\in S}$ corresponding to *distinct* indices in $[\ell]$ and outputs a *succinct* partial decryption hint d_i.
- $\mathsf{Combine}$: The combine algorithm combines the batch of ciphertexts and the partial decryption hints of t or more parties to recover the decrypted messages for every ciphertext in the batch.

The core requirement of a BTE scheme is *succinctness*: The partial decryption hints d_i should be sublinear in the batch size B (otherwise the scheme reduces to a standard threshold encryption scheme), and the ciphertext size should be sublinear in the number of decryption parties n (otherwise the scheme reduces to a non-thresholdized variant).

The BEAT-MEV Template for BTE. Our construction of BTE follows closely a general recipe that is implicit in [BFOQ25]. In the following, we refer to it as the BEAT-MEV template. It consists of the following three building blocks:

- *Keyless Batch Encryption* (KBE): A KBE scheme weakens the notion of BTE in two ways: First, it is "keyless" in the sense that each ciphertext is sampled with its own decryption key, using common public parameters. Second, threshold decryption is not required.

  More specifically, in a KBE scheme: (i) The *encryption* algorithm receives as input common public parameters and a message m_i. It outputs a ciphertext ct_i (with index i) that encrypts m_i together with a decryption key h_i. (ii) The *key batching* algorithm takes as input multiple decryption keys $\{h_i\}_{i\in S}$ and outputs a batched key $\widetilde{h}_S$ of size independent of $|S|$. (iii) The *decryption* algorithm takes as input a batch of ciphertexts $\{\mathsf{ct}_i\}_{i\in S}$ and a batched key $\widetilde{h}_S$ and outputs all the decrypted messages $\{m_i\}_{i\in S}$.

- *Homomorphic Threshold Encryption* (HTE): This is a standard threshold encryption scheme that additionally supports homomorphic computation. In particular, the homomorphism class supported by the scheme must include the key batching operation $\{h_i\} \mapsto \widetilde{h}$ of the KBE scheme.
- *Non-Interactive Zero Knowledge* (NIZK): This is a standard NIZK scheme in the CRS model for proving validity of ciphertexts of KBE and HTE.

Intuitively, the BEAT-MEV template "deconstructs" the design of BTE into three parts: KBE implements the batch decryption property, HTE lifts batch decryption to threshold batch decryption, and NIZK proofs are used for achieving CCA security. Next, we describe this template, focusing on the encryption, partial decryption and combine algorithms.

- The encryption algorithm has three steps: (i) First encrypt a message m_i using the KBE scheme to obtain a ciphertext ct_i and the corresponding decryption key h_i. (ii) Next, encrypt h_i using HTE. (iii) Finally generate a NIZK proof of ciphertext validity. The final ciphertext contains all three components.
- The partial decryption algorithm involves the following steps: (i) Given a batch of BTE ciphertexts containing HTE encryptions of $\{h_i\}_{i \in S}$, use the homomorphism property of HTE to compute an HTE encryption of the batched key $\widetilde{h}_S$. (ii) Release a partial decryption for this ciphertext.
- The combine algorithm first recovers the batched key $\widetilde{h}_S$ from the partial decryptions. It then uses it to decrypt every KBE ciphertext in the batch of input BTE ciphertexts.

The correctness of the above template can be verified by inspection. We omit the security analysis from this overview, as it follows from standard techniques.

In the following, we show how each of the above building blocks can be constructed with optimal succinctness using factoring-based assumptions.

Keyless Batch Encryption from Key-Homomorphic Puncturable PRFs. The work of [BFOQ25] provides a construction of KBE from any *key-homomorphic puncturable* pseudorandom function (PRF). In the following, we recall this generic construction. Looking ahead, our RSA-based construction of KBE will build upon this generic approach.

- The core component of the construction is a key-homomorphic puncturable PRF PRF, which is a PRF with the following additional properties:
  - Puncturable: From a PRF key k, one can efficiently derive a punctured key $k_{\{i\}}$ that allows evaluation of $\mathsf{PRF}(k, x)$ for all $x \neq i$. Furthermore, given the punctured key $k_{\{i\}}$, the PRF output $\mathsf{PRF}(k, i)$ at the punctured point i remains pseudorandom.
  - Key-homomorphic: For any two PRF keys k_1 and k_2 and for all inputs x, $\mathsf{PRF}(k_1 \cdot k_2, x) = \mathsf{PRF}(k_1, x) \cdot \mathsf{PRF}(k_2, x)$.
- To encrypt a message m with index i, the algorithm samples a fresh PRF key k_i and computes the ciphertext $\mathsf{ct}_i = (m \oplus \mathsf{PRF}(k_i, i), k_{i,\{i\}})$, where $k_{i,\{i\}}$ is the key k_i punctured at index i. The key k_i serves as the decryption key.

- The key batching algorithm uses the key homomorphism of the puncturable PRF. Given a set of distinct indices $S \subseteq [\ell]$ and the corresponding decryption keys $\{h_i = k_i\}_{i \in S}$, it computes the batched key $\widetilde{k}_S = \prod_{i \in S} k_i$.
- The batch decryption algorithm evaluates each ciphertext pad $\mathsf{PRF}(k_i, i)$ using the following homomorphic relation:

$$\begin{aligned}
\mathsf{PRF}(k_i, i) &= \mathsf{PRF}\left(\widetilde{k}_S \Big/ \prod_{j \in S \setminus \{i\}} k_j, i\right) \\
&= \mathsf{PRF}(\widetilde{k}_S, i) \Big/ \prod_{j \in S \setminus \{i\}} \mathsf{PRF}(k_j, i) \quad \text{(Homomorphism)} \\
&= \mathsf{PRF}(\widetilde{k}_S, i) \Big/ \prod_{j \in S \setminus \{i\}} \mathsf{PRF}(k_{j,\{j\}}, i) \quad \text{(Evaluate with punctured keys)}
\end{aligned}$$

The work of [BFOQ25] instantiates the key-homomorphic puncturable PRF from the pairing-based construction of [DGM24]. The construction supports a polynomial-size input domain and requires a CRS size polynomial in the domain size. This gives a KBE scheme with a polynomial-size index space and large public parameters.

Keyless Batch Encryption from RSA. We show that using RSA, we can adapt the above construction to achieve a constant-size CRS.

Our starting observation is that a projective PRG is a strengthening of a puncturable PRF with a polynomial-size domain. Recall that a punctured key $k_{\{i\}}$ allows one to evaluate the PRF at all points except the punctured point i, while keeping the evaluation $\mathsf{PRF}(k, i)$ pseudorandom. This is a special case of projecting the PRG seed u to the set of all non-punctured points. Therefore, we can instantiate the above construction with the RSA-based projective PRG described in section 2.1. More specifically:

- For a ciphertext with index i, a random master seed u_i for the projective PRG serves as the decryption key.
- The punctured key $k_{i,\{i\}}$ is instantiated with a projected seed, i.e., a seed that is projected to all the non-punctured points, $u_{i,[B]\setminus\{i\}} = \mathsf{Project}(u_i, [B] \setminus \{i\})$.
- The message pad is instantiated with the PRG evaluation $\mathsf{Eval}(u_i, i) = \mathsf{Eval}(u_{i,\{i\}}, i)$. This evaluation can be computed in constant time given the projected seed $u_{i,\{i\}}$ corresponding to the singleton set $\{i\}$.

The remaining task is aggregation of decryption keys from different ciphertexts. Our key observation is that the enhanced properties of *seed union* and *multi-hop projection* described in Section 2.1 can be used to achieve this goal.

Specifically, for a batch indexed by $S \subseteq [B]$, the batched key is $\widetilde{u}_S = \prod_{i \in S} u_i$. Decryption uses

the following relation:

$$\begin{aligned} u_{i,\{i\}} &= \mathsf{Project}(u_i, \{i\}) \\ &= \mathsf{Project}(\widetilde{u}_S, \{i\}) \Big/ \prod_{j \in S \setminus \{i\}} \mathsf{Project}(u_j, \{i\}) \quad \text{(Seed homomorphism)} \\ &= \mathsf{Project}(\widetilde{u}_S, \{i\}) \Big/ \prod_{j \in S \setminus \{i\}} \mathsf{Project}(u_{j,[B]\setminus\{j\}}, \{i\}) \quad \text{(Multihop projection)} \end{aligned}$$

With this, we obtain a KBE scheme for polynomial-size index spaces. Note that the resulting construction has *optimally succinct* public parameters: the setup of the RSA-based PPRG has size $\mathsf{poly}(\lambda)$, independent of the batch size and the index space size.

KBE with Quasilinear Decryption Time. The KBE construction based on key-homomorphic puncturable PRFs suffers from high decryption time: each batched decryption of B ciphertexts involves B^2 evaluations of the PRF. This is because to decrypt a ciphertext with index i, one needs to evaluate $\mathsf{PRF}(k_j, i)$ at *every other index $j \neq i$ in the batch*. Our RSA-based instantiation described above has even worse decryption time of B^3 because the PPRG evaluation at each index takes linear time.

Recent work [ABD+26] has explored FFT-style techniques to construct KBE with quasilinear decryption time. However, their approach is highly tied to the specific structure of pairing-based PRF construction, and it is unclear how it can be adapted to the RSA setting.

In the following, we describe a new tree-based batch evaluation strategy that decreases the decryption time from cubic to quasilinear in B. Our approach crucially leverages the *multi-hop projection* property of the RSA-based PPRG.

We first examine the projection and evaluation procedures of the RSA-based projective PRG. Projecting a seed u_S to $u_{S'}$ involves raising the seed to the exponent e_i for each index $i \in S \setminus S'$, thereby taking time linear in $|S \setminus S'|$. The evaluation procedure of the PRG at index i is thus dominated by the projection of the master seed u to the set $\{i\}$, which takes time $O(B)$.

Since projection is the main bottleneck, the idea is to *reuse* intermediate projected seeds for multiple evaluations as much as possible. Let's start with a simple case in which, given a master seed u, we want to evaluate the PRG at all indices in $[B]$ efficiently. Naively, this would take $O(B^2)$ time, but we can improve it to $O(B \log B)$ time using the following tree-based strategy:

- Without loss of generality, let $B = 2^d$ be a power of two. Define a tree of depth d in which each node is associated with a prefix and a set. The root corresponds to the empty prefix, and its associated set is $[B]$. The i-th node at depth k corresponds to the length-k binary representation of i, and its associated set contains all indices in $[B]$ with binary representations that share this prefix. By definition, the set associated with a node at depth k is the union of the sets associated with its two children at depth $k+1$.

- The projection procedure computes the projected seed for each node from the root to the leaves. The root seed is the master seed u, corresponding to the projection onto the full set $[B]$. Given the projected seed u_S of an intermediate node associated with a set S, the projected seeds for its children, associated with sets S_1 and S_2, are computed by projecting u_S onto S_1 and S_2. Together, these computations take time linear in $|S|$, or equivalently, linear in 2^{d-k} for a node at depth k.

- The overall runtime for computing all projected seeds from the root to the leaves is the sum of the runtimes at each depth. At depth k, there are 2^k nodes, each taking time $O(2^{d-k})$ to compute its children's projected seeds. Therefore, the total runtime is $\sum_{k=0}^{d} 2^k \cdot O(2^{d-k}) = O(d \cdot 2^d) = O(B \log B)$.

Returning to the batch decryption procedure, the above strategy allows efficient batch evaluation of $\mathsf{Project}(\widetilde{u}_S, \{i\})$ for all $i \in [B]$ in $O(B \log B)$ time. It remains to optimize the computation of the canceling terms $\prod_{j \in S \setminus \{i\}} \mathsf{Project}(u_{j,[B]\setminus\{j\}}, \{i\})$ for all $i \in [B]$. The idea again relies on the prefix-based tree. We first reshape the construction to fit the tree structure. In the ciphertext ct_j at index j, instead of providing a single projected seed $u_{j,[B]\setminus\{j\}}$, we provide d projected seeds corresponding to prefix sets that cover $[B] \setminus \{j\}$.

$$u_{j,S_1}, \ldots, u_{j,S_d} : \quad S_k := \{i : \text{the binary representations of } j \text{ and } i \text{ first differ at the } k\text{-th bit}\}.$$

In other words, the projected seeds correspond to the nodes on the co-path from the root to the leaf representing index j in the prefix tree.

With this change, the batch decryption procedure can first homomorphically batch all the projected seeds associated with each node in the tree. For every node with associated set T, the batched seed is simply

$$\widetilde{u}_T := \prod_{j \in S : T \text{ is on the co-path of } j} u_{j,T}.$$

Because i belongs to exactly one set on the co-path of every $j \in S \setminus \{i\}$, and each such set lies on the path of i, the canceling term can be rewritten as

$$\prod_{j \in S \setminus \{i\}} \mathsf{Project}(u_j, \{i\}) = \prod_{T \text{ on the path of } i} \mathsf{Project}(\widetilde{u}_T, \{i\}).$$

Finally, one can batch evaluate each $\widetilde{u}_T$ in the prefix tree in time $|T| \log |T|$. Summing over the tree, the overall runtime remains $\widetilde{O}(B)$, giving an almost optimal decryption time.

Multiplicatively Homomorphic Threshold Encryption from DCR. The second part of the BTE construction requires a homomorphic threshold encryption scheme (HTE) that supports the key batching operation of the KBE scheme. In our case, the operation is a simple multiplicative function $\{u_i\} \mapsto \prod_i u_i$ in the RSA group $\mathbb{Z}_N^*$.

It is tempting to use threshold ElGamal over $\mathbb{Z}_N^*$, as in the BEAT-MEV construction. Unfortunately, DDH does not hold in $\mathbb{Z}_N^*$, and worse, the threshold ElGamal scheme requires knowledge of the group order to perform reconstruction, which is not available in RSA-based constructions.

To overcome this issue, our starting point is the threshold Paillier encryption scheme [Pai99, FPS01] based on the decisional composite residuosity (DCR) assumption, where threshold decryption does not require knowledge of the group order. However, this scheme only supports *additive* homomorphism over $\mathbb{Z}_M$, whereas we require *multiplicative* homomorphism. We show that, by using threshold Paillier over a sufficiently large RSA modulus M, one can achieve multiplicative homomorphism *generically*. In fact, we build a multiplicative HTE over any cyclic group from any additive HTE scheme over a sufficiently large additive group.

The construction is intuitive. For a cyclic group $\mathcal{G}$ of order $N \ll M$, where the underlying additive threshold encryption scheme is over $\mathbb{Z}_M$, we use the following simple two-layer encryption technique:

$$\mathsf{Enc}(m \in \mathcal{G}) = \left(m * g^r, \mathsf{HTE}_{\mathsf{add}}.\mathsf{Enc}(r)\right),$$

where the message is padded with g^r, and the randomness r is encrypted under an additively homomorphic scheme. It is easy to see that, if the message space $\mathbb{Z}_M$ of the additive scheme is large enough (such that $\sum r \bmod M = \sum r$), the above construction is multiplicatively homomorphic over $\mathcal{G}$:

$$\left((m_1 * g^{r_1}) \cdot (m_2 * g^{r_2}), \mathsf{HTE}_{\mathsf{add}}.\mathsf{Enc}(r_1) \boxplus \mathsf{HTE}_{\mathsf{add}}.\mathsf{Enc}(r_2)\right) \in \mathsf{Enc}(m_1 \cdot m_2).$$

Note that the above construction does not require knowing the exact order of the group $\mathcal{G}$. As long as an upper bound N on the order of $\mathcal{G}$ is known, one can sample the randomness r from $[2^\lambda N]$ so that g^r is statistically close to uniform in $\mathcal{G}$.

While $\mathbb{Z}_N^*$ is not necessarily a cyclic group, we can restrict the key space of the KBE scheme to the subgroup QR_N of quadratic residues modulo N. When N is a product of two safe primes, i.e., $N = pq$ with $p = 2p' + 1$ and $q = 2q' + 1$ for some primes p', q', the subgroup QR_N is cyclic of order $p'q'$, with an efficiently computable generator g. The above construction is then IND-CPA secure under the DCR assumption, as the one-time pad g^r is random in QR_N for random r. This restriction, in turn, requires the PPRG seed u to be sampled from QR_N. This modification does not affect the properties of the PPRG construction: since random elements in $\mathbb{Z}_N^*$ fall in QR_N with non-negligible probability, PPRG security over $\mathbb{Z}_N^*$ directly implies security over QR_N.

CCA Security. Finally, BEAT-MEV relies on a NIZK scheme to upgrade the security of the batched threshold encryption scheme to IND-CCA security, with the NIZK validity proof enabling simulation of the decryption oracles. Constructions of NIZKs from RSA are known [FLS90]; however, the sizes of the CRS and proofs may grow with the size of the relation circuit. In our construction, the encryption algorithm must compute all the projected seeds $u_{S_1}, \ldots, u_{S_b}$ corresponding to the co-path of the index i. Computing these seeds takes time $O(B \log B)$, resulting in a validation circuit of size $O(B \log B)$ and compromising the succinctness guarantee of our construction.

To solve this issue, we observe that the encryption algorithm can be divided into an $O(B \log B)$-time public preprocessing phase and a subsequent $O(\log B)$-time online phase. Therefore, it suffices to provide a validity proof *only for the online phase*, thereby keeping all components of our construction succinct. We refer readers to Section 5 for more details.

2.3 Bounded-query Aggregate Signatures

Signature scheme from verifiable PPRGs. Next, we demonstrate how the RSA-based PPRG from Section 2.1 can be used to build a signature scheme. The key property we rely on is the verifiability of projected seeds: given the verification seed $v = \mathsf{Project}(\mathsf{crs}, u, \emptyset)$ of a master seed u, we can verify any projected seed u_S by projecting it to the empty set. This gives the following candidate signature scheme:

- A fresh PPRG seed u is the signing key, and the verification seed v is the verification key.
- The signature of a set $S \subset [\ell]$ is the projected seed $u_S = \mathsf{Project}(\mathsf{crs}, u, S)$.
- The verification of a signature u_S for a set S is done by checking whether u_S projects to the verification seed v, i.e., $u_S^{\prod_{i \in S} e_i} \stackrel{?}{=} v$.

The scheme gives a weak form of unforgeability: it is computationally hard to forge a valid signature for a set S given multiple valid signatures on sets $S_1, \ldots, S_q$, as long as S is not a subset of $S_1 \cup S_2 \cup \cdots \cup S_q$. This follows from the pseudorandomness of the underlying PPRG. Given $u_{S_1}, \ldots, u_{S_q}$, the PPRG guarantees that any evaluation at an index outside $\cup_{i \in [q]} S_i$ remains

pseudorandom. An adversary that successfully forges u_S can evaluate the PPRG at some index outside $\cup_{i\in[q]} S_i$, thereby breaking the pseudorandomness of the PPRG.

To obtain a fully unforgeable signature scheme from the above template, one needs an embedding $m \mapsto S_m$ such that any challenge set S_{m^*} is not contained in the union of the signing-query sets $S_{m_1}, \ldots, S_{m_q}$. Unfortunately, because the PPRG supports only a polynomially bounded universe of size ℓ, any collection of $\ell + 1$ messages must contain a cover, thereby limiting the message space to at most ℓ messages.

To obtain a large message space, we turn to the notion of *bounded-query* security, where the forging adversary is limited to querying signatures for at most Q messages. In this setting, it suffices to construct set embeddings such that, for every tuple $(S_{m^*}, S_{m_1}, \ldots, S_{m_Q})$ associated with distinct messages, $S_{m^*} \not\subseteq \cup_{i\in[Q]} S_{m_i}$. Such embeddings are also known as Q-cover-free set systems [EFF85], and it was shown in [KRS99] that one can efficiently construct a Q-cover-free set system with a message space of size 2^λ, where every message maps to a subset of $[\ell]$ of size $O(Q\lambda)$ and the universe has size $\ell = O_\lambda(Q^2\lambda^2)$.

Aggregating signatures. Because every signature is a projected seed u_S, we can immediately obtain a *single-signer* aggregate signature scheme using the seed union property of the PPRG. Given a batch of signatures $u_{S_1}, \ldots, u_{S_B}$ for messages $S_1, \ldots, S_B$, one can efficiently compute the aggregated signature as $u_{S_1\cup\cdots\cup S_B}$, whose size is independent of the number of messages B. Verifying the aggregated signature again requires a single exponentiation check:

$$u_{S_1\cup\cdots\cup S_B}^{E} \stackrel{?}{=} v, \quad \text{where } E = \prod_{i\in S_1\cup\cdots\cup S_B} e_i.$$

Finally, to obtain a full-fledged aggregate signature scheme in which signatures from different signers can be combined, we leverage the reusability of the setup so that the modulus N can be shared across different signers. As in [HW18], multiple exponential relations under the same modulus can be batch-verified. After first aggregating all signatures under each distinct verification key, let $B_{\mathsf{vk}} \le B$ be the number of distinct verification keys. The verification relation for each $j \in [B_{\mathsf{vk}}]$ boils down to a single exponentiation check of the form $u_j^{E_j} \stackrel{?}{=} v_j$, where u_j is the aggregated signature under verification key v_j. Assuming all v_j are honest, it suffices to reveal the signature product $\widetilde{u} = \prod_{j\in[B_{\mathsf{vk}}]} u_j$ as the aggregated signature. Again, the aggregated signature has size independent of B. The aggregated signature can then be verified by checking

$$\widetilde{u}^{\prod_{j\in[B_{\mathsf{vk}}]} E_j} \stackrel{?}{=} \prod_{j\in[B_{\mathsf{vk}}]} v_j^{\prod_{k\in[B_{\mathsf{vk}}]\setminus\{j\}} E_k},$$

which raises every individual verification relation to the common exponent $\prod_{j\in[B_{\mathsf{vk}}]} E_j$.

Following the security analysis of the bounded-query signature scheme, the above aggregate signature inherits the same bounded-query unforgeability guarantee in the *honest-key* setting, where all public keys in the system are generated honestly. For any bounded-query adversary, it is hard to forge an aggregated signature $\widetilde{u}$ involving B^* distinct verification keys $\{v_k^*\}_{k\in[B^*]}$, with corresponding signing seeds $\{u_k^*\}_{k\in[B^*]}$ and exponents $\{E_k^*\}_{k\in[B^*]}$, if there exists some $k^* \in [B^*]$ such that a message under $v_{k^*}^*$ was never queried for signing. Let Q_{vk} be the number of honestly generated verification keys. The reduction guesses the challenge key index $j^* \in [Q_{\mathsf{vk}}]$ and simulates all other signatures using the corresponding signing keys. After grouping the forgery by distinct verification keys, let

$k^* \in [B^*]$ be such that $v^*_{k^*} = v_{j^*}$. The batch verification relation then reduces to

$$\widetilde{u}^{\prod_{k\in[B^*]} E^*_k} = \prod_{k\in[B^*]} (v^*_k)^{\prod_{k'\in[B^*]\setminus\{k\}} E^*_{k'}}$$

$$\implies \quad (\widetilde{u} / \prod_{k\in[B^*]\setminus\{k^*\}} u^*_k)^{\prod_{k\in[B^*]} E^*_k} = \Big(\prod_{k\in[B^*]} (v^*_k)^{\prod_{k'\in[B^*]\setminus\{k\}} E^*_{k'}}\Big) \Big/ \Big(\prod_{k\in[B^*]\setminus\{k^*\}} (v^*_k)^{\prod_{k'\in[B^*]\setminus\{k\}} E^*_{k'}}\Big)$$

$$\implies \quad (\widetilde{u} / \prod_{k\in[B^*]\setminus\{k^*\}} u^*_k)^{\prod_{k\in[B^*]} E^*_k} = (v^*_{k^*})^{\prod_{k'\in[B^*]\setminus\{k^*\}} E^*_{k'}}$$

$$\implies \quad (\widetilde{u} / \prod_{k\in[B^*]\setminus\{k^*\}} u^*_k)^{E^*_{k^*}} = v^*_{k^*}.$$

In other words, $\widetilde{u} / \prod_{k\in[B^*]\setminus\{k^*\}} u^*_k$ is a valid signature for the challenge key $v^*_{k^*} = v_{j^*}$, thereby breaking the unforgeability of the underlying bounded-query signature scheme.

Removing the honest-key restriction is simple: We simply include in each public key a NIZKPoK of the corresponding secret key, thereby allowing the reduction to extract all the non-challenge secret keys and simulate the corresponding signatures correctly.

We stress that the setup, signing, and online verification times of the above scheme grow polynomially in Q, while aggregation and verification preprocessing additionally grow linearly in B. This is because the set S_m defined by each message m via the cover-free set system has size $O(Q\lambda)$, while the universe has size $O_\lambda(Q^2\lambda^2)$.

2.4 Bounded-query Secret-key Broadcast Encryption

We start by recalling the definition of a secret-key broadcast encryption scheme, which consists of the following algorithms. For a recipient set S, we write $B = |S|$ for the broadcast size.

- Setup: The setup algorithm generates a master public key and a master secret key for n users.
- KeyGen: For each user $i \in [n]$, the key generation algorithm generates the secret decryption key sk_i based on the master secret key.
- Enc: Given the master *secret* key, a message m, and a subset $S \subseteq [n]$ of users, the encryption algorithm generates a ciphertext ct that can be decrypted by every user in S using their individual secret key.
- Dec: Given a ciphertext ct, the corresponding subset $S \subseteq [n]$ of users, and the secret key sk_i for a user $i \in S$, the decryption algorithm outputs the decrypted message m.

For security, a challenge ciphertext ct for a set S should hide the message m as long as no user in S is corrupted. In particular, security should hold even against adversaries with multiple decryption keys for users outside S. Since the encryption is done using the master secret key, the adversary should additionally be given an encryption oracle that allows it to request arbitrary ciphertexts.

The core requirement of a (secret-key) broadcast encryption scheme is again *succinctness*: the ciphertext size should be sublinear in the encryption set size $|S|$. Note that the ciphertext size does *not* include the description of the set S itself, which must also be provided for decryption.

One-time broadcast encryption. Broadcast encryption and PPRGs share a similar "conditional disclosure" structure. A projected seed u_S reveals only bits within the set S, while a broadcast

ciphertext ct_S reveals the message only to the users in S. Indeed, a PPRG directly implies a one-time broadcast encryption scheme as follows:

- The setup algorithm prepares a PPRG seed u of output length n along with a one-time pad r.
- The secret key for user $i \in [n]$ is set to $b_i \oplus r$, where b_i is the i-th output bit of the PPRG seed u.
- To encrypt a message m for a subset $S \subseteq [n]$, the ciphertext consists of the padded message $c = m \oplus r$ along with the projected seed u_S.
- To decrypt, a user $i \in S$ uses the projected seed u_S and their secret key $b_i \oplus r$ to recover r and then decrypts c to obtain m.

The one-time security (i.e., the adversary sees only the challenge ciphertext) of the scheme follows directly from the pseudorandomness of the PPRG. Given a single u_S, the PPRG outputs b_i for $i \notin S$ remain pseudorandom. Thus, the secret key values $b_i \oplus r$ held by unauthorized users hide the one-time pad r, ensuring that the message m is hidden.

Bounded-query broadcast encryption. To extend the one-time broadcast encryption scheme to a bounded-query setting with query bound Q, we use $2\lambda Q$ message pads $\{p_j\}_{j\in[2\lambda Q]}$. For each user $i \in [n]$, the secret key consists of the slot keys $\{s_{i,j} = b_{i,j} \oplus p_j\}_{j\in[2\lambda Q]}$, where the bits $b_{i,j}$ are outputs of a PPRG of length $2\lambda Q n$, indexed by $(i, j) \in [n] \times [2\lambda Q]$. To encrypt a message m for a recipient set $S \subseteq [n]$, the algorithm samples a random set $T \subseteq [2\lambda Q]$ of size λ and defines the projection set $S_T = S \times T$ under this pair indexing. The ciphertext is

$$\left(m \oplus \bigoplus_{j\in T} p_j,\ u_{S_T},\ T \right).$$

Security follows from the observation that the Q queried ciphertexts select slots whose union has size at most $Q\lambda$. An independently sampled challenge set T^* contains a slot outside this union except with probability at most $2^{-\lambda}$. In this case, PPRG pseudorandomness ensures that the corresponding message pad p_j is pseudorandom to the adversary, and thus the message is hidden.

By leveraging the *adaptive* pseudorandomness of the underlying PPRG, the resulting bounded-query broadcast encryption scheme inherits adaptive security for free. Even if the adversary chooses the encryption sets S adaptively and receives the corresponding PPRG seeds u_{S_T}, all the unrevealed PPRG outputs remain pseudorandom, hiding the unused message pads for the challenge ciphertext.

On bounded-queryset security. Finally, we show that the above construction can be extended to achieve bounded-queryset security almost for free. In the bounded-queryset security game, the adversary can make an arbitrary number of encryption queries, but the number of distinct recipient sets appearing in those queries is bounded by Q. For each recipient set S, a PRF is used to derive fixed broadcast-encryption randomness r_S and an SKE key sk_S. The first ciphertext component deterministically encrypts sk_S to S, while the second component encrypts the actual message under sk_S using fresh randomness. Consequently, all queries for the same recipient set share the first component, reducing Q distinct recipient sets to Q underlying broadcast-encryption queries and allowing a direct reduction from bounded-queryset security to bounded-query security.

3 Preliminaries

Notations. In this work, we use $\mathbb{N}$ to denote positive integers, $\mathbb{Z}$ to denote integers, $\mathbb{Z}_N$ to denote the ring of integers modulo N, and $\mathbb{Z}_N^*$ to denote the multiplicative group of integers in $\mathbb{Z}_N$ that are coprime to N. For $n \in \mathbb{N}$, we use $[n]$ to denote the set $\{1, 2, \ldots, n\}$. We use $\mathbb{P}$ to denote the set of prime numbers, and $\mathbb{P}_\lambda$ to denote the set $\mathbb{P} \cap [2^\lambda]$. We use QR_N to denote the subgroup of $\mathbb{Z}_N^*$ consisting of quadratic residues of $\mathbb{Z}_N^*$.

3.1 Programmable Prime Sampler

Theorem 4 ([HW09, ABI+23], adapted)**.** *A programmable prime sampler consists of three PPT algorithms* $\mathsf{Samp}, \mathsf{Get}, \mathsf{Plant}$ *with the following syntax.*

- $\mathsf{Samp}(1^\lambda, 1^\ell) \to \mathsf{hk}$*: On input the security parameter* λ *and the input domain size* ℓ*, output a key* hk*.*
- $\mathsf{Get}(\mathsf{hk}, i) \to e_i$*: On input the key* hk *and an index* $i \in [\ell]$*, output a prime* $e_i \in \mathbb{P}_\lambda$*.*
- $\mathsf{Plant}(1^\lambda, 1^\ell, i \in [\ell], e_i \in \mathbb{P}_\lambda) \to \mathsf{hk}^*$*: On input a prime* e_i *and an index* i*, output a key* hk^* *where* $\mathsf{Get}(\mathsf{hk}^*, i) = e_i$*.*

Assuming the existence of PRGs, there exists a programmable prime sampler satisfying the following properties:

- **Efficiency.** *The key* $\mathsf{hk} \leftarrow \mathsf{Samp}(1^\lambda, 1^\ell)$ *has size* $\mathsf{poly}(\lambda) \cdot \log \ell$*, and the algorithm* Get *runs in time* $\mathsf{poly}(\lambda, \log \ell)$*.*
- **Correctness.** *The marginal distribution of* $\mathsf{Get}(\mathsf{Samp}(1^\lambda, 1^\ell), i)$ *is statistically close to uniform in* $\mathbb{P}_\lambda$ *for each* $i \in [\ell]$*. Furthermore, with overwhelming probability over the choice of* $\mathsf{hk} \leftarrow \mathsf{Samp}(1^\lambda, 1^\ell)$*, for all* $i, j \in [\ell]$ *with* $i \neq j$*,* $\mathsf{Get}(\mathsf{hk}, i) \neq \mathsf{Get}(\mathsf{hk}, j)$*.*
- **Programmability.** *For all* $i \in [\ell]$ *and* $e_i \in \mathbb{P}_\lambda$*, it holds that* $\mathsf{Get}(\mathsf{Plant}(1^\lambda, 1^\ell, i, e_i), i) = e_i$*. Furthermore, for randomly sampled* $e_i \leftarrow \mathbb{P}_\lambda$*, the output distribution of* $\mathsf{Plant}(1^\lambda, 1^\ell, i, e_i)$ *is statistically close to* $\mathsf{Samp}(1^\lambda, 1^\ell)$*.*

3.2 RSA Assumption

In this section, we recall the RSA assumption. For this work, we focus on RSA moduli with safe prime factors.

Definition 1 (Special Composite Modulus Sampler)**.** *Let* λ *be a security parameter. A special composite modulus sampler is an efficient algorithm* $\mathcal{S}$ *that takes as input the security parameter* 1^λ *and outputs* (N, p, q)*, where* $N = pq$ *and* p, q *are distinct* λ*-bit* safe primes*, i.e.,* $p = 2p' + 1$ *and* $q = 2q' + 1$ *for some primes* p', q'*.*

Lemma 1 (Generator of QR_N [ACJT00])**.** *Let* $N \leftarrow \mathcal{S}(1^\lambda)$ *be a special composite modulus. Then the group* QR_N *of the quadratic residues of* $\mathbb{Z}_N^*$ *is a cyclic group of size* $\phi(N)/4 = p'q'$*, and a generator* g_N *of* QR_N *can be efficiently computed from* N*.*

To support the PPRG construction, we use a variant of RSA, where the public exponent e is guaranteed to be a prime, and the base u is a quadratic residue modulo N. Similar to the argument in [ABI+23], this variant of RSA is implied by the standard RSA assumption, as prime exponents take a non-negligible fraction of all possible exponents, and quadratic residues take a non-negligible fraction of all possible bases.

Assumption 1 (RSA with Prime Exponents)**.** Let $\mathcal{S}$ be a special composite modulus sampler. We say the RSA assumption with prime exponents holds with respect to $\mathcal{S}$ if for all efficient adversaries $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\Pr\left[\mathcal{A}(1^\lambda, N, e, u^e \bmod N) = u : \begin{array}{c}(N,p,q) \leftarrow \mathcal{S}(1^\lambda) \\ e \leftarrow \mathbb{P}_{2\lambda}\,,\ u \leftarrow \mathsf{QR}_N\end{array}\right] = \mathsf{negl}(\lambda).$$

3.3 Goldreich-Levin Hardcore Bit

To achieve pseudorandomness of the PPRG from the search hardness of RSA, we rely on the classic Goldreich-Levin hardcore bit [GL89]. Here, we state a formulation from [HLR07] that applies to any computationally unpredictable random variable.

Definition 2 (Computational Unpredictability)**.** *Let $(X, Y) = \{(X_\lambda, Y_\lambda)\}_{\lambda\in\mathbb{N}}$ be an ensemble of joint distributions over pairs of values. We say that X is computationally unpredictable given Y if for all efficient (and possibly non-uniform) adversaries $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,*

$$\Pr[\mathcal{A}(1^\lambda, y) = x : (x, y) \leftarrow (X_\lambda, Y_\lambda)] = \mathsf{negl}(\lambda).$$

Theorem 5 (Goldreich-Levin [GL89, HLR07], adapted)**.** *Let $(X, Y) = \{(X_\lambda, Y_\lambda)\}_{\lambda\in\mathbb{N}}$ be an ensemble of joint distributions over pairs of values, where the support of X_λ is $\{0,1\}^{\rho(\lambda)}$. Let $\mathsf{hc}(x, r) = \langle x, r\rangle$ be the Goldreich-Levin extractor. Suppose that X is computationally unpredictable given Y. Then, for all efficient (and possibly non-uniform) adversaries $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,*

$$\left|\Pr\left[\mathcal{A}(1^\lambda, r, y, \mathsf{hc}(x, r)) = 1\right] - \Pr\left[\mathcal{A}(1^\lambda, r, y, b) = 1\right]\right| = \mathsf{negl}(\lambda),$$

where $(x, y) \leftarrow (X_\lambda, Y_\lambda)$, $r \leftarrow \{0,1\}^{\rho(\lambda)}$, and $b \leftarrow \{0,1\}$.

3.4 Cover-Free Set System

In this section, we recall the definition of cover-free set systems [EFF85], along with the efficient construction from [KRS99].

Definition 3 (k-cover-free set system)**.** *A set system $(\mathcal{S}, U)$ with the universe U and a family of subsets $\mathcal{S} = \{S \subseteq U\}$ is called k-cover-free if for any $k+1$ distinct sets $S_0, S_1, \ldots, S_k \in \mathcal{S}$, it holds that $S_0 \nsubseteq S_1 \cup S_2 \cup \cdots \cup S_k$.*

We define the following cover-free generator to capture the efficient constructions of cover-free set systems.

Definition 4 (Cover-free generator)**.** *A cover-free generator* $(\mathsf{Gen}, \mathsf{Embed})$ *consists of two efficient algorithms of the following syntax.*

- $\mathsf{Gen}(1^\lambda, 1^k, M) \to (\mathsf{hk}, s, \ell)$*: On input the security parameter λ, the cover-free parameter k, and the number of sets M, output a key hk, a set size $s \in \mathbb{N}$, and a universe size $\ell \in \mathbb{N}$.*
- $\mathsf{Embed}(\mathsf{hk}, i) \to S_i$*: On input the key hk and an index $i \in [M]$, deterministically output a set $S_i \subseteq [\ell]$ of size s.*

We say that $(\mathsf{Gen}, \mathsf{Embed})$ *is a cover-free generator if for all unbounded adversaries* $\mathcal{A}$*, there exists a negligible function* $\mathsf{negl}(\cdot)$ *such that for all* $\lambda \in \mathbb{N}$*,* $k = k(\lambda) = \mathsf{poly}(\lambda)$*, and* $M = M(\lambda) = 2^{O(\lambda)}$*,*

$$\Pr\left[\begin{array}{l} i_0 \notin \{i_t\}_{t\in[k]} \wedge S_{i_0} \subseteq \cup_{t\in[k]} S_{i_t} \ : \end{array} \begin{array}{rl} (\mathsf{hk}, s, \ell) & \leftarrow \mathsf{Gen}(1^\lambda, 1^k, M), \\ (i_0, i_1, \ldots, i_k) & \leftarrow \mathcal{A}(\mathsf{hk}, s, \ell), \\ \forall t \in [k] \cup \{0\}, \quad S_{i_t} & \leftarrow \mathsf{Embed}(\mathsf{hk}, i_t) \end{array}\right] = \mathsf{negl}(\lambda).$$

Lemma 2 ([KRS99, Theorem 3], adapted)**.** *There exists a cover-free generator with* $\mathsf{hk} = \perp$*,* $s = O_\lambda(k \log M)$*,* $\ell = O_\lambda(k^2 \log^2 M)$*, the algorithm* Gen *is dummy, and the algorithm* Embed *runs in time* $O_\lambda(k \log^2 M)$*.*

3.5 Simulation-Extractable NIZKPoK

In this section, we recall the notion of simulation-extractable non-interactive zero-knowledge proof of knowledge (NIZKPoK) [DDO+01].

Definition 5 (NIZKPoK)**.** *A NIZKPoK scheme for an NP relation* R *consists of the following algorithms.*

- $\mathsf{Setup}(1^\lambda) \to \mathsf{crs}$*: On input the security parameter* λ*, output a common reference string* crs*.*
- $\mathsf{TSetup}(1^\lambda) \to (\mathsf{crs}, \mathsf{td})$*: On input the security parameter* λ*, output a common reference string* crs *and a trapdoor* td*.*
- $\mathsf{Prove}(\mathsf{crs}, x, w) \to \pi$*: On input the common reference string* crs*, a statement* x*, and a witness* w *such that* $R(x, w) = 1$*, output a proof* π*.*
- $\mathsf{Verify}(\mathsf{crs}, x, \pi) \to b$*: On input the common reference string* crs*, a statement* x*, and a proof* π*, output* 1 *if* π *is a valid proof for* x*, and* 0 *otherwise.*
- $\mathsf{Sim}(\mathsf{crs}, \mathsf{td}, x) \to \pi$*: On input the common reference string* crs*, a trapdoor* td*, and a statement* x*, output a simulated proof* π*.*
- $\mathsf{Ext}(\mathsf{crs}, \mathsf{td}, x, \pi) \to w/\perp$*: On input the common reference string* crs*, a trapdoor* td*, a statement* x*, and a proof* π*, output a witness* w *or* $\perp$ *for extraction failure.*

A NIZKPoK scheme should satisfy the following property:

- **Completeness***: For all* $\lambda \in \mathbb{N}$*, all statements* x *and witnesses* w *such that* $R(x, w) = 1$*, it holds that*

$$\Pr\left[\mathsf{Verify}(\mathsf{crs}, x, \mathsf{Prove}(\mathsf{crs}, x, w)) = 1\right] = 1.$$

The zero-knowledge and simulation-extractability properties of a NIZKPoK scheme are defined as follows.

Definition 6 (Zero-Knowledge)**.** *Let* λ *be the security parameter,* $\mathcal{A}$ *be a PPT adversary, and* $\beta \in \{0, 1\}$*. The zero-knowledge experiment* $\mathsf{Exp}^\beta_{\mathsf{ZK}}(1^\lambda, \mathcal{A})$ *is defined as follows.*

- *If* $\beta = 0$*, the challenger runs* $\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda)$*. If* $\beta = 1$*, the challenger runs* $(\mathsf{crs}, \mathsf{td}) \leftarrow \mathsf{TSetup}(1^\lambda)$*. The challenger gives* crs *to* $\mathcal{A}$*.*
- $\mathcal{A}$ *now makes adaptive queries of the form* (x, w)*, where* x *is a statement and* w *is a witness. If* $R(x, w) = 0$*, the challenger responds with* $\perp$*. Otherwise,*

- *If $\beta = 0$, the challenger responds with $\pi \leftarrow \mathsf{Prove}(\mathsf{crs}, x, w)$.*
- *If $\beta = 1$, the challenger responds with $\pi \leftarrow \mathsf{Sim}(\mathsf{crs}, \mathsf{td}, x)$.*

- *Finally, the adversary $\mathcal{A}$ outputs a bit b, which is the output of the experiment.*

We say the NIZKPoK scheme is zero-knowledge if for all PPT adversaries $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that

$$\left|\Pr\left[\mathsf{Exp}^0_{\mathsf{ZK}}(1^\lambda, \mathcal{A}) = 1\right] - \Pr\left[\mathsf{Exp}^1_{\mathsf{ZK}}(1^\lambda, \mathcal{A}) = 1\right]\right| \leq \mathsf{negl}(\lambda).$$

Definition 7 (Simulation-Extractability)**.** *Let λ be the security parameter and $\mathcal{A}$ be a PPT adversary. The simulation-extractability experiment $\mathsf{Exp}_{\mathsf{PoK}}(1^\lambda, \mathcal{A})$ is defined as follows.*

- *The challenger runs $(\mathsf{crs}, \mathsf{td}) \leftarrow \mathsf{TSetup}(1^\lambda)$, and gives crs to $\mathcal{A}$.*
- *$\mathcal{A}$ now makes adaptive queries, each consisting of an arbitrary statement x. The challenger responds with $\pi \leftarrow \mathsf{Sim}(\mathsf{crs}, \mathsf{td}, x)$, and stores the pair (x, π) in a set Q.*
- *After $\mathcal{A}$ finishes making queries, it outputs a statement x^* and a proof π^*.*
- *The experiment outputs 1 if $\mathsf{Verify}(\mathsf{crs}, x^*, \pi^*) = 1$, $(x^*, \pi^*) \notin Q$, and the extracted witness $w^* \leftarrow \mathsf{Ext}(\mathsf{crs}, \mathsf{td}, x^*, \pi^*)$ satisfies $R(x^*, w^*) = 0$. Otherwise, the experiment outputs 0.*

We say the NIZKPoK scheme is simulation-extractable if for all PPT adversaries $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that

$$\Pr\left[\mathsf{Exp}_{\mathsf{PoK}}(1^\lambda, \mathcal{A}) = 1\right] \leq \mathsf{negl}(\lambda).$$

NIZKs for NP are known from the RSA assumption [FLS90], and a generic transformation from NIZK to simulation-extractable NIZKPoK is also known from the RSA assumption [DDO+01].

4 Reusable Projective PRG

4.1 Definition

In this section, we define multi-hop reusable projective pseudorandom generator (PPRG) schemes adapted from [ABI+23], which serve as the main building blocks for our applications. Compared to the original definition in [ABI+23], we separate the setup algorithm and the key generation algorithm to highlight the reusability of the setup, and we require the scheme to support public seed projection from arbitrary projected seeds, as opposed to requiring the master seed for each projection.

Definition 8 (Multi-hop Reusable Projective PRG)**.** *A multi-hop reusable projective pseudorandom generator (PPRG) scheme is a tuple of algorithms $(\mathsf{Setup}, \mathsf{SeedGen}, \mathsf{Project}, \mathsf{Eval})$ defined as follows.*

- $\mathsf{Setup}(1^\lambda, 1^\ell)$*: On input the security parameter λ and the output length ℓ, the setup algorithm outputs a common reference string crs.*
- $\mathsf{SeedGen}(\mathsf{crs})$*: On input the common reference string crs, the seed generation algorithm outputs a master seed u. We define $u = u_{[\ell]}$ as the projected seed for the full set $[\ell]$.*

- $\mathsf{Project}(\mathsf{crs}, S, u_S, S')$: *On input the common reference string* crs*, a projected seed* u_S *for a set* S*, and a subset* $S' \subseteq S$*, the projection algorithm outputs a projected seed* $u_{S'}$ *for* S'*.*

- $\mathsf{Eval}(\mathsf{crs}, S, u_S, i)$: *On input the common reference string* crs*, a projected seed* u_S *for a subset* S*, and an index* $i \in S$*, the evaluation algorithm outputs a pseudorandom bit* b_i*.*

A PPRG scheme should satisfy the following properties:

- **(Multi-hop) Correctness.** *For all* $\lambda, \ell \in \mathbb{N}$*, all* crs *generated by* $\mathsf{Setup}(1^\lambda, 1^\ell)$*, all master seeds* $u \leftarrow \mathsf{SeedGen}(\mathsf{crs})$*, all sequences of subsets* $S_1 \subseteq S_2 \subseteq \cdots \subseteq S_k \subseteq [\ell]$*, and all indices* $i \in S_1$*, it holds that*

$$\mathsf{Eval}(\mathsf{crs}, S_1, u_{S_1}, i) = \mathsf{Eval}(\mathsf{crs}, [\ell], u, i),$$

  where $u_{S_j} = \mathsf{Project}(\mathsf{crs}, S_{j+1}, u_{S_{j+1}}, S_j)$ *for each* $j \in [k-1]$ *and* $u_{S_k} = \mathsf{Project}(\mathsf{crs}, [\ell], u, S_k)$*.*

- **Succinctness.** *The projected seed* u_S *has size* $\mathsf{poly}(\lambda, \log \ell)$*. Since* ℓ *is encoded in unary, by restricting* $\ell < 2^\lambda$ *we can equivalently require the size of* u_S *to be* $\mathsf{poly}(\lambda)$*.*

For the security of PPRGs, we define the adaptive pseudorandomness property, guaranteeing the pseudorandomness of the evaluation output on any unqueried index given adaptive queries on projected seeds. The definition is a natural combination of the robustness definition in [ABI+23], which captures static security against multiple seed queries, along with the adaptive pseudorandomness definition in [HWW25b], which captures adaptive security against a single seed query.

Definition 9 (Adaptive Pseudorandomness)**.** *For a security parameter* $\lambda \in \mathbb{N}$*, an adversary* $\mathcal{A}$*, and a bit* $\beta \in \{0,1\}$*, define the following security experiment* $\mathsf{Exp}^{(\beta)}_{\mathsf{prg}}(1^\lambda, \mathcal{A})$*:*

- *Given the input* 1^λ*, the adversary* $\mathcal{A}$ *provides the output length* 1^ℓ*.*

- *The challenger runs* $\mathsf{Setup}(1^\lambda, 1^\ell)$ *to generate* crs *and* $\mathsf{SeedGen}(\mathsf{crs})$ *to generate* u*. The challenger then sends* crs *to the adversary* $\mathcal{A}$*.*

- *The challenger computes the real evaluation outputs* $c_i^0 = \mathsf{Eval}(\mathsf{crs}, [\ell], u, i)$ *for each* $i \in [\ell]$*, and samples random bits* $c_i^1 \leftarrow \{0,1\}$ *for each* $i \in [\ell]$*. It also initializes a seed set* $T = \emptyset$ *and an index set* $I = \emptyset$*.*

- *The adversary* $\mathcal{A}$ *adaptively makes the following types of queries:*

  - ***Projected seed query:*** *The adversary* $\mathcal{A}$ *submits a set* S*. If* $S \cap I \neq \emptyset$*, the challenger responds with* $\perp$*. Otherwise, the challenger responds with the projected seed* $u_S = \mathsf{Project}(\mathsf{crs}, [\ell], u, S)$*. The challenger updates the seed set* $T = T \cup S$*.*

  - ***Evaluation query:*** *The adversary* $\mathcal{A}$ *submits an index* $i \in [\ell]$*. If* $i \in T$*, the challenger responds with* $\perp$*. Otherwise, the challenger responds with the bit* c_i^β*. The challenger updates the index set* $I = I \cup \{i\}$*.*

- *The adversary* $\mathcal{A}$ *outputs a bit* b*.*

We say a PPRG is adaptively pseudorandom if for all efficient adversaries $\mathcal{A}$*, there exists a negligible function* $\mathsf{negl}(\cdot)$ *such that for all* $\lambda \in \mathbb{N}$*,*

$$\left| \Pr\left[\mathsf{Exp}^{(0)}_{\mathsf{prg}}(1^\lambda, \mathcal{A}) = 1\right] - \Pr\left[\mathsf{Exp}^{(1)}_{\mathsf{prg}}(1^\lambda, \mathcal{A}) = 1\right] \right| = \mathsf{negl}(\lambda).$$

4.2 Multi-hop Reusable Projective PRG from RSA

In this section, we give a construction of a multi-hop reusable projective PRG based on the RSA assumption. The construction is largely based on [ABI+23], with some adaptation to the multi-hop setting in Definition 8, and with seeds restricted to QR_N.

Construction 1 (Multi-hop Reusable Projective PRG)**.** Let λ be the security parameter, and let ℓ be a polynomial in λ defining the output length of the PPRG. Let $(\mathsf{Samp}, \mathsf{Get}, \mathsf{Plant})$ be a programmable prime sampler with $k(\lambda, \ell)$-bit key size and λ-bit prime outputs, as defined in Theorem 4. Let $\mathcal{S}$ be a special composite modulus sampler, as defined in Definition 1, such that the RSA assumption with prime exponents (Assumption 1) holds with respect to $\mathcal{S}$. Let $\mathsf{hc}(x, r) = \langle x, r \rangle$ be the Goldreich-Levin hardcore bit, as defined in Theorem 5, which takes in an integer $x \in \mathbb{Z}_N^*$ and parses it as a bit string in $\{0,1\}^{\log N}$.

We construct a projective PRG $(\mathsf{Setup}, \mathsf{SeedGen}, \mathsf{Project}, \mathsf{Eval})$ as follows.

- $\mathsf{Setup}(1^\lambda, 1^\ell)$: On input the security parameter λ and the output length ℓ, the setup algorithm samples
  - RSA modulus $(N, p, q) \leftarrow \mathcal{S}(1^\lambda)$, where p, q are safe primes, along with a generator g of QR_N.
  - Hash key $\mathsf{hk} \leftarrow \mathsf{Samp}(1^{2\lambda}, 1^\ell)$, which defines prime exponents
    $$e_i = \mathsf{Get}(\mathsf{hk}, i) \in \mathbb{P}_{2\lambda} \text{ for each } i \in [\ell].$$
    With overwhelming probability, all e_i are distinct 2λ-bit primes coprime to $\phi(N)$.
  - Goldreich-Levin extractor randomness $r \leftarrow \{0,1\}^{\log N}$.

  The algorithm outputs the common reference string $\mathsf{crs} = (N, g, \mathsf{hk}, r, \ell)$.
- $\mathsf{SeedGen}(\mathsf{crs})$: On input the common reference string $\mathsf{crs} = (N, g, \mathsf{hk}, r, \ell)$, sample a random exponent $a \leftarrow [2^\lambda N]$, and compute the QR_N element $u = g^a$ as the master seed. For every subset $S \subseteq [\ell]$, the master seed u uniquely defines the projected seed
  $$u_S = u^{\prod_{i \in [\ell] \setminus S} e_i} \in \mathsf{QR}_N.$$
  The algorithm outputs the master seed $u = u_{[\ell]}$.
- $\mathsf{Project}(\mathsf{crs}, S, u_S, S')$: On input the common reference string $\mathsf{crs} = (N, g, \mathsf{hk}, r, \ell)$, a projected seed u_S for a set $S \subseteq [\ell]$, and a subset $S' \subseteq S$, compute $e_i = \mathsf{Get}(\mathsf{hk}, i)$ for each $i \in S \setminus S'$. The algorithm computes and outputs the projected seed
  $$u_{S'} = u_S^{\prod_{i \in S \setminus S'} e_i} \in \mathsf{QR}_N.$$
  By construction, honest projected seeds u_S for any subset $S \subseteq [\ell]$ are deterministic given the master seed u and the set S. Therefore, the seed is consistent regardless of the projection path.
- $\mathsf{Eval}(\mathsf{crs}, S, u_S, i)$: On input the common reference string $\mathsf{crs} = (N, g, \mathsf{hk}, r, \ell)$, a projected seed u_S for a subset $S \subseteq [\ell]$, and an index $i \in S$, compute the singleton projected seed $u_{\{i\}} = \mathsf{Project}(\mathsf{crs}, S, u_S, \{i\})$, and output the pseudorandom bit $b_i = \mathsf{hc}(u_{\{i\}}, r)$.

The correctness, succinctness, and pseudorandomness of the above construction are established in [ABI+23] and its follow-up work [HWW25b], with minor adaptations.

Lemma 3 ([ABI+23, HWW25b]). *Construction 1 is correct, succinct, and adaptively pseudorandom under the RSA assumption (Assumption 1).*

Sketch. The correctness and succinctness of the construction follow the same arguments as in [ABI+23, HWW25b]. While our construction restricts the seed space to QR_N instead of $\mathbb{Z}_N^*$, the correctness relation remains unchanged.

For adaptive security, since QR_N is cyclic with order less than N (lemma 1), u is statistically close to a uniformly random element in QR_N. The security of the construction therefore follows from the same arguments as in [ABI+23, HWW25b], reducing to the RSA assumption in QR_N given in Assumption 1. One minor gap is that the adaptive pseudorandomness defined in [HWW25b] only states the security guarantee for a single projected seed query. Nevertheless, in the security proof of [HWW25b], each hybrid step changes the behavior of one index i, and the projected seed for any subset not containing i is computable by the reduction in the hybrid. The proof hence works for arbitrarily many seed queries from the adversary. □

Lemma 4 (Efficiency of Construction 1). *Construction 1 admits the following efficiency guarantees.*

- *The running time of* Setup *is* $\mathsf{poly}(\lambda) \cdot \ell$*, and the size of the common reference string* crs *is* $\mathsf{poly}(\lambda) \cdot \log \ell$*.*
- *The running time of* $\mathsf{SeedGen}$ *is* $\mathsf{poly}(\lambda)$*, and the size of the master seed* u *is* $\mathsf{poly}(\lambda)$*.*
- *The running time of* $\mathsf{Project}$ *is* $\mathsf{poly}(\lambda) \cdot |S \setminus S'|$*, and the size of the projected seed* $u_{S'}$ *is* $\mathsf{poly}(\lambda)$*.*
- *The running time of* Eval *is* $\mathsf{poly}(\lambda) \cdot |S|$*.*

Proof. First, in Setup, the elements N, r have size $\mathsf{poly}(\lambda)$ and can be generated in $\mathsf{poly}(\lambda)$ time. Following Theorem 4, the hash key hk has size $k(2\lambda, \ell) = \mathsf{poly}(\lambda) \cdot O(\log \ell)$, and can be generated in $\mathsf{poly}(\lambda) \cdot O(\ell)$ time. This concludes the efficiency of Setup and the size of crs.

$\mathsf{SeedGen}$ simply samples a random element in QR_N, which can be done in $\mathsf{poly}(\lambda)$ time (Lemma 1). The output seed u has size $\mathsf{poly}(\lambda)$.

For $\mathsf{Project}$, the algorithm computes the difference set $S \setminus S'$, obtains the corresponding exponents e_i for each $i \in S \setminus S'$, where each e_i takes $\mathsf{poly}(\lambda)$ time to compute, and then raises u_S to the product of the exponents, which can be done in $\mathsf{poly}(\lambda) \cdot |S \setminus S'|$ time. This overall takes $\mathsf{poly}(\lambda) \cdot |S \setminus S'|$ time. The output seed is again an element in QR_N, which has size $\mathsf{poly}(\lambda)$.

For Eval, the projection takes time $\mathsf{poly}(\lambda) \cdot |S|$, and the Goldreich-Levin hardcore bit can be computed in $\mathsf{poly}(\lambda)$ time. So the overall running time is $\mathsf{poly}(\lambda) \cdot |S|$. □

4.3 Additional Algebraic Properties of Construction 1

We further identify the following properties of the construction crucial for our constructions.

Efficient seed generation with preprocessing. In our application, we often need to generate projected seeds as part of the key generation / encryption process. One efficiency bottleneck of Construction 1 is that the generation of projected seeds for a set S needs time linear in the size of the complement $[\ell] \setminus S$. We show that with appropriate preprocessing, one can push the cost of generating projected seeds to setup time.

Lemma 5 (Rerandomization of Preprocessed Seeds). *For any set* $S \subseteq [\ell]$*, let* $g_S = \mathsf{Project}(\mathsf{crs}, [\ell], g, S)$ *be the projected seed corresponding to* S *with respect to the (canonical) master seed* g*. Then, for any master seed* $u = g^a$*, it holds that*

$$\mathsf{Project}(\mathsf{crs}, [\ell], u, S) = g_S^a,$$

where the right-hand side expression can be computed in $\mathsf{poly}(\lambda)$ *time given* g_S.

From the above identity, we can define the following rerandomization procedure that uses preprocessing to efficiently generate a fresh seed and its projected seeds. Define the rerandomization algorithm ReRand *as follows.*

- $\mathsf{ReRand}(\mathsf{crs}, \{g_{S_i}\})$*: On input the common reference string* $\mathsf{crs} = (N, g, \mathsf{hk}, r, \ell)$ *and a collection of preprocessed projected seeds* $\{g_{S_i}\}$*, where* $g_{S_i} = \mathsf{Project}(\mathsf{crs}, [\ell], g, S_i)$*, the algorithm samples a random exponent* $a \leftarrow [2^\lambda N]$ *and outputs the rerandomized master seed and projected seeds* $(u = g^a, \{u_{S_i} = g_{S_i}^a\})$.

The distribution of the rerandomized master seed and projected seeds is identical to the distribution of a freshly generated master seed and the corresponding projected seeds, and the circuit ReRand *has size depending only on the number of preprocessed projected seeds, independent of* $[\ell]$.

Seed homomorphism. Next, we identify the homomorphic property between independent seeds.

Lemma 6 (Seed homomorphism)**.** *For all* $\lambda \in \mathbb{N}$*, all* crs *generated by* $\mathsf{Setup}(1^\lambda, 1^\ell)$*, all seeds* u, u' *in the seed space, and all subsets* $S \subseteq [\ell]$*, it holds that*

$$\mathsf{Project}(\mathsf{crs}, [\ell], u \cdot u', S) = \mathsf{Project}(\mathsf{crs}, [\ell], u, S) \cdot \mathsf{Project}(\mathsf{crs}, [\ell], u', S),$$

where the multiplication is taken modulo N.

Proof. It is clear that for any pair of seeds $u, u' \in \mathsf{QR}_N$, the projected seeds $u_S = u^{\prod_{i \in [\ell] \setminus S} e_i}$ and $u'_S = (u')^{\prod_{i \in [\ell] \setminus S} e_i}$ are homomorphic under multiplication. □

Seed Union Next, we identify algebraic properties among multiple projected seeds based on the same master seed. Given projected seeds for two subsets, one can efficiently reconstruct the projected seed for the union of the two subsets. This follows from the classic trick by Shamir [Sha83].

Lemma 7 (Union of Projected Seeds)**.** *Define the algorithm* Union *as follows.*

- $\mathsf{Union}(\mathsf{crs}, S_1, u_{S_1}, S_2, u_{S_2})$*: On input the common reference string* $\mathsf{crs} = (N, g, \mathsf{hk}, r, \ell)$ *and two projected seeds* u_{S_1}, u_{S_2} *for subsets* $S_1, S_2 \subseteq [\ell]$*, the algorithm computes and outputs a projected seed* u_S *for* $S = S_1 \cup S_2$ *as follows:*
  - *If* $S_1 \subseteq S_2$*, output* $u_S = u_{S_2}$*. Similarly, if* $S_2 \subseteq S_1$*, output* $u_S = u_{S_1}$.
  - *Otherwise, let* $T_1 = S_2 \setminus S_1$ *and* $T_2 = S_1 \setminus S_2$*. Let* $E_1 = \prod_{i \in T_1} e_i$ *and* $E_2 = \prod_{i \in T_2} e_i$*. If* $\gcd(E_1, E_2) \neq 1$*, output* $\perp$*. Otherwise, compute* x, y *such that* $x \cdot E_1 + y \cdot E_2 = 1$.
  - *Output*
$$u_S = u_{S_1}^x \cdot u_{S_2}^y.$$

For all $\lambda \in \mathbb{N}$*, all but a negligible fraction of* crs *generated by* $\mathsf{Setup}(1^\lambda, 1^\ell)$*, all seeds* u *generated by* $\mathsf{SeedGen}(\mathsf{crs})$*, and all subsets* $S_1, S_2 \subseteq [\ell]$*, it holds that*

$$\mathsf{Union}(\mathsf{crs}, S_1, \mathsf{Project}(\mathsf{crs}, [\ell], u, S_1), S_2, \mathsf{Project}(\mathsf{crs}, [\ell], u, S_2)) = \mathsf{Project}(\mathsf{crs}, [\ell], u, S_1 \cup S_2).$$

Furthermore, the running time of Union *is* $\mathsf{poly}(\lambda) \cdot |S_1 \cup S_2|$.

We also extend the definition of Union *to compute the projected seed for the union of* k *subsets* $S_1, \ldots, S_k$ *by iteratively applying* Union *over two sets, which runs in time* $\mathsf{poly}(\lambda) \cdot |\cup_{j \in [k]} S_j| \cdot k$.

Proof. By definition, the projected seeds are computed by $u_S = u^{\prod_{i\in[\ell]\setminus S} e_i}$ for any subset S, where u is the master seed. Therefore, for any two subsets S_1, S_2 and their union $S = S_1 \cup S_2$, it holds that

$$u_{S_1} = u_S^{E_1}, u_{S_2} = u_S^{E_2},$$

where $E_1 = \prod_{i\in S\setminus S_1} e_i$ and $E_2 = \prod_{i\in S\setminus S_2} e_i$. Note that with overwhelming probability over hk, all e_i are distinct primes. Combined with the fact that $S \setminus S_1$ and $S \setminus S_2$ are disjoint, we have $\gcd(E_1, E_2) = 1$. Then, by leveraging Bézout's identity, we can efficiently compute x, y where $x \cdot E_1 + y \cdot E_2 = 1$, implying that

$$u_{S_1}^x \cdot u_{S_2}^y = u_S^{x\cdot E_1 + y\cdot E_2} = u_S.$$

This concludes the correctness of the algorithm Union.

Finally, for efficiency, the Bézout coefficients x, y can be computed in time $O(\log \max(E_1, E_2))$, which is bounded by $\mathsf{poly}(\lambda)\cdot|S_1 \cup S_2|$ since each e_i is a 2λ-bit prime. Furthermore, x, y are bounded by $\max(E_1, E_2)$. Therefore, computing u_S requires $\mathsf{poly}(\lambda)\cdot|S_1 \cup S_2|$ time. □

Public seed verification. Next, we observe that, since $\mathsf{Project}$ is deterministic and consistent among multi-hop compositions, one can efficiently verify the consistency of two projected seeds $u_S, u_{S'}$ for $S' \subseteq S$ by checking whether $\mathsf{Project}(\mathsf{crs}, S, u_S, S') = u_{S'}$. Therefore, by revealing the projected seed of the empty set $v = u_\emptyset = u^{\prod_{i\in[\ell]} e_i}$, which we call the verification seed, one can verify the validity of any projected seed u_S by checking $\mathsf{Project}(\mathsf{crs}, S, u_S, \emptyset) = u_S^{\prod_{i\in S} e_i} \stackrel{?}{=} v$. Moreover, since the verification is just a simple exponentiation operation, one can further batch the verification of multiple projected seeds into verifying the exponentiation of the product of the projected seeds. This leads to the following batch verification algorithm $\mathsf{BatchVerify}$.

Definition 10. *Define the batch verification algorithm* $\mathsf{BatchVerify}$ *to be the following two-step algorithm.*

- $\mathsf{BatchVerify}(\mathsf{crs}, \{S_j, v_j\}_{j\in[B]}, \widetilde{u})$*: On input the common reference string* $\mathsf{crs} = (N, g, \mathsf{hk}, r, \ell)$*, a sequence of subsets* $S_1, \ldots, S_B \subseteq [\ell]$*, distinct verification seeds* $v_1, \ldots, v_B$*, and a batched projected seed* $\widetilde{u}$*, the algorithm verifies whether the batched projected seed* $\widetilde{u}$ *is the product of valid projected seeds* u_{j,S_j} *such that* $\mathsf{Project}(\mathsf{crs}, S_j, u_{j,S_j}, \emptyset) = v_j$ *for each* $j \in [B]$*.*

  We break the verification into two steps: (i) a preprocessing step that takes time linear in B *but independent of the batched projected seed* $\widetilde{u}$*, and outputs a batched verification seed* $\widetilde{v}$*, and (ii) an online* $\widetilde{u}$ *verification step that takes the set union of all subsets, and verifies with respect to the batched verification seed* $\widetilde{v}$*.*

  - $\mathsf{PreBatchVerify}(\mathsf{crs}, \{S_j, v_j\}_{j\in[B]})$*: The preprocessing step sets* $S = \cup_{j\in[B]} S_j$*, and computes the batched verification seed* $\widetilde{v} \in \mathsf{QR}_N$ *by*

    $$\widetilde{v} = \prod_{j\in[q]} v_j^{E_{S\setminus S_j}}, \text{ where } E_{S\setminus S_j} = \prod_{i\in S\setminus S_j} e_i.$$

    It outputs $(S, \widetilde{v})$*.*
  - $\mathsf{OnlineBatchVerify}(\mathsf{crs}, S, \widetilde{v}, \widetilde{u})$*: The online verification step checks whether*

    $$\widetilde{u}^{E_S} \stackrel{?}{=} \widetilde{v}, \text{ where } E_S = \prod_{i\in S} e_i.$$

For batch size $B = 1$, the preprocessing step is trivial and outputs $\widetilde{v} = v_1$, and the verification reduces to the base verification relation for the single projected seed u_{1,S_1}:

$$\mathsf{Project}(\mathsf{crs}, S_1, \widetilde{u}, \emptyset) = \widetilde{u}^{\prod_{i \in S_1} e_i} \stackrel{?}{=} v_1.$$

It is straightforward to verify that the batch verification algorithm accepts the correct batched projected seed $\widetilde{u} = \prod_{j \in [B]} u_{j,S_j}$.

Lemma 8 (Correctness of Batch Verification)**.** *For all $\lambda \in \mathbb{N}$, all* crs *generated by* $\mathsf{Setup}(1^\lambda, 1^\ell)$*, all (u_j, v_j) satisfying $v_j = \mathsf{Project}(\mathsf{crs}, [\ell], u_j, \emptyset)$, all subsets $S_1, \ldots, S_B \subseteq [\ell]$ and the corresponding projected seeds $u_{j,S_j} = \mathsf{Project}(\mathsf{crs}, [\ell], u_j, S_j)$ for each $j \in [B]$, it holds that*

$$\mathsf{BatchVerify}(\mathsf{crs}, \{S_j, v_j\}_{j \in [B]}, \widetilde{u} = \prod_{j \in [B]} u_{j,S_j}) = 1.$$

Proof. By the definition of the projected seeds, we know that each u_{j,S_j} must satisfy the relation

$$(u_{j,S_j})^{\prod_{i \in S_j} e_i} = v_j.$$

Therefore, by setting $S = \cup_{j \in [B]} S_j$ and $E_S = \prod_{i \in S} e_i$, we have

$$\widetilde{u}^{E_S} = \prod_{j \in [B]} \left((u_{j,S_j})^{E_{S_j}}\right)^{E_{S \setminus S_j}} = \prod_{j \in [B]} v_j^{E_{S \setminus S_j}} = \widetilde{v},$$

which completes the proof. □

For efficiency, observe that the online verification step only requires one exponentiation of the batched projected seed $\widetilde{u}$ with respect to the product of all primes e_i for $i \in S$. Thus, the running time is only linear in $|S|$ and independent of the batch size B. For cases where the batch size B is much larger than the output length ℓ, or when the batch consists of many sets with a small union, the online runtime can be significantly smaller than the full verification algorithm.

Lemma 9 (Efficiency of Batch Verification)**.** *The running time of* PreBatchVerify *is* $\mathsf{poly}(\lambda) \cdot B \cdot |S|$*. The output size of* PreBatchVerify *and the running time of* OnlineBatchVerify *are* $\mathsf{poly}(\lambda) \cdot |S|$*, where S is the union of all subsets in the batch.*

For soundness, we argue that the batched seed $\widetilde{u}$ passing verification is unique given the verification seeds v_j and the subsets S_j. Furthermore, if all verification seeds are generated honestly and independently, each seed $\widetilde{u}_{j,S_j}$ in the batch is extractable.

Lemma 10 (Soundness of Batch Verification)**.** *For all $\lambda \in \mathbb{N}$, for an overwhelming fraction of* crs *generated by* $\mathsf{Setup}(1^\lambda, 1^\ell)$*, for all seed pairs (u_j, v_j) such that $v_j = \mathsf{Project}(\mathsf{crs}, [\ell], u_j, \emptyset)$, and for any subsets $S_1, \ldots, S_B \subseteq [\ell]$, it holds that*

$$\mathsf{BatchVerify}(\mathsf{crs}, \{S_j, v_j\}_{j \in [B]}, \widetilde{u}) = 1 \implies \widetilde{u} = \prod_{j \in [B]} \mathsf{Project}(\mathsf{crs}, [\ell], u_j, S_j).$$

As a direct implication, for any index $j^* \in [B]$, given the master seeds u_j for all $j \in [B] \setminus \{j^*\}$,

$$\mathsf{BatchVerify}(\mathsf{crs}, \{S_j, v_j\}_{j \in [B]}, \widetilde{u}) = 1 \implies \mathsf{Project}(\mathsf{crs}, [\ell], u_{j^*}, S_{j^*}) = \widetilde{u} \cdot \prod_{j \in [B] \setminus \{j^*\}} \mathsf{Project}(\mathsf{crs}, [\ell], u_j, S_j)^{-1}.$$

Proof. First, we know that with overwhelming probability over the choice of $\mathsf{hk} \leftarrow \mathsf{Samp}(1^\lambda, 1^\ell)$, all primes $e_i = \mathsf{Get}(\mathsf{hk}, i)$ are distinct and coprime to $\phi(N)$. In such cases, $E_S = \prod_{i \in S} e_i$ is coprime to $\phi(N)$, and the equation $\widetilde{u}^{E_S} = \widetilde{v}$ has a unique solution $\widetilde{u} = \widetilde{v}^{E_S^{-1} \bmod \phi(N)}$. By the definition of $\widetilde{v}$, we thus have

$$\mathsf{BatchVerify}(\mathsf{crs}, \{S_j, v_j\}_{j\in[B]}, \widetilde{u}) = 1 \implies \widetilde{u}^{E_S} = \widetilde{v} = \prod_{j\in[B]} v_j^{E_{S\setminus S_j}}$$

$$\implies \widetilde{u} = \prod_{j\in[B]} v_j^{E_{S\setminus S_j}\cdot E_S^{-1} \bmod \phi(N)} = \prod_{j\in[B]} u_j^{E_{[\ell]}\cdot E_S^{-1}\cdot E_{S\setminus S_j}} = \prod_{j\in[B]} u_j^{E_{[\ell]\setminus S_j}} = \prod_{j\in[B]} \mathsf{Project}(\mathsf{crs}, [\ell], u_j, S_j),$$

which completes the proof. □

Remark 1 (Extractability implies (Semi-malicious) Unforgeability)**.** *We note that the extractability guarantee (Lemma 10) along with the pseudorandomness guarantee (Lemma 3) implies hardness of finding $\widetilde{u}$ against* semi-malicious *verification seeds v_j. For an adversary given a challenge public seed v, some projected seed u_S with respect to v, finding a verifying batch seed $\widetilde{u}$ for the set $\{T \not\subseteq S, v\} \cup \{S_j, v_j\}_{j\in[B]}$ for some "honest" verification seeds v_j implies the ability to find u_T with respect to v given u_S, which breaks the pseudorandomness as u_T perfectly predicts the PPRG evaluation on indices in $T \setminus S$.*

However, the extraction of u_T crucially relies on the knowledge of the master seeds u_j for all $j \in [B] \setminus \{j^*\}$, thereby posing the requirement that all v_j must be generated with the knowledge of u_j. Indeed, for fully malicious v_j, it is easy to find a verifying batch seed simply by setting $v_1 = v^{-1}$, $S_1 = T$, and $\widetilde{u} = 1$.

Batch evaluation. Finally, we provide an optimization for evaluating multiple output bits of the projective PRG efficiently. As discussed in Lemma 4, given a projected seed u_S, evaluating the PRG on index $i \in S$ takes time linear in $|S|$. In other words, the naive way to evaluate every output bit in S would take time $O(|S|^2)$. We show that this can be improved to $O(|S| \log |S|)$ with a simple divide-and-conquer approach, leveraging the power of multi-hop projection.

Lemma 11. *Define the batch evaluation algorithm* BatchEval *as follows.*

- $\mathsf{BatchEval}(\mathsf{crs}, u_S, S)$*: On input the common reference string* crs*, a projected seed u_S for a subset $S \subseteq [\ell]$, and the set S, the algorithm proceeds as follows:*
  1. *If $|S| = 1$, output (i, u_S), where i is the single element in S.*
  2. *Otherwise, split S into two disjoint subsets S_0 and S_1, with $|S_0| = \lfloor |S|/2 \rfloor$.*
  3. *Compute the projected seeds u_{S_0} and u_{S_1} from u_S via*
  $$u_{S_0} = \mathsf{Project}(\mathsf{crs}, S, u_S, S_0), \quad u_{S_1} = \mathsf{Project}(\mathsf{crs}, S, u_S, S_1).$$
  4. *Recursively call* $\mathsf{BatchEval}(\mathsf{crs}, u_{S_0}, S_0)$ *and* $\mathsf{BatchEval}(\mathsf{crs}, u_{S_1}, S_1)$ *to obtain the evaluations on S_0 and S_1, and output the concatenation of the results.*

The running time of BatchEval *is $O(|S| \log |S|)$.*

Proof. Note that the two projection operations in BatchEval run in time $\Theta(|S|)$. The recurrence relation of the running time $T(|S|)$ of BatchEval is therefore $T(|S|) = 2T(|S|/2) + \Theta(|S|)$, which solves to $T(|S|) = \Theta(|S| \log |S|)$. □

Note that the output of $\mathsf{BatchEval}$ consists of pairs $(i, u_{\{i\}})$ for each $i \in S$. Each single-element projected seed $u_{\{i\}}$ can then be used to efficiently evaluate the PRG on the corresponding index i in time $\mathsf{poly}(\lambda)$.

Remark 2. *In Lemma 11, one can additionally record and output the intermediate projected seeds u_S during the recursion. In settings where $S = [2^d]$, one can obtain all projected seeds u_s corresponding to every prefix $s \in \{0,1\}^{\leq d}$ in time $O(|S| \log |S|)$.*

5 Keyless Batch Encryption and Batched Threshold Encryption

In this section, we construct a batched threshold encryption scheme from RSA and DCR. Our construction follows the (implicit) recipe of [BFOQ25], which combines a keyless batch encryption (KBE) scheme with a homomorphic threshold encryption scheme and a NIZK for the language of valid ciphertexts. In Section 5.1, we formally define KBE, which is a simplified variant of BTE highlighting only the batching functionality, implicitly constructed by [BFOQ25]. In Section 5.2, we provide a new construction of a KBE scheme from the RSA-based PPRG in Construction 1, which supports a polynomial-size index space. In Section 5.3, we provide the syntax for batched threshold encryption, with its security definition deferred to Appendix A.1. We also provide additional discussion on the index space in Section 5.3. Finally, in Section 5.4, we follow the recipe of [BFOQ25], combining the KBE scheme with keys in QR_N, a homomorphic threshold encryption scheme for QR_N, and a NIZK for the language of valid ciphertexts, to obtain a batched threshold encryption scheme. The construction of the homomorphic threshold encryption scheme for QR_N is deferred to Section 8.

5.1 Keyless Batch Encryption

We now formalize the notion of keyless batch encryption. In a KBE scheme, a stateless encryption algorithm encrypts messages under a public setup and an index. The algorithm provides a ciphertext along with a short "decryption key". These keys can be batched into a succinct batch decryption key, which can be used to decrypt all ciphertexts in the batch, as long as the ciphertexts are encrypted under distinct indices.

Definition 11 (Keyless batch encryption scheme)**.** *A KBE scheme consists of the following PPT algorithms.*

- $\mathsf{Setup}(1^\lambda, 1^B) \to \mathsf{pp}$*: On input the security parameter λ and the maximal batch size B defining the index space $[B]$, output the public parameters* pp*.*
- $\mathsf{Enc}(\mathsf{pp}, m, i) \to (\mathsf{ct}, h)$*: On input the public parameters* pp*, a message m, and an index $i \in [B]$, output a ciphertext* ct *and a decryption key h.*
- $\mathsf{BatchKey}(\mathsf{pp}, \{i, h_i\}_{i \in S}) \to \mathsf{bk}$*: On input the public parameters* pp*, a set of decryption keys $\{h_i\}_{i \in S}$ corresponding to distinct indices in S, where $|S| \leq B$, output a batch decryption key* bk*.*
- $\mathsf{BatchDec}(\mathsf{pp}, \mathsf{bk}, \{\mathsf{ct}_i\}_{i \in S}) \to \{m_i\}_{i \in S}$*: On input the public parameters* pp*, a batch decryption key* bk*, and a set of ciphertexts $\{\mathsf{ct}_i\}_{i \in S}$, output the list of messages $\{m_i\}_{i \in S}$.*

The KBE scheme should satisfy the following properties:

- **Correctness**: *For all $\lambda \in \mathbb{N}$, all $B \le 2^\lambda$, all sets $S \subseteq [B]$ of distinct indices, and all messages $\{m_i\}_{i\in S}$, it holds that*

$$\Pr\left[\mathsf{BatchDec}(\mathsf{pp}, \mathsf{bk}, \{\mathsf{ct}_i\}_{i\in S}) = \{m_i\}_{i\in S} \;\middle|\; \begin{array}{ll} & \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, 1^B) \\ (\mathsf{ct}_i, h_i) & \leftarrow \mathsf{Enc}(\mathsf{pp}, m_i, i) \quad i \in S \\ & \mathsf{bk} \leftarrow \mathsf{BatchKey}(\mathsf{pp}, \{i, h_i\}_{i\in S}) \end{array}\right] = 1.$$

- **Succinctness**: *The batch decryption key* bk *has size* $\mathsf{poly}(\lambda)$.

- **Security**: *For all polynomial $B = B(\lambda)$, for all PPT adversaries $\mathcal{A}$, there exists a negligible function* $\mathsf{negl}(\cdot)$ *such that for all m_0, m_1 in the message space and all $i \in [B]$, it holds that*

$$|\Pr[\mathcal{A}(\mathsf{pp}, \mathsf{ct}_0) = 1] - \Pr[\mathcal{A}(\mathsf{pp}, \mathsf{ct}_1) = 1]| \le \mathsf{negl}(\lambda),$$

  where $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, 1^B)$, $(\mathsf{ct}_b, h_b) \leftarrow \mathsf{Enc}(\mathsf{pp}, m_b, i)$ *for* $b \in \{0, 1\}$.

Note that Definition 11 is defined with index space of size B, which is polynomially bounded and is equal to the maximum batch size supported by the scheme. This leads to a batched threshold encryption scheme supporting a polynomial number of indices. As discussed in [BFOQ25], one can expand the index space to be exponential with a logarithmic-size blowup in the batch decryption key or the ciphertext size. However, such a transformation also comes with caveats in the security guarantees. See Section 5.3 for more discussion.

5.2 Constructing Keyless Batch Encryption from RSA

Notation. Throughout the remainder of this subsection, we adopt the following notation to describe subsets defined by the bit representation of indices.

- For an index $i \in [B]$, we define the prefix strings $i[:k]$ for $k \in [d = \lceil \log B \rceil]$ as the first k bits of the binary representation of i. We also define the co-prefix string $\overline{i[:k]}$, which has identical bits to $i[:k]$ except that the k-th bit is flipped.

- For any bit string $s \in \{0, 1\}$ with length at most d, we define the prefix set

$$\mathsf{Pfx}(s) = \{i \in [B] : i[:|s|] = s\}.$$

  For an index $i \in [B]$, we also define the prefix set of each prefix and co-prefix string of i as

$$\mathsf{Pfx}_k(i) := \mathsf{Pfx}(i[:k]), \quad \overline{\mathsf{Pfx}_k}(i) := \mathsf{Pfx}(\overline{i[:k]}), \quad k \in [d].$$

  By definition, $\mathsf{Pfx}_k(i)$ and $\overline{\mathsf{Pfx}_k}(i)$ form a partition of $\mathsf{Pfx}_{k-1}(i)$, where $\mathsf{Pfx}_0(i)$ is defined as the full set $[B]$.

Construction 2 (Keyless Batch Encryption from RSA-based PPRG)**.** Let $(\mathsf{Setup}, \mathsf{SeedGen}, \mathsf{Project}, \mathsf{Eval})$ be the algorithms of the RSA-based projective PRG from Construction 1. We construct the KBE scheme as follows.

- $\mathsf{Setup}(1^\lambda, 1^B)$: The algorithm samples the PPRG common reference string with the index size B being the output length, $\mathsf{crs} \leftarrow \mathsf{PPRG.Setup}(1^\lambda, 1^B)$ and outputs $\mathsf{pp} = \mathsf{crs}$.

- $\mathsf{Enc}(\mathsf{pp}, m \in \{0,1\}, i)$ can be separated into two steps, a public preprocessing procedure dependent only on the public parameters and the index, and a randomized online encryption procedure dependent on the message.
  - $\mathsf{PreEnc}(\mathsf{pp}, i \in [B])$: Parse $\mathsf{crs} = (N, g, \ldots)$. The algorithm uses g as a canonical seed to compute projected seeds for all prefix sets $\mathsf{Pfx}(s)$ for $s \in \{0,1\}^{\leq d}$ as follows:
    * Starting with the empty string ϵ, the algorithm sets $g_{\mathsf{Pfx}(\epsilon)} = g_{[B]} = g$.
    * For $k = 0, \ldots, d-1$, given the projected seed of $\mathsf{Pfx}(s)$ with $|s| = k$, the algorithm computes the projected seeds for $\mathsf{Pfx}(s||0)$ and $\mathsf{Pfx}(s||1)$ using the projection algorithm
    $$g_{\mathsf{Pfx}(s||b)} = \mathsf{Project}(\mathsf{crs}, \mathsf{Pfx}(s), g_{\mathsf{Pfx}(s)}, \mathsf{Pfx}(s||b)), \quad b \in \{0,1\}.$$

    Overall, the algorithm computes expanded public parameters
    $$\mathsf{expp} = \{g_{\mathsf{Pfx}(s)}\}_{s \in \{0,1\}^{\leq d}}.$$
    For an index i, the algorithm selects the leaf node $\mathsf{Pfx}_d(i)$ corresponding to the singleton set $\{i\}$, along with all the co-path nodes $\overline{\mathsf{Pfx}_k}(i)$ for $k \in [d]$. The algorithm outputs
    $$\mathsf{expp}_i := \{g_{\mathsf{Pfx}_d(i)}, g_{\overline{\mathsf{Pfx}_1}(i)}, \ldots, g_{\overline{\mathsf{Pfx}_d}(i)}\}.$$
  - $\mathsf{OnlineEnc}(\mathsf{pp}, \mathsf{expp}_i, m \in \{0,1\}, i)$: The online encryption algorithm takes as input the expanded public parameters expp_i for index i, a message $m \in \{0,1\}$, and an index $i \in [B]$. The algorithm first rerandomizes the preprocessed seeds for the leaf and co-path nodes as follows.
    $$\left(u = g^a, \{u_{\mathsf{Pfx}_d(i)}, u_{\overline{\mathsf{Pfx}_1}(i)}, \ldots, u_{\overline{\mathsf{Pfx}_d}(i)}\}\right) \leftarrow \mathsf{ReRand}\left(\mathsf{crs}, \{g_{\mathsf{Pfx}_d(i)}, g_{\overline{\mathsf{Pfx}_1}(i)}, \ldots, g_{\overline{\mathsf{Pfx}_d}(i)}\}\right).$$
    It then computes the PPRG evaluation on index i by $b_i \leftarrow \mathsf{Eval}(\mathsf{crs}, \mathsf{Pfx}_d(i) = \{i\}, u_{\mathsf{Pfx}_d(i)}, i)$, and outputs
    $$\left(\mathsf{ct} = (c = b_i \oplus m, \{u_{\overline{\mathsf{Pfx}_k}(i)}\}_{k \in [d]}), \quad h = u\right).$$
- $\mathsf{BatchKey}(\mathsf{pp}, \{i, h_i\}_{i \in S})$ takes as input the public parameters pp and a set of decryption keys $\{h_i = u_i\}_{i \in S}$ corresponding to distinct indices in S, where $|S| \leq B$. It outputs the batch decryption key $\mathsf{bk} = \prod_{i \in S} u_i$, where the multiplication is defined in QR_N.
- $\mathsf{BatchDec}(\mathsf{pp}, \mathsf{bk}, \{\mathsf{ct}_i = (c_i, \{u_{i,\overline{\mathsf{Pfx}_k}(i)}\}_{k \in [d]})\}_{i \in S})$ decrypts the batch of ciphertexts as follows:
  - View $\mathsf{bk} = \widetilde{u}$ as a PPRG seed, and batch evaluate all the singleton projected seeds
  $$\{\widetilde{u}_{\{i\}}\}_{i \in [B]} = \mathsf{BatchEval}(\mathsf{crs}, [B], \widetilde{u}).$$
  - For every bit string $s \in \{0,1\}^{\leq d}$, collect and homomorphically batch all the projected seeds from ciphertexts with index i having s as a co-prefix. Formally, for each s, the algorithm computes the homomorphic seed
  $$w_s = \prod_{j \in S \wedge s = \overline{j[:|s|]}} u_{j,\overline{\mathsf{Pfx}_{|s|}(j)}}.$$
  - For every bit string $s \in \{0,1\}^{\leq d}$, define the seed t_s recursively based on the aggregated projected seeds w_s as follows.

* For $k = 1$, define $t_0 = w_0, t_1 = w_1$.
* For prefix $s = s'||b$ with length $k > 1$ and $b \in \{0,1\}$, define

$$t_s = w_s \cdot \mathsf{Project}(\mathsf{crs}, \mathsf{Pfx}(s'), t_{s'}, \mathsf{Pfx}(s)).$$

– Finally, for each index $i \in S$, compute the decryption seed

$$u_{i,\{i\}} = \widetilde{u}_{\{i\}}/t_i$$

and compute

$$m_i = c_i \oplus \mathsf{Eval}(\mathsf{crs}, \{i\}, u_{i,\{i\}}, i).$$

Following the properties of the RSA-based PPRG from Construction 1, it is immediate that Construction 2 is a secure KBE scheme. We provide formal analysis as follows.

Theorem 6. *Construction 2 satisfies the following efficiency guarantees:*

- *The size of the public parameters* pp *is* $\mathsf{poly}(\lambda)$*, and the running time of* Setup *is* $B \cdot \mathsf{poly}(\lambda)$*.*
- *The size of the ciphertext* ct *is* $\log B \cdot \mathsf{poly}(\lambda)$*, and the size of the decryption key* h *is* $\mathsf{poly}(\lambda)$*. The running time of* Enc *consists of a preprocessing step taking* $B \log B \cdot \mathsf{poly}(\lambda)$ *time and an online encryption step taking* $\log B \cdot \mathsf{poly}(\lambda)$ *time.*
- *The size of the batch decryption key* bk *is* $\mathsf{poly}(\lambda)$*, and the running time of* $\mathsf{BatchKey}$ *is* $|S| \cdot \mathsf{poly}(\lambda)$*.*
- *The running time of* $\mathsf{BatchDec}$ *is* $B \log B \cdot \mathsf{poly}(\lambda)$ *for* $|S|$ *ciphertexts.*

Proof. The efficiency guarantees follow from the efficiency of the RSA-based PPRG given in Lemmas 4, 5 and 11. In particular,

- The Setup algorithm is identical to the PPRG Setup algorithm, which has output size $\mathsf{poly}(\lambda)$ and running time $B \cdot \mathsf{poly}(\lambda)$.
- The size of the ciphertext is 1 bit plus $d = O(\log B)$ projected seeds, while the size of the decryption key is just one master seed. The PreEnc algorithm follows the same tree structure as the batch evaluation procedure of the PPRG, which has running time $B \log B \cdot \mathsf{poly}(\lambda)$. The $\mathsf{OnlineEnc}$ algorithm is dominated by the rerandomization of $d + 1$ projected seeds, taking time $\log B \cdot \mathsf{poly}(\lambda)$. All remaining operations run in time $\mathsf{poly}(\lambda)$.
- The batch decryption key is just a single element in QR_N, which has size $\mathsf{poly}(\lambda)$. The running time of $\mathsf{BatchKey}$ is the time to compute the product of $|S|$ elements in QR_N, which is $|S| \cdot \mathsf{poly}(\lambda)$.
- In $\mathsf{BatchDec}$, we can expand the running time of each step as follows:
  - The first $\mathsf{BatchEval}$ call runs in time $B \log B \cdot \mathsf{poly}(\lambda)$.
  - Since there are $2B$ distinct prefix strings $s \in \{0,1\}^{\leq d}$ and each of the $|S|$ ciphertexts in the batch provides $\log B$ projected seeds corresponding to its co-prefixes, the total runtime of the aggregation step is bounded by $(2B + |S| \log B) \cdot \mathsf{poly}(\lambda) \leq B \log B \cdot \mathsf{poly}(\lambda)$.
  - During the tree-propagation step, at each level $k \leq d$, there are 2^k seeds t to be computed, each taking time $2^{d-k} \cdot \mathsf{poly}(\lambda)$. Summing over all levels, the total running time of the tree-propagation step is $B \log B \cdot \mathsf{poly}(\lambda)$.

– The final decryption step involves an evaluation of the PPRG on the singleton projected seed corresponding to the index i for each $i \in S$, which takes time $|S| \cdot \mathsf{poly}(\lambda)$.

Overall, the running time of $\mathsf{BatchDec}$ is $B \log B \cdot \mathsf{poly}(\lambda)$, regardless of the batch size $|S|$.

□

Theorem 7. *Construction 2 is correct.*

Proof. First, following Lemma 6, we know that the honest batch key $\mathsf{bk} = \widetilde{u} = \prod_{j \in S} u_j$ satisfies the following equation when projected to index i:

$$\begin{aligned}\mathsf{Project}(\mathsf{crs}, [B], \widetilde{u}, \{i\}) &= \prod_{j \in S} \mathsf{Project}(\mathsf{crs}, [B], u_j, \{i\}) \\ &= u_{i,\{i\}} \cdot \prod_{j \in S \setminus \{i\}} \mathsf{Project}(\mathsf{crs}, [B] \setminus \{j\}, u_{j,[B]\setminus\{j\}}, \{i\}) = \prod_{j \in S} u_{j,\{i\}}.\end{aligned}$$

To recover the message pad $u_{i,\{i\}}$, we need to argue that $t[i] = \prod_{j \in S \setminus \{i\}} u_{j,\{i\}}$. To achieve this, we show the following invariant of $t[s]$ subsuming the case $s = i$:

$$t[s] = \prod_{j \in S \setminus \mathsf{Pfx}(s)} u_{j,\mathsf{Pfx}(s)}.$$

The proof of the invariant follows from an induction over the prefix length k.

- For $k = 1$, $s \in \{0,1\}$, $t_s = w_s$ is defined by

  $$w_s = \prod_{j \in S \wedge s = \overline{j[:1]}} u_{j,\mathsf{Pfx}(s)} = \prod_{j \in S \wedge j \notin \mathsf{Pfx}(s)} u_{j,\mathsf{Pfx}(s)},$$

  satisfying the invariant.

- Assuming the invariant holds for all prefixes of length $k-1$, consider a prefix $s = s'||b$ of length k. We have

  $$w_s = \prod_{j \in S \wedge s = \overline{j[:k]}} u_{j,\mathsf{Pfx}(s)} = \prod_{j \in S \cap (\mathsf{Pfx}(s') \setminus \mathsf{Pfx}(s))} u_{j,\mathsf{Pfx}(s)}.$$

  Combined with the induction hypothesis for s' where $t_{s'} = \prod_{j \in S \setminus \mathsf{Pfx}(s')} u_{j,\mathsf{Pfx}(s')}$, we get

  $$t_s = t_{s'} \cdot w_s = \prod_{j \in S \setminus \mathsf{Pfx}(s)} u_{j,\mathsf{Pfx}(s)}.$$

  completing the induction proof.

□

Theorem 8. *Assuming the RSA assumption with prime exponents (Assumption 1), Construction 2 is secure.*

Proof. First, by the rerandomization property of the PPRG (Lemma 5), the decryption key $h = u$ is identical to the honest $\mathsf{SeedGen}$ output. With this, the security of Construction 2 follows immediately from the pseudorandomness of the RSA-based PPRG. In particular, by Lemma 3, for any adversary $\mathcal{A}$ and any index i, it holds that

$$\left| \begin{array}{l} \Pr\left[\mathcal{A}(\mathsf{pp}, \{u_{\mathsf{Pfx}(\overline{i[:k]})}\}_{k\in[d]}, b = \mathsf{Eval}(\mathsf{crs}, [B], u, i)) = 1\right] \\ - \Pr\left[\mathcal{A}(\mathsf{pp}, \{u_{\mathsf{Pfx}(\overline{i[:k]})}\}_{k\in[d]}, b \leftarrow \{0,1\}) = 1\right] \end{array} \right| \leq \mathsf{negl}(\lambda),$$

as the co-prefix sets $\{\mathsf{Pfx}(\overline{i[:k]})\}_{k\in[d]}$ do not contain the index i. Therefore, b is a pseudorandom one-time pad of the message m, which implies the security of the scheme. □

Remark 3 (On Encryption Preprocessing)**.** *We note that while the overall encryption time in Construction 2 is (quasi)linear in B, the expanded public parameter expp_i can be reused across multiple encryptions for the same index i, thereby improving the amortized encryption time to just $\log B \cdot \mathsf{poly}(\lambda)$. One can as well store the full expanded public parameter expp to support encryption for any index i, at the cost of $O(B \log B)$ online storage.*

Alternatively, expp can also be computed during setup as part of the public parameter. This gives an alternative construction with the same setup time, an $O(B \log B)$-size pp, and $O(\log B)$ encryption time.

5.3 Batched Threshold Encryption

Definition 12 (Batched Threshold Encryption)**.** *An epochless batched threshold encryption scheme (BTE) consists of the following algorithms.*

- $\mathsf{Setup}(1^\lambda, 1^B, \ell) \to \mathsf{pp}$*: On input the security parameter λ, the maximal batch size B, and the index space size ℓ, output the public parameters pp.*
- $\mathsf{KeyGen}(\mathsf{pp}, 1^n, t) \to (\mathsf{pk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n)$*: On input the public parameters pp, the number of decrypting parties n, and the decryption threshold t, output the public key pk and the secret keys $\mathsf{sk}_1, \ldots, \mathsf{sk}_n$ for the n parties.*
- $\mathsf{Enc}(\mathsf{pp}, \mathsf{pk}, j, m) \to \mathsf{ct}$*: On input the public parameters pp, the public key pk, an index $j \in [\ell]$, and a message m, output a ciphertext ct.*
- $\mathsf{PartialDec}(\mathsf{pp}, \mathsf{sk}_i, \{j, \mathsf{ct}_j\}_{j\in S}) \to d_i/\perp$*: On input the public parameters pp, a secret key sk_i for party i, and a set of ciphertexts $\{\mathsf{ct}_j\}_{j\in S}$ corresponding to distinct indices in S, where $|S| \leq B$, output a succinct partial decryption key d_i or $\perp$ for decryption failure.*
- $\mathsf{Combine}(\mathsf{pp}, \mathsf{pk}, \{d_i\}_{i\in T}, \{j, \mathsf{ct}_j\}_{j\in S}) \to \{m_j\}_{j\in S}/\perp$*: On input the public parameters pp, the public key pk, a set of partial decryption keys $\{d_i\}_{i\in T}$ with $|T| \geq t$, and a set of ciphertexts $\{\mathsf{ct}_j\}_{j\in S}$ corresponding to partial decryptions, output the decrypted messages $\{m_j\}_{j\in S}$ or $\perp$.*

The BTE scheme should satisfy the following properties:

- **Correctness***: For all $\lambda, B, \ell, n, t \in \mathbb{N}$, all sets $S \subseteq [\ell]$ of distinct indices of size $|S| \leq B$, all sets $T \subseteq [n]$ of parties of size $|T| \geq t$, and all messages $\{m_j\}_{j\in S}$, it holds that*

$$\mathsf{Combine}(\mathsf{pp}, \mathsf{pk}, \{\mathsf{PartialDec}(\mathsf{pp}, \mathsf{sk}_i, \{j, \mathsf{ct}_j\}_{j\in S})\}_{i\in T}, \{j, \mathsf{ct}_j\}_{j\in S}) = \{m_j\}_{j\in S},$$

where $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, 1^B, \ell)$, $(\mathsf{pk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n) \leftarrow \mathsf{KeyGen}(\mathsf{pp}, 1^n, t)$, and $\mathsf{ct}_j \leftarrow \mathsf{Enc}(\mathsf{pp}, \mathsf{pk}, j, m_j)$ for all $j \in S$.

- **Succinctness**: *The size of each partial decryption key d_i is* $\mathsf{poly}(\lambda, \log B, \log n, \log \ell)$.

[BFOQ25] defines the following two security requirements for BTE schemes: the *IND-CCA* security, which guarantees that the ciphertexts hide the messages even given access to the batch decryption oracle; and the *rogue ciphertext* security, which guarantees that a malformed ciphertext does not affect the decryption correctness of other honest ciphertexts in a batch. We defer the formal definition of the two notions to Appendix A.1.

Subbatch Techniques. Following the BEAT-MEV construction, we will construct a BTE scheme with $B = \ell$, i.e., the maximal batch size is identical to the index space size, and the index space size is polynomially bounded. Furthermore, the running time of the setup, encryption, and decryption algorithms all grow with the size of the index space. Ideally, one would want to decouple the batch size B from the size of the index space, so that the running time of the scheme only grows with the batch size, but not the index space. To achieve this, BEAT-MEV [BFOQ25] provides two versions of *subbatch techniques* for index space expansion with a logarithmic-size blowup in either the ciphertext size or the batch decryption key size.

- (Subbatch Technique I with key size loss.) Given a batched threshold encryption scheme with batch size B and index space $[B]$, build a new scheme with index space $[n]$, where each ciphertext with index $i \in [n]$ is mapped to the base index $(i \bmod B) \in [B]$ using the original scheme. On a batch of B ciphertexts, *with overwhelming probability over random indices*, the number of index collisions in $[B]$ is bounded by $\alpha = O(\log B)$. Therefore, one can partition the batch into α subbatches of distinct indices and compute batched keys for each subbatch, achieving a scheme with an $O(\log B)$ blowup in the size of the batched key.
- (Subbatch Technique II with ciphertext size loss.) Given a batched threshold encryption scheme with batch size B and index space $[B]$, build a new scheme with index space $[n = \binom{B}{\alpha}]$, where $\alpha = O(\log B)$, i.e., each index $i \in [n]$ can be represented as a subset $S_i \subseteq [B]$ of size α, and a ciphertext with index i consists of α ciphertexts of the underlying scheme using base indices from S_i. On a batch of B ciphertexts, *with overwhelming probability over random indices*, Hall's marriage theorem [Hal35] guarantees that there exists a representing index $j_i \in S_i$ for each ciphertext such that all representing indices j_i are distinct. Therefore, one can directly apply the original scheme on the batch of ciphertexts, achieving a scheme with an $O(\log B)$ blowup in the size of the ciphertext.
- It was also shown in [ABD+26] that one can implement a fine-grained mixture of the two subbatch transformations using cuckoo hashing.

The subbatch techniques are especially suited for *indexless* batched threshold encryption, where the ciphertexts are not associated with any fixed index. In this case, one can simply sample a random index for each ciphertext as part of the encryption randomness.

The biggest disadvantage of the subbatch techniques is that they sacrifice the *worst-case* correctness/succinctness guarantee of the scheme. When all the indices are maliciously chosen to create many collisions, the above batching either results in an $O(B)$-size batch decryption key or a decryption failure. This issue is identified in [BLT25] as the *succinctness vulnerability*. Recently, it was pointed out in [BNRT26, ADG+26, Pol26] that the succinctness vulnerability could also be a security hazard in certain contexts. As the decryption committee would typically only choose a batch of ciphertexts that can be succinctly decrypted, the adversary could create targeted collisions in the ciphertext pool, avoiding the inclusion of honest ciphertexts carrying a targeted index, leading to a *censorship attack*.

In this work, we mainly focus on constructing a batched threshold encryption scheme with a polynomial-size index space that achieves succinctness in both the batch size and the number of decryption parties. We leave the construction of a batched threshold encryption scheme with an exponential-size index space from factoring-based assumptions as an interesting open question for future work.

5.4 Constructing Batched Threshold Encryption

In this section, we formally describe how a BTE scheme can be built by combining appropriate KBE schemes, HTE schemes, and NIZK schemes. The construction and security proof are essentially a reinterpretation of the BEAT-MEV framework [BFOQ25], with some minor adaptations to our KBE construction.

Construction 3 (Batched Threshold Encryption)**.** Consider the following components:

- Let $\mathsf{KBE} = (\mathsf{Setup}, \mathsf{Enc}, \mathsf{BatchKey}, \mathsf{BatchDec})$ be a keyless batch encryption scheme (Section 5.1) with efficient online encryption, i.e., $\mathsf{Enc} = (\mathsf{PreEnc}, \mathsf{OnlineEnc})$ and $\mathsf{OnlineEnc}$ has circuit size independent of the batch size B (ignoring log factors).
- Let $\mathsf{HTE} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Eval}, \mathsf{PartialDec}, \mathsf{Combine})$ be a homomorphic threshold encryption scheme (Section 8.1) supporting the key batching operation $\mathsf{KBE.BatchKey}$. Additionally, we require that the scheme is simulatable (Definition 22), that the randomness of an evaluated ciphertext can be efficiently derived from the input encryption randomness, and that Enc has circuit size independent of the number of decryption parties n.
- Let $\mathsf{NIZK} = (\mathsf{Setup}, \mathsf{TSetup}, \mathsf{Prove}, \mathsf{Verify}, \mathsf{Sim}, \mathsf{Ext})$ be a simulation-extractable NIZKPoK (Section 3.5) for the ciphertext validity relation

$$\mathcal{R} = \left\{ \begin{array}{l} \big(x = (\mathsf{pp}_{\mathsf{KBE}}, \mathsf{expp}_{j,\mathsf{KBE}}, \mathsf{pp}_{\mathsf{HTE}}, \mathsf{pk}_{\mathsf{HTE}}, j, \mathsf{ct}_{\mathsf{KBE}}, \mathsf{ct}_{\mathsf{HTE}}), w = (m, h, r_{\mathsf{KBE}}, r_{\mathsf{HTE}})\big) \\ \quad : \begin{array}{l} \mathsf{KBE.OnlineEnc}(\mathsf{pp}_{\mathsf{KBE}}, \mathsf{expp}_{j,\mathsf{KBE}}, m, j; r_{\mathsf{KBE}}) = (\mathsf{ct}_{\mathsf{KBE}}, h) \\ \wedge \mathsf{HTE.Enc}(\mathsf{pp}_{\mathsf{HTE}}, \mathsf{pk}_{\mathsf{HTE}}, h; r_{\mathsf{HTE}}) = \mathsf{ct}_{\mathsf{HTE}} \end{array} \end{array} \right\}.$$

We construct a batched threshold encryption scheme supporting a polynomial-size index space. As discussed in Section 5.3, we focus on constructions with $B = \ell$, and defer the index space expansion with subbatch techniques.

- $\mathsf{Setup}(1^\lambda, 1^B, \ell)$: The algorithm runs the setup algorithms of each primitive to get $\mathsf{pp}_{\mathsf{KBE}} \leftarrow \mathsf{KBE.Setup}(1^\lambda, 1^\ell)$, $\mathsf{pp}_{\mathsf{HTE}} \leftarrow \mathsf{HTE.Setup}(1^\lambda)$, and $\mathsf{crs} \leftarrow \mathsf{NIZK.Setup}(1^\lambda)$. It outputs the public parameters $\mathsf{pp} = (\mathsf{pp}_{\mathsf{KBE}}, \mathsf{pp}_{\mathsf{HTE}}, \mathsf{crs})$[8].
- $\mathsf{KeyGen}(\mathsf{pp}, 1^n, t)$: The algorithm runs the key generation algorithm of the homomorphic threshold encryption scheme to get $(\mathsf{pk}_{\mathsf{HTE}}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n) \leftarrow \mathsf{HTE.KeyGen}(\mathsf{pp}_{\mathsf{HTE}}, 1^n, t)$. It outputs the public key $\mathsf{pk} = \mathsf{pk}_{\mathsf{HTE}}$ and the secret keys $\mathsf{sk}_1, \ldots, \mathsf{sk}_n$ for the n parties.
- $\mathsf{Enc}(\mathsf{pp}, \mathsf{pk}, j, m)$: The algorithm computes the expanded public parameters for index j and runs the online encryption algorithm of the KBE scheme using randomness r_{KBE}:

$$\begin{aligned} &\mathsf{expp}_{j,\mathsf{KBE}} \leftarrow \mathsf{KBE.PreEnc}(\mathsf{pp}_{\mathsf{KBE}}, j), \\ &(\mathsf{ct}_{\mathsf{KBE}}, h) \leftarrow \mathsf{KBE.OnlineEnc}(\mathsf{pp}_{\mathsf{KBE}}, \mathsf{expp}_{j,\mathsf{KBE}}, m, j; r_{\mathsf{KBE}}). \end{aligned}$$

[8]Technically, our instantiation requires HTE setup to depend on the RSA group chosen by KBE.Setup, as the HTE construction only supports multiplicative homomorphism in one group. We omit this detail here for simplicity.

It then runs the encryption algorithm of the homomorphic threshold encryption scheme using randomness r_{HTE} to get $\mathsf{ct}_{\mathsf{HTE}} \leftarrow \mathsf{HTE.Enc}(\mathsf{pp}_{\mathsf{HTE}}, \mathsf{pk}_{\mathsf{HTE}}, h; r_{\mathsf{HTE}})$. Finally, it sets

$$x := (\mathsf{pp}_{\mathsf{KBE}}, \mathsf{expp}_{j,\mathsf{KBE}}, \mathsf{pp}_{\mathsf{HTE}}, \mathsf{pk}_{\mathsf{HTE}}, j, \mathsf{ct}_{\mathsf{KBE}}, \mathsf{ct}_{\mathsf{HTE}}),$$
$$w := (m, h, r_{\mathsf{KBE}}, r_{\mathsf{HTE}}),$$
$$\pi \leftarrow \mathsf{NIZK.Prove}(\mathsf{crs}, x, w).$$

It outputs the ciphertext $\mathsf{ct} = (\mathsf{ct}_{\mathsf{KBE}}, \mathsf{ct}_{\mathsf{HTE}}, \pi)$.

- $\mathsf{PartialDec}(\mathsf{pp}, \mathsf{sk}_i, \{j, \mathsf{ct}_j\}_{j \in S})$: The algorithm parses each ciphertext as $\mathsf{ct}_j = (\mathsf{ct}_{\mathsf{KBE},j}, \mathsf{ct}_{\mathsf{HTE},j}, \pi_j)$. The algorithm first verifies the NIZK proofs π_j for all $j \in S$ via

$$b_j \leftarrow \mathsf{NIZK.Verify}(\mathsf{crs}, (\mathsf{pp}_{\mathsf{KBE}}, \mathsf{expp}_{j,\mathsf{KBE}}, \mathsf{pp}_{\mathsf{HTE}}, \mathsf{pk}_{\mathsf{HTE}}, j, \mathsf{ct}_{\mathsf{KBE},j}, \mathsf{ct}_{\mathsf{HTE},j}), \pi_j) \quad \forall j \in S,$$

where the expanded public parameters of the KBE scheme are computed via[9]

$$\mathsf{expp}_{j,\mathsf{KBE}} \leftarrow \mathsf{KBE.PreEnc}(\mathsf{pp}_{\mathsf{KBE}}, j).$$

If any verification fails, it outputs $\perp$.

If all verifications pass, define the function

$$f_S(\{h_j\}_{j \in S}) := \mathsf{KBE.BatchKey}(\mathsf{pp}_{\mathsf{KBE}}, \{j, h_j\}_{j \in S}).$$

The algorithm then batches the HTE ciphertexts by running

$$\mathsf{ct}_{\mathsf{HTE},f} \leftarrow \mathsf{HTE.Eval}(\mathsf{pp}_{\mathsf{HTE}}, \mathsf{pk}_{\mathsf{HTE}}, f_S, \{\mathsf{ct}_{\mathsf{HTE},j}\}_{j \in S}).$$

Note that since HTE.Eval is deterministic, each party agrees on the same $\mathsf{ct}_{\mathsf{HTE},f}$. The algorithm then runs the partial decryption algorithm of the batched HTE ciphertext to get $d_i \leftarrow \mathsf{HTE.PartialDec}(\mathsf{pp}_{\mathsf{HTE}}, \mathsf{sk}_i, \mathsf{ct}_{\mathsf{HTE},f})$. It outputs the partial decryption key d_i.

- $\mathsf{Combine}(\mathsf{pp}, \mathsf{pk}, \{d_i\}_{i \in T}, \{j, \mathsf{ct}_j\}_{j \in S})$: The algorithm parses each ciphertext as $\mathsf{ct}_j = (\mathsf{ct}_{\mathsf{KBE},j}, \mathsf{ct}_{\mathsf{HTE},j}, \pi_j)$, and ignores all party indices for which $d_i = \perp$. If the remaining valid partial decryptions are insufficient to meet the threshold, it outputs $\perp$. Otherwise, it defines f_S as above and proceeds to batch the HTE ciphertexts by running

$$\mathsf{ct}_{\mathsf{HTE},f} \leftarrow \mathsf{HTE.Eval}(\mathsf{pp}_{\mathsf{HTE}}, \mathsf{pk}_{\mathsf{HTE}}, f_S, \{\mathsf{ct}_{\mathsf{HTE},j}\}_{j \in S}),$$

and combines the partial decryptions to get

$$\mathsf{bk} = \mathsf{HTE.Combine}(\mathsf{pp}_{\mathsf{HTE}}, \mathsf{pk}_{\mathsf{HTE}}, T, \{d_i\}_{i \in T}, \mathsf{ct}_{\mathsf{HTE},f}),$$

where bk is the batch decryption key of KBE. Finally, it runs the batch decryption algorithm of KBE to get $\{m_j\}_{j \in S} \leftarrow \mathsf{KBE.BatchDec}(\mathsf{pp}_{\mathsf{KBE}}, \mathsf{bk}, \{\mathsf{ct}_{\mathsf{KBE},j}\}_{j \in S})$. It outputs the decrypted messages $\{m_j\}_{j \in S}$.

The correctness of the construction follows immediately from the correctness of the underlying primitives. For succinctness, observe that all the KBE components $(\mathsf{pp}_{\mathsf{KBE}}, \mathsf{ct}_{\mathsf{KBE}})$ and all the HTE components $(\mathsf{pp}_{\mathsf{HTE}}, \mathsf{pk}_{\mathsf{HTE}}, \mathsf{sk}_i, \mathsf{ct}_{\mathsf{HTE}}, d_i)$ are succinct. The NIZK proofs are also succinct, as the

[9] Specific to our KBE instantiation in Construction 2, the expanded public parameters $\mathsf{expp}_{j,\mathsf{KBE}}$ for every index j can be computed in a batch in time $O(B \log B)$.

relation $\mathcal{R}$ is represented by a circuit of size $\mathsf{poly}(\lambda)$, independent of the batch size B and number of decryption parties n. (Here, we crucially rely on the succinctness of $\mathsf{KBE.OnlineEnc}$ and $\mathsf{HTE.Enc}$.)

We defer the security proof of the construction to Appendix A.2.

Combining the RSA-based KBE scheme from Construction 2, the DCR-based homomorphic threshold encryption scheme from Theorem 19, and the NIZK described above, we obtain the following theorem.

Theorem 9. *Assuming the RSA assumption (Assumption 1) and the DCR assumption, there exists an IND-CCA secure and rogue ciphertext secure epochless batched threshold encryption scheme supporting a polynomial-size index space. Furthermore, the setup size, public and secret key sizes, ciphertext size, and partial decryption size are* $\mathsf{poly}(\lambda)$ *and hence succinct in the batch size B, the index space size ℓ, and the number of decryption parties n.*

6 Aggregate Signatures

In this section, we show how to construct a bounded-query aggregate signature scheme. In Section 6.1, we define the syntax and security of aggregate signatures, along with the relaxed notion of bounded-query aggregate signatures. In Section 6.2, we show how the RSA-based PPRG construction from Construction 1 (leveraging its additional properties in Section 4.3) can be used to construct a bounded-query aggregate signature scheme.

6.1 Definition

We start by defining (fully secure) aggregate signature schemes, adapted from [BGLS03], where we allow a common reference string setup.

Definition 13. *An aggregate signature scheme for message space $\mathcal{M}$ consists of the following algorithms:*

- $\mathsf{Setup}(1^\lambda)$*: on input the security parameter λ, output the public parameters* pp*.*
- $\mathsf{KeyGen}(\mathsf{pp})$*: on input the public parameters* pp*, output a signing key* sk *and a verification key* vk*.*
- $\mathsf{Sign}(\mathsf{sk}, m \in \mathcal{M})$*: on input a signing key* sk *and a message m, output a signature σ.*
- $\mathsf{Agg}(\mathsf{pp}, \{\mathsf{vk}_i, m_i, \sigma_i\}_{i\in[B]})$*: On input the public parameters* pp *and a batch of B signatures σ_i to aggregate, where each σ_i is a signature for the message m_i under the verification key vk_i, output an aggregated signature σ_{agg}.*
- $\mathsf{Verify}(\mathsf{pp}, \{\mathsf{vk}_i, m_i\}_{i\in[B]}, \sigma_{\mathsf{agg}})$*: On input the public parameters* pp*, a set of B verification keys vk_i and messages m_i, and an aggregated signature σ_{agg}, output a bit indicating whether σ_{agg} is a valid aggregated signature on messages $\{m_i\}$ under verification keys $\{\mathsf{vk}_i\}$.*

We require the aggregate signature scheme to satisfy the following correctness and succinctness properties:

- **Correctness.** *For all $\lambda \in \mathbb{N}$, all* pp *generated by* $\mathsf{Setup}(1^\lambda)$*, all key pairs $(\mathsf{sk}_i, \mathsf{vk}_i)$ generated by* $\mathsf{KeyGen}(\mathsf{pp})$*, all messages $m_i \in \mathcal{M}$, and all signatures σ_i generated by* $\mathsf{Sign}(\mathsf{sk}_i, m_i)$*, it holds that*

$$\mathsf{Verify}(\mathsf{pp}, \{\mathsf{vk}_i, m_i\}_{i\in[B]}, \mathsf{Agg}(\mathsf{pp}, \{\mathsf{vk}_i, m_i, \sigma_i\}_{i\in[B]})) = 1.$$

- **Succinctness.** *The signatures $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$ and the aggregated signature $\sigma_{\mathsf{agg}} \leftarrow \mathsf{Agg}(\mathsf{pp}, \{\mathsf{vk}_i, m_i, \sigma_i\}_{i\in[B]})$ have size* $\mathsf{poly}(\lambda)$*.*

Additionally, we say that the aggregate signature scheme has efficient online verification *if the verification algorithm* Verify *can be separated into (i) a signature-independent preprocessing algorithm* $\mathsf{PreVerify}$*, which takes as input only the public parameters* pp*, verification keys* vk_i*, and messages* m_i *and outputs a succinct aggregated verification key* $\mathsf{vk}_{\mathsf{agg}}$*, and (ii) a signature-dependent online verification algorithm* $\mathsf{OnlineVerify}$*, which takes as input the aggregated verification key* $\mathsf{vk}_{\mathsf{agg}}$ *and signature* σ_{agg} *and efficiently (in time independent of B) verifies the aggregated signature.*

The security of the aggregate signature scheme is defined by the following unforgeability experiment.

Definition 14 (Security of Aggregate Signatures)**.** *For a security parameter* $\lambda \in \mathbb{N}$ *and an adversary* $\mathcal{A}$*, define the following unforgeability experiment* $\mathsf{Exp}_{\mathsf{agg}}(1^\lambda, \mathcal{A})$*.*

- *The challenger runs* $\mathsf{Setup}(1^\lambda)$ *to generate the public parameters* pp*, and gives* pp *to the adversary* $\mathcal{A}$*.*
- *The adversary* $\mathcal{A}$ *is allowed to make the following queries adaptively:*
  - **Key generation query.** *The challenger samples a key pair* $(\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{KeyGen}(\mathsf{pp})$*, gives* vk *to* $\mathcal{A}$*, and adds* $(\mathsf{sk}, \mathsf{vk})$ *to the set of honest key pairs* $\mathcal{K}$*.*
  - **Signing query.** *The adversary submits a message* m *and a verification key* vk *to the challenger. If* vk *is not in the set* $\mathcal{K}$ *of honest keys, the challenger returns* $\perp$*. Otherwise, the challenger retrieves the corresponding signing key* sk *from* $\mathcal{K}$*, computes* $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$*, gives* σ *to* $\mathcal{A}$*, and adds* (vk, m) *to the set of signing queries* Q*.*
  - **Corruption query.** *The adversary submits a verification key* vk *to the challenger. If* vk *is not in the set of generated verification keys in* $\mathcal{K}$*, the challenger returns* $\perp$*. Otherwise, the challenger retrieves the corresponding signing key* sk *from* $\mathcal{K}$*, gives* sk *to* $\mathcal{A}$*, and removes* $(\mathsf{sk}, \mathsf{vk})$ *from* $\mathcal{K}$*.*
- *Finally, the adversary outputs a forgery* $(\{\mathsf{vk}_i, m_i\}_{i\in[B]}, \sigma^*)$*. The adversary wins if*

$$\mathsf{Verify}(\mathsf{pp}, \{\mathsf{vk}_i, m_i\}_{i\in[B]}, \sigma^*) = 1,$$

  and there exists some index i^* *such that* $\mathsf{vk}_{i^*} \in \mathcal{K}$ *(i.e.,* vk_{i^*} *is honestly generated and not corrupted), and* $(\mathsf{vk}_{i^*}, m_{i^*}) \notin Q$ *(i.e., the winning key-message pair has not been queried). The experiment outputs* 1 *if the adversary wins, and* 0 *otherwise.*

We say that the aggregate signature scheme is secure if for all efficient adversaries $\mathcal{A}$*, there exists a negligible function* $\mathsf{negl}(\cdot)$ *such that for all* $\lambda \in \mathbb{N}$*,*

$$\Pr[\mathsf{Exp}_{\mathsf{agg}}(1^\lambda, \mathcal{A}) = 1] \leq \mathsf{negl}(\lambda).$$

We also define the relaxed notion of honest-key *security, where the adversary's winning condition requires that all verification keys* vk_i *in the forgery output must be in the honest key set* $\mathcal{K}$*.*

A *bounded-query aggregate signature scheme* is a relaxation of the above definition, where the setup algorithm takes a query bound 1^Q as input, the efficiency of the algorithms is allowed to depend on Q, and the security experiment is modified such that the adversary is allowed to make at most Q signing queries on the "winning" verification key vk_{i^*}.

Definition 15 (Bounded-Query Aggregate Signature)**.** *A bounded-query aggregate signature scheme is defined as in Definition 13 and Definition 14, except with the following modifications:*

- **Syntax.** *The* Setup *algorithm takes an additional input* 1^Q *which specifies the query bound. The efficiency of all algorithms is allowed to depend on* Q*.*
- **Succinctness.** *The signatures* $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$ *and the aggregated signature* $\sigma_{\mathsf{agg}} \leftarrow \mathsf{Agg}(\mathsf{pp}, \{\mathsf{vk}_i, m_i, \sigma_i\}_{i\in[B]})$ *have size* $\mathsf{poly}(Q, \lambda)$*. We say that the scheme is* strongly succinct *if the sizes are* $\mathsf{poly}(\lambda)$*, independent of* Q*.*
- **Efficient online verification.** *The scheme has efficient online verification if* OnlineVerify *runs in time* $\mathsf{poly}(\lambda, Q)$*. We say that it has* strongly efficient online verification *if* OnlineVerify *runs in time* $\mathsf{poly}(\lambda)$ *independent of* Q*.*
- **Security.** *The security experiment is defined as in Definition 14, except that the adversary specifies the query bound* 1^Q *at the start of the experiment, and that there is an additional restriction on the winning condition. For the output forgery* $(\{\mathsf{vk}_i, m_i\}_{i\in[B]}, \sigma^*)$*, the adversary wins if the signature verifies, and there exists some index* i^* *such that* $\mathsf{vk}_{i^*} \in \mathcal{K}$ *is in the set of honest keys,* $(\mathsf{vk}_{i^*}, m_{i^*}) \notin \boldsymbol{Q}$ *is not in the set of signing queries,* and there are at most Q queries to vk_{i^*} in the query set $\boldsymbol{Q}$*.*

Honest-key security for bounded-query aggregate signatures is defined similarly to Definition 14.

6.2 Construction

In this section, we show how the RSA-based PPRG construction from Construction 1 with its additional algebraic properties can be used to construct a bounded-query aggregate signature scheme with efficient online verification.

Construction 4 (Bounded-Query Aggregate Signature)**.** Let (Setup, SeedGen, Project, Eval) be the projective PRG from Construction 1, and let Union, BatchVerify = (PreBatchVerify, OnlineBatchVerify), and ReRand be the additional algorithms defined in Lemma 7, Definition 10, and Lemma 5 respectively. Let (Gen, Embed) be a cover-free generator defined in Definition 4. We construct the scheme for message space $\mathcal{M} = [2^\lambda]$ as follows.

- $\mathsf{Setup}(1^\lambda, 1^Q)$: On input the security parameter λ and the query bound Q, sample the Q-cover-free set system by running $(\mathsf{hk}, s, \ell) \leftarrow \mathsf{Gen}(1^\lambda, 1^Q, 2^\lambda)$. The algorithm then runs the PPRG setup $\mathsf{PPRG.Setup}(1^\lambda, 1^\ell)$ to generate the common reference string $\mathsf{crs} = (N, g, \mathsf{hk}_{\mathsf{PPRG}}, r, \ell)$, and computes the canonical verification seed $g_\emptyset = \mathsf{Project}(\mathsf{crs}, [\ell], g, \emptyset)$. The algorithm outputs the public parameters $\mathsf{pp} = (\mathsf{hk}, \mathsf{crs}, g_\emptyset)$.
- $\mathsf{KeyGen}(\mathsf{pp})$: On input the public parameters $\mathsf{pp} = (\mathsf{hk}, \mathsf{crs}, g_\emptyset)$, the algorithm generates a new PPRG seed pair $(u, \{v\}) \leftarrow \mathsf{ReRand}(\mathsf{crs}, \{g_\emptyset\})$ via rerandomization. It outputs the key pair $(\mathsf{sk}, \mathsf{vk}) = (u, v)$.
- $\mathsf{Sign}(\mathsf{sk}, m)$: On input the signing key $\mathsf{sk} = u$ and message $m \in \mathcal{M}$, the algorithm computes the embedding $S_m \leftarrow \mathsf{Embed}(\mathsf{hk}, m)$ to get the set $S_m \subset [\ell]$ of size s, and computes the projected seed $u_{S_m} = \mathsf{Project}(\mathsf{crs}, [\ell], u, S_m)$. It outputs $\sigma = u_{S_m}$.
- $\mathsf{Agg}(\mathsf{pp}, \{\mathsf{vk}_i, m_i, \sigma_i\}_{i\in[B]})$: On input the public parameters $\mathsf{pp} = (\mathsf{hk}, \mathsf{crs}, g_\emptyset)$ and a set of B tuples $(\mathsf{vk}_i, m_i, \sigma_i)$, the algorithm aggregates the signatures as follows.
  - Let $B_{\mathsf{vk}} \le B$ be the number of *distinct* verification keys, listed as $\mathsf{vk}'_1, \ldots, \mathsf{vk}'_{B_{\mathsf{vk}}}$. Let $\{m_{j,t}\}_t$ be the list of messages signed under vk'_j and let $\{\sigma_{j,t}\}_t$ be the corresponding signatures.

– For each $j \in [B_{\mathsf{vk}}]$, compute the aggregated signature under vk'_j by

$$\sigma'_j = \mathsf{Union}(\mathsf{crs}, \{S_{m_{j,t}}, \sigma_{j,t}\}_t),$$

where $S_{m_{j,t}}$ is the embedding of message $m_{j,t}$ computed by $\mathsf{Embed}(\mathsf{hk}, m_{j,t})$.

– Finally, aggregate the signatures under distinct verification keys by

$$\sigma_{\mathsf{agg}} = \prod_{j \in [B_{\mathsf{vk}}]} \sigma'_j.$$

- $\mathsf{Verify}(\mathsf{pp}, \{\mathsf{vk}_i, m_i\}_{i\in[B]}, \sigma_{\mathsf{agg}})$: On input the public parameters $\mathsf{pp} = (\mathsf{hk}, \mathsf{crs}, g_\emptyset)$, a set of B verification keys and messages, and an aggregated signature σ_{agg}, the preprocessing and online verification steps are defined respectively as follows.

  – $\mathsf{PreVerify}(\mathsf{pp}, \{\mathsf{vk}_i, m_i\}_{i\in[B]})$: Similar to the Agg algorithm, let $\mathsf{vk}'_1, \ldots, \mathsf{vk}'_{B_{\mathsf{vk}}}$ be the list of distinct verification keys, and let $m_{j,t}$ be the messages signed under vk'_j. For each $j \in [B_{\mathsf{vk}}]$, define the union set $S_j = \cup_t S_{m_{j,t}}$, where $S_{m_{j,t}} \leftarrow \mathsf{Embed}(\mathsf{hk}, m_{j,t})$, and compute the batch verification seed $\widetilde{v}$ by

$$(S, \widetilde{v}) = \mathsf{PreBatchVerify}(\mathsf{crs}, \{S_j, \mathsf{vk}'_j\}_j).$$

  The algorithm outputs the aggregated verification key $\mathsf{vk}_{\mathsf{agg}} = (S, \widetilde{v})$.

  – $\mathsf{OnlineVerify}(\mathsf{pp}, \mathsf{vk}_{\mathsf{agg}}, \sigma_{\mathsf{agg}})$: The online verification step checks whether

$$\mathsf{OnlineBatchVerify}(\mathsf{crs}, S, \widetilde{v}, \sigma_{\mathsf{agg}}) = 1.$$

We start by stating the efficiency of the construction.

Theorem 10 (Efficiency of Construction 4)**.** *When instantiated with the cover-free generator from Lemma 2 and the projective PRG from Construction 1, Construction 4 provides the following efficiency:*

- *The size of the public parameters* pp *is* $\mathsf{poly}(\lambda)$*, and the running time of* Setup *is* $Q^2 \cdot \mathsf{poly}(\lambda)$*.*
- *The sizes of the signing key* sk *and verification key* vk *are* $\mathsf{poly}(\lambda)$*, and the running time of* KeyGen *is* $\mathsf{poly}(\lambda)$*.*
- *The size of the signature* σ *is* $\mathsf{poly}(\lambda)$*, and the running time of* Sign *is* $Q^2 \cdot \mathsf{poly}(\lambda)$*. In particular, the signature scheme is strongly succinct.*
- *The size of the aggregated signature* σ_{agg} *is* $\mathsf{poly}(\lambda)$*, the running times of* Agg *and* $\mathsf{PreVerify}$ *are* $B \cdot Q \cdot \mathsf{poly}(\lambda)$*, and the running time of* $\mathsf{OnlineVerify}$ *is* $Q^2 \cdot \mathsf{poly}(\lambda)$*.*

In particular, Construction 4 is a strongly succinct bounded-query aggregate signature scheme with efficient online verification.

Proof. The efficiency of the algorithms follows from the efficiency of the cover-free generator (Lemma 2) and the PPRG (Construction 1). We briefly sketch the analysis for each algorithm. In the following, let (s, ℓ) be the parameters determined by the cover-free generator, where s is the size of the embedding set S_m for each message m, and ℓ is the size of the universe for the embedding sets.

- The Setup algorithm runs the cover-free generator, the PPRG setup, and the projection of the canonical generator to $\emptyset$. Following Lemma 4, the overall running time is $T_{\mathsf{Gen}} + \ell \cdot \mathsf{poly}(\lambda)$, where T_{Gen} is the running time of the cover-free generator. For size efficiency, again by Lemma 4, crs and $g_\emptyset$ have size $\mathsf{poly}(\lambda)$. The public parameters pp thus have size $|\mathsf{hk}| + \mathsf{poly}(\lambda)$.
- The KeyGen algorithm runs the rerandomization algorithm, which by Lemma 5 runs in time $\mathsf{poly}(\lambda)$ and outputs seeds of size $\mathsf{poly}(\lambda)$.
- The Sign algorithm computes the embedding S_m and the projected seed u_{S_m}. By Lemma 4, the projection runs in time $\mathsf{poly}(\lambda) \cdot |[\ell] \setminus S_m| \leq \mathsf{poly}(\lambda) \cdot \ell$. The overall running time is thus $T_{\mathsf{Embed}} + \ell \cdot \mathsf{poly}(\lambda)$, where T_{Embed} is the running time of the embedding algorithm. The size of the signature σ is the size of the projected seed, which is $\mathsf{poly}(\lambda)$.
- The Agg algorithm first computes the set embeddings for all messages, which runs in time $B \cdot T_{\mathsf{Embed}}$. Then, it computes the seed union for each distinct verification key. By Lemma 7, for each key vk'_j, the union runs in time $\mathsf{poly}(\lambda) \cdot \sum_t |S_{j,t}| \cdot T_j$, where T_j is the number of signatures under vk'_j. Summing over all distinct keys, the running time is $\mathsf{poly}(\lambda) \cdot B \cdot |S|$, where S is the union of all embeddings. Finally, the product of all union seeds can be computed in time $\mathsf{poly}(\lambda) \cdot B$. The overall running time of Agg is thus $B \cdot (|S| + T_{\mathsf{Embed}}) \cdot \mathsf{poly}(\lambda)$. The output aggregated signature σ_{agg} has size $\mathsf{poly}(\lambda)$ as it is in the same group as the projected seeds.
- The $\mathsf{PreVerify}$ algorithm first computes the embeddings of all messages in time $B \cdot T_{\mathsf{Embed}}$, then runs the $\mathsf{PreBatchVerify}$ algorithm, which by Lemma 9 runs in time $\mathsf{poly}(\lambda) \cdot B_{\mathsf{vk}} \cdot |S|$, where S is the union of all embedding sets. The output aggregated verification key $\mathsf{vk}_{\mathsf{agg}} = (S, \widetilde{v})$ has size $|S| + \mathsf{poly}(\lambda)$. Overall, the running time of $\mathsf{PreVerify}$ is $B \cdot (|S| + T_{\mathsf{Embed}}) \cdot \mathsf{poly}(\lambda)$.
- The $\mathsf{OnlineVerify}$ algorithm runs the $\mathsf{OnlineBatchVerify}$ algorithm, which by Lemma 9 runs in time $\mathsf{poly}(\lambda) \cdot |S|$, where S is the union of all embedding sets.

When instantiated with the cover-free generator from Lemma 2, we have $T_{\mathsf{Gen}} = |\mathsf{hk}| = 0$, $\ell = Q^2 \cdot \mathsf{poly}(\lambda)$, and $T_{\mathsf{Embed}} = Q \cdot \mathsf{poly}(\lambda)$. The claimed efficiency then follows by plugging in these parameters. □

The correctness of Construction 4 follows immediately from the correctness of $\mathsf{BatchVerify}$ (Lemma 8).

Theorem 11 (Correctness of Construction 4)**.** *Construction 4 is correct.*

Proof. By definition, $\mathsf{Agg}(\mathsf{pp}, \{\mathsf{vk}_i, m_i, \sigma_i\}_{i\in[B]})$ outputs

$$\sigma_{\mathsf{agg}} = \prod_{j\in[B_{\mathsf{vk}}]} \sigma'_j,$$

where

$$\sigma'_j = \mathsf{Union}(\mathsf{crs}, \{S_{m_{j,t}}, \sigma_{j,t}\}_t) = \mathsf{Project}(\mathsf{crs}, [\ell], u_j, S_j = \cup_t S_{m_{j,t}}).$$

The second equality follows from the correctness of Union in Lemma 7. By the correctness of $\mathsf{BatchVerify}$ in Lemma 8, it thus holds that

$$\mathsf{BatchVerify}(\mathsf{crs}, \{S_j, \mathsf{vk}'_j\}_j, \widetilde{u} = \prod_{j\in[B_{\mathsf{vk}}]} \sigma'_j) = 1,$$

which immediately implies that $\mathsf{Verify}(\mathsf{pp}, \{\mathsf{vk}_i, m_i\}_{i\in[B]}, \sigma_{\mathsf{agg}}) = 1$. □

Finally, we show that Construction 4 satisfies honest-key security. The core intuition behind the proof is that an unqueried challenge $(\mathsf{vk}_{i^*} = v, m_{i^*})$ in the forgery response corresponds to a set $S_{m_{i^*}}$ that is not contained in the union of any Q sets corresponding to the signing queries under vk_{i^*}. Therefore, by extracting the projected seed $u_{S_{m_{i^*}}}$, we break the pseudorandomness of the PPRG scheme with respect to the seed (u, v).

Theorem 12. *Assuming the RSA assumption (Assumption 1), Construction 4, when instantiated with the cover-free generator from Lemma 2, is an honest-key secure bounded-query aggregate signature scheme.*

Proof. We prove the contrapositive. Assume there exists an efficient adversary $\mathcal{A}$ that wins the honest-key forgery experiment $\mathsf{Exp}_{\mathsf{agg}}(1^\lambda, \mathcal{A})$ with non-negligible probability. We construct an efficient adversary $\mathcal{B}$ that breaks the pseudorandomness of the PPRG (Definition 9) with non-negligible probability as follows.

- The adversary $\mathcal{A}$ receives 1^λ and provides the query bound 1^Q.
- The adversary $\mathcal{B}$ samples the Q-cover-free set system $(\mathsf{hk}, s, \ell) \leftarrow \mathsf{Gen}(1^\lambda, Q, M = 2^\lambda)$, provides the length ℓ to its challenger, and receives the PPRG public parameters $\mathsf{crs} = (N, g, \mathsf{hk}_{\mathsf{PPRG}}, r, \ell)$. It computes $g_\emptyset = \mathsf{Project}(\mathsf{crs}, [\ell], g, \emptyset)$ and sends the public parameters $\mathsf{pp} = (\mathsf{hk}, \mathsf{crs}, g_\emptyset)$ to $\mathcal{A}$.
- Let Q_{vk} be the upper bound on the number of verification keys queried by $\mathcal{A}$ (which is upper bounded by the running time of $\mathcal{A}$). The adversary $\mathcal{B}$ guesses the index $j^* \in [Q_{\mathsf{vk}}]$ uniformly at random, where j^* is expected to be the index at which the forgery of $\mathcal{A}$ provides an unqueried message. $\mathcal{B}$ then samples $Q_{\mathsf{vk}} - 1$ fresh seeds $(u_j, v_j) \leftarrow \mathsf{KeyGen}(\mathsf{pp})$ for $j \in [Q_{\mathsf{vk}}] \setminus \{j^*\}$. It then makes a projected seed query on the empty set $\emptyset$ and receives $u_\emptyset = v$, and programs $v_{j^*} = v$. $\mathcal{B}$ also initializes an empty set $S^* = \emptyset$ to keep track of the projected seeds it queries to the PPRG challenger.
- $\mathcal{B}$ handles the queries from $\mathcal{A}$ as follows.
  - **Key generation query.** When $\mathcal{A}$ makes the j-th key generation query, $\mathcal{B}$ provides v_j as the verification key vk_j to $\mathcal{A}$.
  - **Signing query.** When $\mathcal{A}$ makes a signing query on a message m under a verification key $\mathsf{vk}_j = v_j$, for $j \neq j^*$, $\mathcal{B}$ computes and sends $\sigma = \mathsf{Sign}(\mathsf{sk} = u_j, m)$. For $j = j^*$, $\mathcal{B}$ computes $S_m \leftarrow \mathsf{Embed}(\mathsf{hk}, m)$, makes a projected seed query on the set S_m and receives $u_{S_m} = \mathsf{Project}(\mathsf{crs}, [\ell], u, S_m)$, and provides $u_{j^*, S_m} = u_{S_m}$ as the signature σ to $\mathcal{A}$. It also updates $S^* = S^* \cup S_m$ to keep track of the queried projected seeds.
  - **Corruption query.** When $\mathcal{A}$ makes a corruption query on a verification key $\mathsf{vk}_j = v_j$, $\mathcal{B}$ provides u_j if $j \neq j^*$, and aborts and outputs a random bit if $j = j^*$.
- After the queries, $\mathcal{A}$ outputs a forgery $(\{\mathsf{vk}_i^*, m_i\}_{i \in [B]}, \sigma^* = \widetilde{u})$. $\mathcal{B}$ aborts and outputs a random bit if:
  1. The forgery verification $\mathsf{Ver}(\mathsf{pp}, \{\mathsf{vk}_i^*, m_i\}_{i \in [B]}, \sigma^*)$ fails.
  2. Some verification key vk_i^* in the forgery is not in the set of honestly generated verification keys $\{v_j\}_{j \in [Q_{\mathsf{vk}}]}$.
  3. The guessed verification key v_{j^*} does not appear among the distinct verification keys in the forgery.

4. Every message m_i paired with the guessed verification key $\mathsf{vk}_i^* = v_{j^*}$ is in the set of signing queries under v_{j^*}.

Otherwise, let $B^* \le B$ be the number of distinct verification keys in the forgery. The adversary $\mathcal{B}$ reorders the forgery tuples $\{\mathsf{vk}_i^*, m_i\}_{i\in[B]}$ by these distinct verification keys, writing them as $(\{v_k^*, \{m_{k,t}\}_t\}_{k\in[B^*]})$. By the abort condition, there exists some $k^* \in [B^*]$ such that $v_{k^*}^* = v_{j^*}$ and there exists at least one message m_{k^*,t^*} under $v_{k^*}^*$ that was not queried in the signing queries under v_{j^*}. Furthermore, for every other index $k \in [B^*] \setminus \{k^*\}$, the signing seed u_k^* corresponding to v_k^* is known to $\mathcal{B}$.

Let $T^* = \cup_t S_{m_{k^*,t}}$ be the union of the sets corresponding to the messages signed under $v_{k^*}^*$ in the forgery. Since $\mathcal{A}$ is only allowed to query signatures on at most Q messages under v_{j^*}, and the cover-free generator guarantees that any set from $\mathsf{Embed}(\mathsf{hk}, \cdot)$ is not covered by the union of Q other sets, it holds that $T^* \not\subseteq S^*$.

- $\mathcal{B}$ then extracts the projected seed corresponding to $v_{k^*}^* = v_{j^*}$ given the batch seed $\widetilde{u}$ following the extraction procedure in Lemma 10:

$$u_{k^*,T^*}^* = \widetilde{u} \cdot \prod_{k\in[B^*]\setminus\{k^*\}} \mathsf{Union}(\mathsf{crs}, \{S_{m_{k,t}}, \mathsf{Sign}(\mathsf{sk} = u_k^*, m_{k,t})\}_t)^{-1}.$$

- $\mathcal{B}$ picks an index $i^* \in T^* \setminus S^*$ and computes the projected seed evaluation $b_{k^*,i^*} = \mathsf{Eval}(\mathsf{crs}, T^*, u_{k^*,T^*}^*, i^*)$. It then makes an evaluation query on i^* to the projective PRG challenger to obtain a bit b, and checks whether $b = b_{k^*,i^*}$. If they are equal, $\mathcal{B}$ outputs 1. Otherwise, it outputs a random bit.

We now analyze the winning probability of $\mathcal{B}$. First, we verify that, regardless of whether $\mathcal{B}$ is interacting with the experiment $\mathsf{Exp}_{\mathsf{prg}}^0$, which answers evaluation queries honestly, or the experiment $\mathsf{Exp}_{\mathsf{prg}}^1$, which answers evaluation queries with random bits, $\mathcal{B}$ simulates the honest-key forgery experiment $\mathsf{Exp}_{\mathsf{agg}}(1^\lambda, \mathcal{A})$ for $\mathcal{A}$, conditioned on the event that j^* is never queried for corruption.

- The challenge seed v_{j^*} provided by the PPRG challenger is statistically close to a key v sampled via rerandomization as in KeyGen (Lemma 5).

- For all signing queries under v_{j^*}, both $\mathsf{Exp}_{\mathsf{prg}}^0$ and $\mathsf{Exp}_{\mathsf{prg}}^1$ provide the projected seed $u_{j^*,S_m} = u_{S_m}$, which is identical to the real signature computed by $\mathsf{Sign}(\mathsf{sk} = u, m) = \mathsf{Project}(\mathsf{crs}, [\ell], u, S_m)$.

Combined with the fact that j^* is chosen uniformly at random, it is easy to see that

$$\Pr[\mathcal{A} \text{ provides a valid forgery to } \mathcal{B}] = \varepsilon \ge \frac{1}{Q_{\mathsf{vk}}} \cdot \Pr[\mathsf{Exp}_{\mathsf{agg}}(1^\lambda, \mathcal{A}) = 1] - \mathsf{negl}(\lambda)$$

for some non-negligible ε.

Finally, by the extractability guarantee of the PPRG (Lemma 10), whenever $\mathcal{A}$ provides a valid forgery, i.e., $\sigma^* = \widetilde{u}$ satisfies the PPRG batch verification

$$\mathsf{BatchVerify}(\mathsf{crs}, \{T_k, v_k^*\}_{k\in[B^*]}, \widetilde{u}) = 1, \text{ where } T_k = \cup_t S_{m_{k,t}},$$

the extracted seed $u_{k^*,T^*}^* = u_{j^*,T^*}$ is identical to the real projected seed $\mathsf{Project}(\mathsf{crs}, [\ell], u, T^*)$. Therefore, when interacting with $\mathsf{Exp}_{\mathsf{prg}}^0$, $\mathcal{B}$ always outputs 1, implying that

$$\Pr[\mathsf{Exp}_{\mathsf{prg}}^0(1^\lambda, \mathcal{B}) = 1] = (1-\varepsilon)\cdot\frac{1}{2} + \varepsilon = \frac{1+\varepsilon}{2}.$$

On the other hand, when interacting with $\mathsf{Exp}^1_{\mathsf{prg}}$, the probability that $b = b_{k^*,i^*}$ is exactly $1/2$, implying that

$$\Pr[\mathsf{Exp}^1_{\mathsf{prg}}(1^\lambda, \mathcal{B}) = 1] = \frac{1}{2},$$

which completes the proof. □

Remark 4 (Security against malicious keys)**.** *Theorem 12 only shows honest-key security of Construction 4. Indeed, similar to the discussion in Remark 1, an adversary can forge an aggregated signature by generating a malicious verification key* $\mathsf{vk}^* = \mathsf{vk}^{-1}$ *for some honest verification key* vk *generated by the challenger.*

Nevertheless, since the reduction to Lemma 10 only relies on the fact that every (u_j, v_j) *satisfies the relation* $u_j^{E_\ell} = u_{j,\emptyset} = v_j$*, Construction 4 is actually secure against semi-malicious adversaries who can generate malicious keys, but are required to provide a valid signing key along with the malicious verification key.*

One can thus generically transform Construction 4 into a fully secure aggregate signature scheme by adding a NIZKPoK of the key generation randomness a *to the public key. The reduction for Theorem 12 extends naturally, with the additional step of extracting the randomness* a_j *from every verification key* $\mathsf{vk}_j = (g_\emptyset^{a_j}, \pi_j)$ *in the forgery to retrieve the secret key* $\mathsf{sk}_j = g^{a_j}$*.*

Importantly, since the ReRand *algorithm has a circuit size that is independent of* Q*, the NIZK size, proving time, and verification time only depend on* λ *and are independent of* Q*. Therefore, after the transformation, the* KeyGen *running time and the* vk *size of the resulting scheme remain* $\mathsf{poly}(\lambda)$*, and the verification overhead is* $B \cdot \mathsf{poly}(\lambda)$*, which is subsumed.*

7 Bounded-query Secret-key Broadcast Encryption

In this section, we construct a bounded-query secret-key broadcast encryption scheme from RSA. In Section 7.1, we recall the definition of secret-key broadcast encryption and formally define its bounded-query variant. In Section 7.2, we present our construction using the PPRG scheme from Construction 1, and prove its security.

7.1 Definition

We start by recalling the definition of secret-key broadcast encryption. For simplicity, we stick to the case where the encryption algorithm encrypts a single bit. The multi-bit encryption can be achieved trivially by encrypting each bit separately. For a recipient set S, we write $B = |S|$ for the broadcast size.

Definition 16 (Secret-key Broadcast Encryption)**.** *A secret-key broadcast encryption scheme* BE *consists of the following algorithms:*

- $\mathsf{Setup}(1^\lambda, 1^n)$ *takes as input the security parameter* λ *and the number of users* n *and outputs a master public key* mpk *and a master secret key* msk*.*
- $\mathsf{KeyGen}(\mathsf{msk}, i)$ *takes as input the master secret key* msk *and a user index* $i \in [n]$*, and outputs a secret key* sk_i *for user* i*.*
- $\mathsf{Enc}(\mathsf{msk}, S, m)$ *takes as input the master* secret *key* msk*, a set of user indices* $S \subseteq [n]$*, and a message* $m \in \{0,1\}$*, and outputs a ciphertext* ct_S*.*

- $\mathsf{Dec}(\mathsf{mpk}, \mathsf{sk}_i, i, \mathsf{ct}_S, S)$ *takes as input the master public key* mpk*, a secret key* sk_i *for user* i*, and a ciphertext* ct_S *encrypted to the user set* $S \subseteq [n]$*, and outputs either a message* m *or* $\perp$ *for decryption failure.*

The broadcast encryption scheme is required to satisfy the following correctness and efficiency properties.

- **Correctness.** *For every* $\lambda \in \mathbb{N}$*, every master key pair* $(\mathsf{mpk}, \mathsf{msk})$ *output by* $\mathsf{Setup}(1^\lambda, 1^n)$*, every set* $S \subseteq [n]$*, every message* m*, and every user index* $i \in S$*, it holds that*

$$\Pr[\mathsf{Dec}(\mathsf{mpk}, \mathsf{KeyGen}(\mathsf{msk}, i), i, \mathsf{Enc}(\mathsf{msk}, S, m), S) = m] = 1.$$

- **Succinctness.** *The size of* $\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{msk}, S, m)$ *is* $\mathsf{poly}(\lambda, \log B)$*.*

The adaptive security of a secret-key broadcast encryption scheme is defined by the following experiment.

Definition 17 (Adaptive Security of Secret-key Broadcast Encryption)**.** *For a security parameter* λ*, an adversary* $\mathcal{A}$*, and* $\beta \in \{0,1\}$*, define the following adaptive security experiment* $\mathsf{Exp}^{\beta}_{\mathsf{BE}}(1^\lambda, \mathcal{A})$*:*

- *The adversary* $\mathcal{A}$ *is given* 1^λ*, and outputs the number of users* 1^n*.*
- *The challenger runs* $(\mathsf{mpk}, \mathsf{msk}) \leftarrow \mathsf{Setup}(1^\lambda, 1^n)$ *and gives* mpk *to* $\mathcal{A}$*. The challenger also initializes an empty set* $\mathcal{I}$ *to record the corrupted users.*
- $\mathcal{A}$ *can adaptively make the following queries:*
  - **Corruption query.** $\mathcal{A}$ *specifies a user index* $i \in [n]$*. The challenger computes the secret key* $\mathsf{sk}_i \leftarrow \mathsf{KeyGen}(\mathsf{msk}, i)$ *and sends* sk_i *to* $\mathcal{A}$*. The index* i *is added to the set* $\mathcal{I}$ *of corrupted users.*
  - **Encryption query.** $\mathcal{A}$ *specifies a set of user indices* $S \subseteq [n]$ *and a message* $m \in \{0,1\}$*. The challenger computes* $\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{msk}, S, m)$ *and sends* ct *to* $\mathcal{A}$*.*
  - **Challenge query.** $\mathcal{A}$ *specifies a set of user indices* $S^* \subseteq [n]$*. The challenger computes* $\mathsf{ct}^* \leftarrow \mathsf{Enc}(\mathsf{msk}, S^*, \beta)$ *and sends* ct^* *to* $\mathcal{A}$*. The adversary can make at most one challenge query.*
- *Finally,* $\mathcal{A}$ *outputs a guess* $b \in \{0,1\}$*. If the corrupted set* $\mathcal{I}$ *does not intersect with the challenge set* S^**, the experiment outputs* b*. Otherwise, the experiment outputs* 0*.*

We say that a secret-key broadcast encryption scheme BE *is adaptively secure if for every efficient adversary* $\mathcal{A}$*, there exists a negligible function* $\mathsf{negl}(\cdot)$ *such that*

$$\left|\Pr[\mathsf{Exp}^0_{\mathsf{BE}}(1^\lambda, \mathcal{A}) = 1] - \Pr[\mathsf{Exp}^1_{\mathsf{BE}}(1^\lambda, \mathcal{A}) = 1]\right| \leq \mathsf{negl}(\lambda).$$

Definition 18 (Bounded-query Secret-key Broadcast Encryption)**.** *A* bounded-query *secret-key broadcast encryption scheme is defined as in Definition 16 and Definition 17, with the following modifications:*

- **Syntax.** *The* Setup *algorithm takes an additional input* 1^Q *which specifies the query bound. All algorithms are allowed to have efficiency dependent on* Q*.*
- **Succinctness.** *The size of* $\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{msk}, S, m)$ *is* $\mathsf{poly}(\lambda, \log B, \log Q)$*.*
- **Security.** *The adaptive security experiment is defined as in Definition 17, except that the adversary specifies the query bound* 1^Q *at the beginning of the experiment, and is allowed to make at most* Q *encryption queries.*

Remark 5 (Bounded-Queryset Secret-key Broadcast Encryption)**.** *One can define a slightly strengthened variant of bounded-query secret-key broadcast encryption, which we call* bounded-queryset secret-key broadcast encryption*, where the adversary is allowed to make arbitrary encryption queries of the form* (S, m)*, with the restriction that it can query at most* Q *distinct sets. A bounded-query secret-key broadcast encryption scheme can be generically lifted to a bounded-queryset scheme via the following transformation:*

- *Use* $\mathsf{PRF}(k, S) \to (r_S, \mathsf{SKE.sk}_S)$ *to derandomize the encryption randomness for each set* S*, and deterministically generate a SKE secret key for set* S*.*
- *To encrypt a message* m *for a set* S*, compute* $\mathsf{ct} = \mathsf{BE.Enc}(\mathsf{msk}, S, \mathsf{SKE.sk}_S; r_S)$ *that deterministically encrypts the secret key under broadcast, and compute* $\mathsf{ct}_m = \mathsf{SKE.Enc}(\mathsf{SKE.sk}_S, m)$ *encrypting the message with fresh randomness. Output the ciphertext* $(\mathsf{ct}, \mathsf{ct}_m)$*.*

Notice that different ciphertexts for the same set S *will have the same first part* ct*. Therefore, the secret key* sk_S *for an honest set* S *is hidden under the security of the underlying bounded-query secret-key broadcast encryption scheme. The security of the message thus follows from the* PRF *and* SKE *security.*

7.2 Construction

In this section, we show how to generically construct a bounded-query secret-key broadcast encryption scheme from a PPRG. With the RSA-based PPRG construction (Construction 1), we achieve almost optimal size on the master public key and the ciphertext size.

Construction 5. Let $(\mathsf{Setup}, \mathsf{SeedGen}, \mathsf{Project}, \mathsf{Eval})$ be the projective PRG from Construction 1. Let PRF be a pseudorandom function with one-bit output. We construct a bounded-query secret-key broadcast encryption scheme as follows:

- $\mathsf{Setup}(1^\lambda, 1^n, 1^Q)$ sets the PPRG output length $\ell = 2\lambda Q n$. It runs $\mathsf{crs} \leftarrow \mathsf{PPRG.Setup}(1^\lambda, 1^\ell)$ and samples a PPRG seed $u \leftarrow \mathsf{SeedGen}(\mathsf{crs})$ and a PRF key $k \leftarrow \{0,1\}^\lambda$. The master public key is set to $\mathsf{mpk} = \mathsf{crs}$, and the master secret key is set to $\mathsf{msk} = (\mathsf{crs}, u, k)$. The algorithm outputs $(\mathsf{mpk}, \mathsf{msk})$.
- $\mathsf{KeyGen}(\mathsf{msk}, i)$ computes a slot key for each slot $j \in [2\lambda Q]$ as follows.
  - Compute PPRG evaluation $b_{i,j} = \mathsf{Eval}(\mathsf{crs}, [\ell], u, 2\lambda Q(i-1)+j)$, where the index mapping $(i, j) \mapsto 2\lambda Q(i-1) + j$ maps $[n] \times [2\lambda Q]$ to $[\ell]$.
  - Compute message pad $p_j = \mathsf{PRF}(k, j)$.
  - Compute slot key $s_{i,j} = b_{i,j} \oplus p_j$.

  The algorithm outputs the secret key for user i as $\mathsf{sk}_i = \{s_{i,j}\}_{j \in [2\lambda Q]}$.
- $\mathsf{Enc}(\mathsf{msk}, S, m)$ parses $\mathsf{msk} = (\mathsf{crs}, u, k)$. It samples a random subset of slots $T \subseteq [2\lambda Q]$ of size λ, and defines the projection set $S_T = \{2\lambda Q(i-1) + j : i \in S, j \in T\} \subseteq [\ell]$. The algorithm computes the decryption hint $u_{S_T} = \mathsf{Project}(\mathsf{crs}, [\ell], u, S_T)$, along with message pads $\{p_j = \mathsf{PRF}(k, j)\}_{j \in T}$. It outputs the ciphertext

$$\mathsf{ct} = (m \oplus (\bigoplus_{j \in T} p_j), u_{S_T}, T).$$

- $\mathsf{Dec}(\mathsf{mpk}, \mathsf{sk}_i, i, \mathsf{ct}, S)$ parses $\mathsf{mpk} = \mathsf{crs}$, $\mathsf{sk}_i = \{s_{i,j}\}_{j\in[2\lambda Q]}$, and $\mathsf{ct} = (c, u_{S_T}, T)$. It similarly reconstructs the projection set $S_T = \{2\lambda Q(i-1)+j : i \in S, j \in T\} \subseteq [\ell]$. If $i \notin S$, it outputs $\perp$. Otherwise, using the projected seed u_{S_T}, the algorithm computes the PPRG evaluation

$$b_{i,j} = \mathsf{Eval}(\mathsf{crs}, S_T, u_{S_T}, 2\lambda Q(i-1)+j) \text{ for each } j \in T,$$

recovers the corresponding message pads $p_j = s_{i,j} \oplus b_{i,j}$ for each $j \in T$, and outputs the decrypted message

$$m^* = c \oplus \Big(\bigoplus_{j\in T} p_j\Big).$$

Theorem 13. *Construction 5 satisfies the following efficiency guarantees.*

- *The master public key* mpk *has size* $\mathsf{poly}(\lambda)$*, and the setup algorithm* Setup *runs in* $Qn \cdot \mathsf{poly}(\lambda)$ *time.*
- *The secret key* sk_i *for each user* i *has size* $Q \cdot \mathsf{poly}(\lambda)$*, and the key generation algorithm* KeyGen *runs in* $Q^2 n \cdot \mathsf{poly}(\lambda)$ *time.*
- *The encryption algorithm* Enc *runs in* $Qn \cdot \mathsf{poly}(\lambda)$ *time, and the ciphertext size is* $\mathsf{poly}(\lambda)$*.*
- *The decryption algorithm* Dec *runs in time* $B \cdot \mathsf{poly}(\lambda)$ *(assuming RAM access to the secret key* sk_i*).*

Proof. The efficiency of the scheme follows from the efficiency of the underlying PPRG (Lemma 4). We sketch the efficiency analysis for each algorithm as follows:

- The running time for Setup is dominated by the PPRG setup with output length $\ell = 2\lambda Qn$, which runs in time $\ell \cdot \mathsf{poly}(\lambda) = Qn \cdot \mathsf{poly}(\lambda)$. All remaining steps run in $\mathsf{poly}(\lambda)$ time. The common reference string crs and the master seed u both have size $\mathsf{poly}(\lambda)$.
- The KeyGen algorithm computes $2\lambda Q$ PPRG evaluations, each of which runs in time $\ell \cdot \mathsf{poly}(\lambda) = Qn \cdot \mathsf{poly}(\lambda)$. The message pads p_j and the slot keys $s_{i,j}$ can each be computed in $\mathsf{poly}(\lambda)$ time. Therefore, the total running time is $Q^2 n \cdot \mathsf{poly}(\lambda)$. The secret key sk_i consists of $2\lambda Q$ bits, and thus has size $Q \cdot \mathsf{poly}(\lambda)$.
- The running time for Enc is dominated by the computation of the decryption hint $u_{S_T} = \mathsf{Project}(\mathsf{crs}, [\ell], u, S_T)$, which runs in time $(\ell - |S_T|) \cdot \mathsf{poly}(\lambda) \le Qn \cdot \mathsf{poly}(\lambda)$. The output ciphertext consists of a bit for the encrypted message, the decryption hint u_{S_T} of size $\mathsf{poly}(\lambda)$, and the slot selection set T with description size $\lambda \cdot \log 2\lambda Q = \mathsf{poly}(\lambda)$. The total ciphertext size is thus $\mathsf{poly}(\lambda)$.
- The algorithm Dec first computes the PPRG evaluation on each $j \in T$, which runs in time $|T| \cdot |S_T| \cdot \mathsf{poly}(\lambda) = B \cdot |T|^2 \cdot \mathsf{poly}(\lambda) = B \cdot \mathsf{poly}(\lambda)$. It then reads the corresponding slot keys $s_{i,j}$ from the secret key sk_i and computes the message pads p_j, which runs in time $O(\lambda)$ (or $O(\lambda) + Q$ if we include the reading time of the secret key).

□

Theorem 14. *Construction 5 is correct.*

Proof. The correctness of the scheme follows from the correctness of the PPRG. For any user $i \in S$, $j \in [2\lambda Q]$, the PPRG evaluation $\mathsf{Eval}(\mathsf{crs}, S_T, u_{S_T}, 2\lambda Q(i-1)+j)$ computed during decryption

is exactly the same as the evaluation $\mathsf{Eval}(\mathsf{crs}, [\ell], u, 2\lambda Q(i-1)+j)$ computed during key generation. Therefore,

$$p_j = s_{i,j} \oplus b_{i,j} = (b_{i,j} \oplus \mathsf{PRF}(k, j)) \oplus b_{i,j} = \mathsf{PRF}(k, j),$$

and the decrypted message is

$$m^* = c \oplus \Big(\bigoplus_{j \in T} p_j\Big) = m.$$

This completes the proof. □

Theorem 15. *Assuming the adaptive pseudorandomness of the underlying PPRG (Lemma 3), and assuming the pseudorandomness of* PRF*, Construction 5 is an adaptively secure bounded-query secret-key broadcast encryption scheme.*

Proof. To show the security of the scheme, we argue that the challenge ciphertext is pseudorandom to the adversary by relying on the pseudorandomness of the message pads p_j. While some of the message pads p_j are revealed to the adversary through encryption queries, the randomness of the slot selection T ensures that with overwhelming probability, the challenge ciphertext contains at least one message pad p_{j^*} that is not revealed. The only remaining variables in the adversary's view that depend on p_{j^*} are secret keys $s_{i,j^*} = b_{i,j^*} \oplus p_{j^*}$, where the PPRG evaluations b_{i,j^*} are pseudorandom.

Formally, we define the following hybrid experiments:

- $\mathcal{H}_0^\beta$ is the broadcast encryption security experiment $\mathsf{Exp}_{\mathsf{BE}}^\beta(\mathcal{A})$.
- $\mathcal{H}_1^\beta$ is the same as $\mathcal{H}_0^\beta$, except that the challenger samples all message pads $p_1, \ldots, p_{2\lambda Q}$ at the beginning of the experiment, and instead of computing them by $p_j = \mathsf{PRF}(k, j)$, the challenger uses random bits. In all queries, the challenger uses the message pads sampled at the beginning of the experiment.

  To see that $\mathcal{H}_0^\beta$ and $\mathcal{H}_1^\beta$ are indistinguishable, we first observe that in $\mathcal{H}_0^\beta$, the view of the adversary $\mathcal{A}$ does not change if we sample all (pseudorandom) message pads at the beginning of the experiment. Furthermore, by the pseudorandomness of PRF, for every efficient adversary $\mathcal{A}$, the experiment using pseudorandom message pads is computationally indistinguishable from the experiment using truly random message pads. Therefore, there exists a negligible function $\mathsf{negl}(\cdot)$ such that

  $$|\Pr[\mathcal{H}_0^\beta(\mathcal{A}) = 1] - \Pr[\mathcal{H}_1^\beta(\mathcal{A}) = 1]| \leq \mathsf{negl}(\lambda).$$

- $\mathcal{H}_2^\beta$ is the same as $\mathcal{H}_1^\beta$, except that the challenger samples during the setup phase all random sets $T_1, \ldots, T_Q$ to be used in each encryption query, along with the set T_{ch} to be used in the challenge query. If $T_{\mathsf{ch}} \subseteq \bigcup_{q \in [Q]} T_q$, the challenger aborts and the experiment outputs 0. Otherwise, the challenger proceeds as in $\mathcal{H}_1^\beta$.

  First, observe that in the encryption algorithm, T is sampled randomly and independently of the input. Therefore, sampling all sets at the beginning of the experiment does not change the distribution of the adversary's view. Moreover, since T_{ch} is a random subset of size λ sampled independently from the sets $T_1, \ldots, T_Q$, and since $\bigcup_{q \in [Q]} T_q$ has size at most λQ, the probability that $T_{\mathsf{ch}} \subseteq \bigcup_{q \in [Q]} T_q$ is at most $\binom{\lambda Q}{\lambda} / \binom{2\lambda Q}{\lambda} \leq 2^{-\lambda}$. Therefore, for every efficient adversary $\mathcal{A}$,

  $$|\Pr[\mathcal{H}_1^\beta(\mathcal{A}) = 1] - \Pr[\mathcal{H}_2^\beta(\mathcal{A}) = 1]| \leq 2^{-\lambda}.$$

- $\mathcal{H}_3$ is modified from $\mathcal{H}_2^\beta$ as follows. After sampling the sets $T_1, \ldots, T_Q$ and T_{ch}, the challenger sets j^* to be the first index in $T_{\mathsf{ch}} \setminus \bigcup_{q\in[Q]} T_q$. It then changes the response to the adversary's queries as follows.
  - **Corruption query.** For a corruption query on user $i \in [n]$, the challenger computes $s_{i,j} = b_{i,j} \oplus p_j$ honestly for each slot $j \in [2\lambda Q] \setminus \{j^*\}$, and samples s_{i,j^*} uniformly at random.
  - **Challenge query.** The challenger sets the first element c of the ciphertext to a uniformly random bit, $c \leftarrow \{0,1\}$.

We now argue that the experiments $\mathcal{H}_2^\beta$ and $\mathcal{H}_3$ are indistinguishable, which we show via a reduction to the pseudorandomness of the underlying PPRG. Given an adversary $\mathcal{A}$ that distinguishes $\mathcal{H}_2^\beta$ and $\mathcal{H}_3$, we construct an efficient adversary $\mathcal{B}$ that interacts with the pseudorandomness game of the underlying PPRG (Definition 9) as follows.

- The adversary $\mathcal{A}$ specifies the number of users n and the upper bound Q on the number of encryption queries. The adversary $\mathcal{B}$ sets the PPRG output length $\ell = 2\lambda Q n$, and submits the length ℓ to the PPRG pseudorandomness challenger and receives the common reference string crs.
- The adversary $\mathcal{B}$ samples all message pads $p_1, \ldots, p_{2\lambda Q}$ along with the random sets $T_1, \ldots, T_Q$ and T_{ch}. It sets j^* to be the first index in $T_{\mathsf{ch}} \setminus \bigcup_{q\in[Q]} T_q$ as in $\mathcal{H}_3$, and aborts and outputs 0 if j^* does not exist, as in $\mathcal{H}_2^\beta$.
- $\mathcal{B}$ defines the challenge set $\overline{S}_{j^*} = [\ell] \setminus \{2\lambda Q(i-1) + j^* : i \in [n]\} \subseteq [\ell]$, makes a projected seed query on $\overline{S}_{j^*}$ and receives the projected seed $u_{\overline{S}_{j^*}}$.
- The adversary $\mathcal{B}$ sends $\mathsf{mpk} = \mathsf{crs}$ to $\mathcal{A}$, and simulates all queries from $\mathcal{A}$ as follows.
  - **Corruption query.** For a corruption query on user $i \in [n]$, $\mathcal{B}$ computes $s_{i,j} = b_{i,j} \oplus p_j$ for each slot $j \in [2\lambda Q]$, where $b_{i,j}$ is computed as
    - If $j \neq j^*$, compute via the honest PPRG evaluation $b_{i,j} = \mathsf{Eval}(\mathsf{crs}, \overline{S}_{j^*}, u_{\overline{S}_{j^*}}, 2\lambda Q(i-1)+j)$ using the projected seed $u_{\overline{S}_{j^*}}$.
    - If $j = j^*$, obtain b_{i,j^*} by making an evaluation query to the pseudorandomness challenger on the index $2\lambda Q(i-1) + j^*$.
  - **Encryption query.** For the q-th encryption query on set $S \subseteq [n]$ and message m, $\mathcal{B}$ picks the set T_q, defines S_{T_q} as in the honest algorithm, and computes the projected seed $u_{S_{T_q}}$ by $\mathsf{Project}(\mathsf{crs}, \overline{S}_{j^*}, u_{\overline{S}_{j^*}}, S_{T_q})$. Note that $j^* \notin T_q$ by the definition of j^*, and therefore $S_{T_q} \subseteq \overline{S}_{j^*}$ and the projected seed query is valid. $\mathcal{B}$ then computes the ciphertext as in the honest algorithm by
$$\mathsf{ct} = (m \oplus (\bigoplus_{j\in T_q} p_j), u_{S_{T_q}}, T_q).$$
  - **Challenge query.** For the challenge query on set S^*, $\mathcal{B}$ picks the set T_{ch}, defines $S^*_{T_{\mathsf{ch}}}$ as in the honest algorithm, and obtains the projected seed $u_{S^*_{T_{\mathsf{ch}}}}$ by making a projected seed query to the pseudorandomness challenger on the set $S^*_{T_{\mathsf{ch}}}$. $\mathcal{B}$ then computes the challenge ciphertext as in the honest algorithm by
$$\mathsf{ct}^* = (\beta \oplus (\bigoplus_{j\in T_{\mathsf{ch}}} p_j), u_{S^*_{T_{\mathsf{ch}}}}, T_{\mathsf{ch}}).$$

- Finally, $\mathcal{A}$ outputs a guess $b \in \{0,1\}$. If none of the user indices in the challenge set S^* is corrupted, $\mathcal{B}$ outputs b. Otherwise, $\mathcal{B}$ outputs 0.

We argue that the advantage of $\mathcal{B}$ against the pseudorandomness of the underlying PPRG is equal to the advantage of $\mathcal{A}$ in distinguishing $\mathcal{H}_2^\beta$ and $\mathcal{H}_3$ as follows.

- When $\mathcal{B}$ interacts with the pseudorandomness experiment $\mathsf{Exp}^0_{\mathsf{prg}}$, where all evaluation queries are answered consistently with the master projected seed, the view of $\mathcal{A}$ in the simulation is exactly the same as $\mathcal{H}_2^\beta$. In particular, for every corruption query on user i, it is guaranteed that $i \notin S^*$ and that the index $2\lambda Q(i-1)+j^*$ is not in the sets $S^*_{T_{\mathsf{ch}}}$ and $\overline{S}_{j^*}$, which were the only two projected seed queries to the pseudorandomness challenger. Therefore, the evaluation query for index $2\lambda Q(i-1)+j^*$ is answered with $\mathsf{Eval}(\mathsf{crs}, [\ell], u, 2\lambda Q(i-1)+j^*)$, which is identical to the honest computation of b_{i,j^*} in $\mathcal{H}_2^\beta$.

- When $\mathcal{B}$ interacts with the pseudorandomness experiment $\mathsf{Exp}^1_{\mathsf{prg}}$, where the evaluation query on index $2\lambda Q(i-1)+j^*$ is answered with a uniformly random bit, the view of $\mathcal{A}$ in the simulation is exactly the same as $\mathcal{H}_3$. In particular, by an argument similar to the one above, each b_{i,j^*} is answered with a uniformly random bit during a corruption query. Now, by looking at all the elements in the adversary's view that depend on the j^*-th message pad p_{j^*}, they are computed by

$$\{s_{i,j^*} = b_{i,j^*} \oplus p_{j^*}\}_{i\in\mathcal{I}}, c = p_{j^*} \oplus m_\beta \oplus \Big(\bigoplus_{j\in T_{\mathsf{ch}}\setminus\{j^*\}} p_j\Big),$$

  which has the same distribution as $|\mathcal{I}|+1$ uniformly random bits. Therefore, the adversary's view in the simulation is exactly the same as $\mathcal{H}_3$.

Finally, following Lemma 3, the advantage of $\mathcal{B}$ in the pseudorandomness experiment is negligible, which implies that the advantage of $\mathcal{A}$ in distinguishing $\mathcal{H}_2^\beta$ and $\mathcal{H}_3$ is also negligible. Combining all the hybrid indistinguishability arguments, we conclude that the advantage of $\mathcal{A}$ in distinguishing $\mathcal{H}_0^0$ and $\mathcal{H}_0^1$ is negligible, which completes the proof. □

Remark 6 (Optimization leveraging the RSA trapdoor)**.** *In the PPRG construction in Construction 1, the computation of the projected seed $u_{S_T} = u^{\prod_{i\in[\ell]\setminus S_T} e_i}$ takes time linear in $\ell - |S_T| = 2\lambda Qn - |S_T|$. In the context of Construction 5, $|S_T| = B\lambda$ is much smaller than ℓ. One can optimize this by including the RSA factors p and q in the master secret key and precomputing $v = u_\emptyset = u^{\prod_{i\in[\ell]} e_i}$ at the setup phase. Then, to compute u_{S_T}, one can compute $u_{S_T} = v^{\prod_{i\in S_T} e_i^{-1} \bmod \phi(N)}$ using the RSA trapdoor, which takes time $|S_T|\cdot\mathsf{poly}(\lambda)$. This reduces the key generation time from $Q^2 n\cdot\mathsf{poly}(\lambda)$ to $Q\cdot\mathsf{poly}(\lambda)$, and the encryption time from $Qn\cdot\mathsf{poly}(\lambda)$ to $B\cdot\mathsf{poly}(\lambda)$.*

8 Homomorphic Threshold Encryption for Cyclic Groups

In this section, we construct a multiplicatively homomorphic threshold encryption scheme (HTE) for cyclic groups, assuming an additive HTE over $\mathbb{Z}_M$ with tunable modulus M. In particular, the only requirements for the cyclic group are a bounded order, efficient group operations, and an efficiently computable generator. No computational assumption is required for the cyclic group itself.

In Section 8.1, we recall the definition of homomorphic threshold encryption. We then proceed to construct such a multiplicative HTE for cyclic groups in Section 8.2, and prove its security. Finally, we provide our concrete instantiation from DCR at the end of Section 8.2.

8.1 Homomorphic Threshold Encryption

In this section, we recall the notion of homomorphic threshold encryption. Our formulation largely follows [BFOQ25].

Definition 19. *A homomorphic threshold encryption scheme* HTE *consists of the following algorithms.*

- $\mathsf{Setup}(1^\lambda) \to \mathsf{pp}$*: On input the security parameter* λ*, output the public parameters* pp*.*
- $\mathsf{KeyGen}(\mathsf{pp}, 1^n, t) \to (\mathsf{pk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n)$*: On input the public parameters* pp*, the number of decrypting parties* n*, and the decryption threshold* $t \leq n$*, output the public key* pk *and the secret keys* $\mathsf{sk}_1, \ldots, \mathsf{sk}_n$ *for the* n *parties.*
- $\mathsf{Enc}(\mathsf{pp}, \mathsf{pk}, m) \to \mathsf{ct}$*: On input the public parameters* pp*, the public key* pk*, and a message* m*, output a ciphertext* ct*.*
- $\mathsf{Eval}(\mathsf{pp}, \mathsf{pk}, f, \{\mathsf{ct}_i\}_{i\in[k]}) \to \mathsf{ct}_f$*: On input the public parameters* pp*, the public key* pk*, a function* f *in the supported function class with* k *inputs, and a set of* k *ciphertexts* $\{\mathsf{ct}_i\}_{i\in[k]}$*,* deterministically *output the evaluated ciphertext* ct_f*.*
- $\mathsf{PartialDec}(\mathsf{pp}, \mathsf{sk}_i, \mathsf{ct}) \to d_i/\perp$*: On input the public parameters* pp*, a secret key* sk_i *for party* i*, and a ciphertext* ct*, output a partial decryption share* d_i*.*
- $\mathsf{Combine}(\mathsf{pp}, \mathsf{pk}, S, \{d_i\}_{i\in S}, \mathsf{ct}) \to m/\perp$*: On input the public parameters* pp*, the public key* pk*, a set of decryption parties* $S \subseteq [n]$ *with* $|S| \geq t$*, the corresponding partial decryption shares* $\{d_i\}_{i\in S}$*, and a ciphertext* ct*, output the decrypted message* m *reconstructed from the partial decryption shares.*

An HTE scheme should satisfy the following properties:

- **Correctness***: For all* $\lambda, n, t \in \mathbb{N}$*, all sets* $S \subseteq [n]$ *of parties of size* $|S| \geq t$*, and all messages* m *in the message space, it holds that*

$$\Pr\left[\mathsf{Combine}(\mathsf{pp}, \mathsf{pk}, S, \{\mathsf{PartialDec}(\mathsf{pp}, \mathsf{sk}_i, \mathsf{ct})\}_{i\in S}, \mathsf{ct}) = m\right] = 1,$$

  where $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$*,* $(\mathsf{pk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n) \leftarrow \mathsf{KeyGen}(\mathsf{pp}, 1^n, t)$*, and* $\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{pp}, \mathsf{pk}, m)$*.*

- **Evaluation Correctness***: For all* $\lambda, n, t \in \mathbb{N}$*, all sets* $S \subseteq [n]$ *of parties of size* $|S| \geq t$*, all functions* f *in the supported function class with* k *inputs, and all messages* $\{m_i\}_{i\in[k]}$ *in the message space, it holds that*

$$\Pr\left[\mathsf{Combine}(\mathsf{pp}, \mathsf{pk}, S, \{\mathsf{PartialDec}(\mathsf{pp}, \mathsf{sk}_i, \mathsf{ct}_f)\}_{i\in S}, \mathsf{ct}_f) = f(m_1, \ldots, m_k)\right] = 1,$$

  where $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$*,* $(\mathsf{pk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n) \leftarrow \mathsf{KeyGen}(\mathsf{pp}, 1^n, t)$*,* $\mathsf{ct}_i \leftarrow \mathsf{Enc}(\mathsf{pp}, \mathsf{pk}, m_i)$ *for all* $i \in [k]$*, and* $\mathsf{ct}_f \leftarrow \mathsf{Eval}(\mathsf{pp}, \mathsf{pk}, f, \{\mathsf{ct}_i\}_{i\in[k]})$*.*

- **Evaluation Succinctness***: For all functions* f *mapping* $\mathbb{M}^k \to \mathbb{M}$ *in the supported function class, where* $\mathbb{M}$ *is the message space, the evaluated ciphertext* $\mathsf{ct}_f \leftarrow \mathsf{Eval}(\mathsf{pp}, \mathsf{pk}, f, \{\mathsf{ct}_i\}_{i\in[k]})$ *lies in the ciphertext space for message* $m_f = f(m_1, \ldots, m_k)$*, where* $\mathsf{ct}_i \leftarrow \mathsf{Enc}(\mathsf{pp}, \mathsf{pk}, m_i)$ *for all* $i \in [k]$*.*

Definition 20 (Additive and Multiplicative HTE)**.** *For an HTE scheme with message space $\mathbb{Z}_M$ for some modulus $M \in \mathbb{N}$, we say that* HTE *is* additively homomorphic *if it supports the evaluation function $f(m_1, \ldots, m_k) = \sum_{i\in[k]} m_i \bmod M$ for all $k \leq 2^\lambda$. We call such a scheme an* additive HTE *scheme.*

For an HTE scheme with message space $\mathcal{G}$ for some multiplicative group $\mathcal{G}$, we say that HTE *is* multiplicatively homomorphic *if it supports the evaluation function $f(m_1, \ldots, m_k) = \prod_{i\in[k]} m_i$ for all $k \leq 2^\lambda$. We call such a scheme a* multiplicative HTE *scheme.*

Definition 21 (IND-CPA security for HTE)**.** *Let λ be the security parameter, $\mathcal{A}$ be a PPT adversary, and β be a challenge bit. The IND-CPA security experiment $\mathsf{Exp}^{\beta}_{\mathsf{CPA}}(1^\lambda, \mathcal{A})$ of an HTE scheme is defined as follows.*

- *The adversary $\mathcal{A}$ submits the number of decryption parties 1^n, and the decryption threshold t. The challenger runs $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$ and $(\mathsf{pk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n) \leftarrow \mathsf{KeyGen}(\mathsf{pp}, 1^n, t)$, and gives pp and pk to $\mathcal{A}$.*
- *The adversary $\mathcal{A}$ can specify a set of corrupted parties $\mathcal{I} \subseteq [n]$ with $|\mathcal{I}| < t$, and receives the secret keys $\{\mathsf{sk}_i\}_{i\in\mathcal{I}}$. If $|\mathcal{I}| \geq t$, the challenger aborts the experiment and outputs 0.*
- *The adversary $\mathcal{A}$ submits a pair of messages (m_0, m_1) in the message space, and receives the challenge ciphertext $\mathsf{ct}^* \leftarrow \mathsf{Enc}(\mathsf{pp}, \mathsf{pk}, m_\beta)$.*
- *Finally, the adversary $\mathcal{A}$ outputs a guess $b \in \{0, 1\}$, which is the output of the experiment.*

We say the homomorphic threshold encryption scheme is IND-CPA secure if for all PPT adversaries $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that

$$\left|\Pr\left[\mathsf{Exp}^0_{\mathsf{CPA}}(1^\lambda, \mathcal{A}) = 1\right] - \Pr\left[\mathsf{Exp}^1_{\mathsf{CPA}}(1^\lambda, \mathcal{A}) = 1\right]\right| \leq \mathsf{negl}(\lambda).$$

For our application, we additionally need a specific simulation property of the HTE scheme.

Definition 22 (Simulatable HTE)**.** *We say that an HTE scheme is* simulatable *if there exists a PPT simulator with the syntax*

- $\mathsf{Sim}(\mathsf{pp}, \mathsf{pk}, \mathcal{I}, \{\mathsf{sk}_i\}_{i\in\mathcal{I}}, m, r) \to \{d_i\}_{i\in[n]\setminus\mathcal{I}}$,

which satisfies the following property: for all $\lambda, n, t \in \mathbb{N}$ and all PPT adversaries $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for all sets $\mathcal{I} \subseteq [n]$ of parties of size $|\mathcal{I}| < t$, all messages m in the message space, all randomness r used in the encryption algorithm,

$$\left|\begin{array}{l} \Pr\left[\mathcal{A}(\{d_i\}_{i\in[n]\setminus\mathcal{I}}) = 1 \;:\; \forall i \in [n]\setminus\mathcal{I}, d_i \leftarrow \mathsf{PartialDec}(\mathsf{pp}, \mathsf{sk}_i, \mathsf{ct})\right] \\ -\Pr\left[\mathcal{A}(\{d_i\}_{i\in[n]\setminus\mathcal{I}}) = 1 \;:\; \{d_i\} \leftarrow \mathsf{Sim}(\mathsf{pp}, \mathsf{pk}, \mathcal{I}, \{\mathsf{sk}_i\}_{i\in\mathcal{I}}, m, r)\right] \end{array}\right| \leq \mathsf{negl}(\lambda),$$

where $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$, $(\mathsf{pk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n) \leftarrow \mathsf{KeyGen}(\mathsf{pp}, 1^n, t)$, and $\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{pp}, \mathsf{pk}, m; r)$. In other words, the partial decryption shares of the honest parties can be simulated without the knowledge of the honest parties' secret keys, as long as the encryption randomness is known.

8.2 Constructing Multiplicative HTE

In this section, we show how to build a multiplicative HTE scheme for any cyclic group from an additive HTE scheme with sufficiently large message space.

Construction 6. Let $\mathsf{HTE}_{\mathsf{add}}$ be an additive HTE scheme which, on security parameter λ, supports additive homomorphism over $\mathbb{Z}_M$ with $M \geq 2^\lambda$. Let $\mathcal{G} = (g, N)$ be a finite cyclic group with generator g and an upper bound N of its order. Then, we construct a multiplicative HTE scheme HTE over $\mathcal{G}$ as follows.

- $\mathsf{Setup}(1^\lambda, \mathcal{G} = (g, N))$: Set the parameter $\kappa \geq 2\lambda + \log N$. Run $\mathsf{pp}_{\mathsf{add}} \leftarrow \mathsf{HTE}_{\mathsf{add}}.\mathsf{Setup}(1^\kappa)$ to initialize the scheme over $\mathbb{Z}_M$ with $M \geq 2^\kappa \geq 2^{2\lambda} N$. Output the public parameters $\mathsf{pp} = (\mathcal{G}, \mathsf{pp}_{\mathsf{add}})$.
- $\mathsf{KeyGen}(\mathsf{pp}, 1^n, t)$: Run $(\mathsf{pk}_{\mathsf{add}}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n) \leftarrow \mathsf{HTE}_{\mathsf{add}}.\mathsf{KeyGen}(\mathsf{pp}_{\mathsf{add}}, 1^n, t)$ and output $(\mathsf{pk} = \mathsf{pk}_{\mathsf{add}}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n)$, i.e., the scheme uses the underlying keys of $\mathsf{HTE}_{\mathsf{add}}$ directly.
- $\mathsf{Enc}(\mathsf{pp}, \mathsf{pk}, m)$: On message $m \in \mathcal{G}$, sample a random exponent $r \leftarrow [2^\lambda N]$. Compute and output the ciphertext:

$$\mathsf{ct} = (m \cdot g^r, \mathsf{ct}_{\mathsf{add}} = \mathsf{HTE}_{\mathsf{add}}.\mathsf{Enc}(\mathsf{pp}_{\mathsf{add}}, \mathsf{pk}_{\mathsf{add}}, r)).$$

- $\mathsf{Eval}(\mathsf{pp}, \mathsf{pk}, f, \{\mathsf{ct}_i\}_{i\in[k]})$: On input the multiplicative function $f : (m_1, \ldots, m_k) \mapsto \prod_{i\in[k]} m_i$, abort if $k > 2^\lambda$. Otherwise, parse each ciphertext $\mathsf{ct}_i = (c_i, \mathsf{ct}_{\mathsf{add},i})$, and compute the evaluated ciphertext as

$$\mathsf{ct}_f = \left(\prod_{i\in[k]} c_i, \mathsf{ct}_{\mathsf{add},f} = \mathsf{HTE}_{\mathsf{add}}.\mathsf{Eval}(\mathsf{pp}_{\mathsf{add}}, \mathsf{pk}_{\mathsf{add}}, f', \{\mathsf{ct}_{\mathsf{add},i}\}_{i\in[k]}) \right),$$

  where $f' : (r_1, \ldots, r_k) \mapsto \sum_{i\in[k]} r_i$ is the additive function over $\mathbb{Z}_M$ supported by $\mathsf{HTE}_{\mathsf{add}}$.
- $\mathsf{PartialDec}(\mathsf{pp}, \mathsf{sk}_i, \mathsf{ct})$: On input a ciphertext $\mathsf{ct} = (c, \mathsf{ct}_{\mathsf{add}})$, compute and output the partial decryption share $d_i = \mathsf{HTE}_{\mathsf{add}}.\mathsf{PartialDec}(\mathsf{pp}_{\mathsf{add}}, \mathsf{sk}_i, \mathsf{ct}_{\mathsf{add}})$, i.e., the partial decryption share of the additively homomorphic ciphertext $\mathsf{ct}_{\mathsf{add}}$.
- $\mathsf{Combine}(\mathsf{pp}, \mathsf{pk}, S, \{d_i\}_{i\in S}, \mathsf{ct})$: On input a ciphertext $\mathsf{ct} = (c, \mathsf{ct}_{\mathsf{add}})$, compute the decrypted exponent $r = \mathsf{HTE}_{\mathsf{add}}.\mathsf{Combine}(\mathsf{pp}_{\mathsf{add}}, \mathsf{pk}_{\mathsf{add}}, S, \{d_i\}_{i\in S}, \mathsf{ct}_{\mathsf{add}})$, and output the decrypted message $m = c \cdot g^{-r}$.

Theorem 16 (Correctness of Construction 6)**.** *Assuming the correctness, evaluation correctness, and evaluation succinctness of the* $\mathsf{HTE}_{\mathsf{add}}$*, Construction 6 is also correct, evaluation correct, and evaluation succinct.*

Proof. The correctness of the construction follows directly from the correctness of $\mathsf{HTE}_{\mathsf{add}}$. Indeed, for any honest ciphertext

$$\mathsf{ct} = (c, \mathsf{ct}_{\mathsf{add}}) = (m \cdot g^r, \mathsf{HTE}_{\mathsf{add}}.\mathsf{Enc}(\mathsf{pp}_{\mathsf{add}}, \mathsf{pk}_{\mathsf{add}}, r)),$$

each honest partial decryption share is computed by $\mathsf{PartialDec}(\mathsf{pp}, \mathsf{sk}_i, \mathsf{ct}) = \mathsf{HTE}_{\mathsf{add}}.\mathsf{PartialDec}(\mathsf{pp}_{\mathsf{add}}, \mathsf{sk}_i, \mathsf{ct}_{\mathsf{add}})$. Since $r \in [2^\lambda N]$ is an integer smaller than M, the combined decryption satisfies

$$r = \mathsf{HTE}_{\mathsf{add}}.\mathsf{Combine}(\mathsf{pp}_{\mathsf{add}}, \mathsf{pk}_{\mathsf{add}}, S, \{d_i\}_{i\in S}, \mathsf{ct}_{\mathsf{add}})$$

and thus the final decryption of Construction 6 yields the correct plaintext $m = c/g^r$.

The evaluation correctness follows similarly. For a k-ary multiplication function f with $k \leq 2^\lambda$, the evaluated ciphertext is of the form

$$\mathsf{ct}_f = (c_f, \mathsf{ct}_{\mathsf{add},f}) = \left(\prod_{i=1}^{k} c_i, \mathsf{HTE}_{\mathsf{add}}.\mathsf{Eval}(\mathsf{pp}_{\mathsf{add}}, \mathsf{pk}_{\mathsf{add}}, f', \{\mathsf{ct}_{\mathsf{add},i}\}_{i=1}^{k}) \right),$$

where $(c_i, \mathsf{ct}_{\mathsf{add},i}) = (m_i \cdot g^{r_i}, \mathsf{HTE}_{\mathsf{add}}.\mathsf{Enc}(\mathsf{pp}_{\mathsf{add}}, \mathsf{pk}_{\mathsf{add}}, r_i))$ for some randomness $r_i \in [2^\lambda N]$, messages $m_i \in \mathcal{G}$ for all $i \in [k]$, and $f' : (r_1, \ldots, r_k) \mapsto \sum_{i=1}^{k} r_i$ is the k-ary addition function over $\mathbb{Z}_M$.

By the evaluation correctness of $\mathsf{HTE}_{\mathsf{add}}$, the overall decryption outcome of the $\mathsf{HTE}_{\mathsf{add}}$ part satisfies

$$\begin{aligned} &\mathsf{HTE}_{\mathsf{add}}.\mathsf{Combine}(\mathsf{pp}_{\mathsf{add}}, \mathsf{pk}_{\mathsf{add}}, S, \{\mathsf{HTE}_{\mathsf{add}}.\mathsf{PartialDec}(\mathsf{pp}_{\mathsf{add}}, \mathsf{sk}_i, \mathsf{ct}_{\mathsf{add},f})\}_{i \in S}, \mathsf{ct}_{\mathsf{add},f}) \\ &= \sum_{i=1}^{k} r_i \bmod M = \sum_{i=1}^{k} r_i, \end{aligned}$$

where the last equality follows from the guarantee that $\sum_{i=1}^{k} r_i \leq k \cdot 2^\lambda N \leq 2^{2\lambda} N \leq M$. Therefore, the final decryption of the evaluated ciphertext yields the correct plaintext

$$m_f = c_f / g^{\sum_{i=1}^{k} r_i} = \prod_{i=1}^{k} m_i.$$

Finally, it is clear to see that the homomorphic ciphertext

$$\mathsf{ct}_f = (c_f, \mathsf{ct}_{\mathsf{add},f}) \in (m_f \cdot g^{\sum r_i}, \mathsf{HTE}_{\mathsf{add}}.\mathsf{Enc}(\mathsf{pp}_{\mathsf{add}}, \mathsf{pk}_{\mathsf{add}}, \sum r_i)),$$

thereby preserving the evaluation succinctness property. □

Theorem 17. *Assuming* $\mathsf{HTE}_{\mathsf{add}}$ *is IND-CPA secure, Construction 6 is also IND-CPA secure.*

Proof. Consider the following hybrid experiments modified from the IND-CPA experiment of Construction 6, as defined in Definition 21. All the following hybrids only modify the distribution of the challenge ciphertexts.

- $\mathcal{H}_0^\beta$: This is the original IND-CPA experiment $\mathsf{Exp}_{\mathsf{CPA}}^\beta$ as defined in Definition 21. In this hybrid, the challenge ciphertext is generated by

$$\mathsf{ct}^* = (m_\beta \cdot g^r, \mathsf{HTE}_{\mathsf{add}}.\mathsf{Enc}(\mathsf{pp}_{\mathsf{add}}, \mathsf{pk}_{\mathsf{add}}, r)).$$

- $\mathcal{H}_1^\beta$: In this hybrid, the challenge ciphertext is generated by

$$\mathsf{ct}^* = (m_\beta \cdot g^r, \mathsf{HTE}_{\mathsf{add}}.\mathsf{Enc}(\mathsf{pp}_{\mathsf{add}}, \mathsf{pk}_{\mathsf{add}}, 0)).$$

  The indistinguishability between $\mathcal{H}_0^\beta$ and $\mathcal{H}_1^\beta$ follows from the IND-CPA security of $\mathsf{HTE}_{\mathsf{add}}$. Note that in both hybrids, before the challenger responds with the challenge ciphertext, the view of the adversary consists of $\mathsf{pp} = (\mathcal{G}, \mathsf{pp}_{\mathsf{add}})$ and $(\mathsf{pk}, \{\mathsf{sk}_i\}_{i \in I})$, where by construction, everything beyond the public information $\mathcal{G}$ is generated as in the IND-CPA experiment of $\mathsf{HTE}_{\mathsf{add}}$. Therefore the indistinguishability immediately reduces to the IND-CPA security of $\mathsf{HTE}_{\mathsf{add}}$.

- $\mathcal{H}_2^\beta$: In this hybrid, the challenge ciphertext is generated by

$$\mathsf{ct}^* = (g^r, \mathsf{HTE}_{\mathsf{add}}.\mathsf{Enc}(\mathsf{pp}_{\mathsf{add}}, \mathsf{pk}_{\mathsf{add}}, 0)),$$

  $\mathcal{H}_1^\beta$ and $\mathcal{H}_2^\beta$ are statistically indistinguishable. To see this, observe that since $m \in \mathcal{G}$, there exists some exponent $a \leq N$ such that $m = g^a$. By a standard flooding argument, for $r \leftarrow [2^\lambda N]$, the distributions of r and $r + a$ are $2^{-\lambda}$-close. Therefore, the adversary cannot distinguish between $\mathcal{H}_1^\beta$ and $\mathcal{H}_2^\beta$ except with negligible probability.

Finally, observe that $\mathcal{H}_2^0$ and $\mathcal{H}_2^1$ are identical. Combining all the hybrids, we conclude that Construction 6 is IND-CPA secure. □

Finally, we show that simulatability is preserved in Construction 6.

Theorem 18. *If* $\mathsf{HTE}_{\mathsf{add}}$ *is simulatable, then so is Construction 6.*

Proof. Observe that the partial decryption shares and the secret keys of Construction 6 are identical to those of $\mathsf{HTE}_{\mathsf{add}}$. Therefore, the simulator of $\mathsf{HTE}_{\mathsf{add}}$ can be used directly to simulate the partial decryption shares of Construction 6, with r being the message of $\mathsf{HTE}_{\mathsf{add}}$. □

Instantiating multiplicative HTE from DCR For our BTE scheme in Section 5.4, we need a multiplicative HTE scheme for the group QR_N, which is a cyclic group with efficiently computable generator (Lemma 1) and a group order upper bound N.

We adopt the threshold Paillier encryption scheme [Pai99, FPS01] as the additive HTE scheme $\mathsf{HTE}_{\mathsf{add}}$. The threshold Paillier encryption scheme supports additive homomorphism over $\mathbb{Z}_M$ for some RSA modulus M. By sampling M to be a product of two $(\kappa/2 + 1)$-bit primes with leading bit 1, we have the guarantee that $M \geq 2^\kappa$. The scheme supports all t-out-of-n thresholds, with public parameters, public/secret keys, ciphertexts, and partial decryptions all of size $\mathsf{poly}(\lambda)$, independent of the number of decryption parties n. The encryption time is also independent of the number of decryption parties n.

The homomorphic ciphertext of the scheme is also succinct, following the equation

$$c_1 = g^{m_1} r_1^M \bmod M^2, \quad c_2 = g^{m_2} r_2^M \bmod M^2 \implies c_1 \cdot c_2 = g^{m_1+m_2}(r_1 r_2)^M \bmod M^2.$$

Finally, it is pointed out in [FPS01] that the threshold Paillier encryption scheme is simulatable: with $t - 1$ honest partial decryption shares and the decryption outcome, all remaining partial decryption shares can be efficiently simulated via interpolation.

This leads to the following theorem.

Theorem 19. *Assuming the DCR assumption, there exists a homomorphic threshold encryption scheme* HTE *supporting the* BatchKey *operation of Construction 2 that is IND-CPA secure and simulatable. Furthermore, all components of the scheme have size* $\mathsf{poly}(\lambda)$*, the encryption circuit is independent of the number of decryption parties* n*, and the randomness of an evaluated ciphertext can be efficiently derived from the input encryption randomness.*

References

[AAB+24] Marius A. Aardal, Diego F. Aranha, Katharina Boudgoust, Sebastian Kolby, and Akira Takahashi. Aggregating falcon signatures with LaBRADOR. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part I*, volume 14920 of *LNCS*, pages 71–106. Springer, Cham, August 2024.

[ABD+26] Amit Agarwal, Kushal Babel, Sourav Das, Babak Poorebrahim Gilakaye, Arup Mondal, Benny Pinkas, Peter Rindal, and Aayush Yadav. Weighted Batched Threshold Encryption With Applications to Mempool Privacy . In *2026 IEEE Symposium on Security and Privacy (SP)*, pages 2462–2481, Los Alamitos, CA, USA, May 2026. IEEE Computer Society.

[ABI+23] Benny Applebaum, Amos Beimel, Yuval Ishai, Eyal Kushilevitz, Tianren Liu, and Vinod Vaikuntanathan. Succinct computational secret sharing. In Barna Saha and Rocco A. Servedio, editors, *55th ACM STOC*, pages 1553–1566. ACM Press, June 2023.

[ACJT00] Giuseppe Ateniese, Jan Camenisch, Marc Joye, and Gene Tsudik. A practical and provably secure coalition-resistant group signature scheme. In Mihir Bellare, editor, *CRYPTO 2000*, volume 1880 of *LNCS*, pages 255–270. Springer, Berlin, Heidelberg, August 2000.

[ADG+26] Amit Agarwal, Sourav Das, Babak Poorebrahim Gilkalaye, Peter Rindal, and Victor Shoup. BTX: Simple and efficient batch threshold encryption. Cryptology ePrint Archive, Paper 2026/754, 2026.

[AFP25] Amit Agarwal, Rex Fernando, and Benny Pinkas. Efficiently-thresholdizable batched identity based encryption, with applications. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part III*, volume 16002 of *LNCS*, pages 69–100. Springer, Cham, August 2025.

[AGH10] Jae Hyun Ahn, Matthew Green, and Susan Hohenberger. Synchronized aggregate signatures: new definitions, constructions and applications. In Ehab Al-Shaer, Angelos D. Keromytis, and Vitaly Shmatikov, editors, *ACM CCS 2010*, pages 473–484. ACM Press, October 2010.

[AIKW13] Benny Applebaum, Yuval Ishai, Eyal Kushilevitz, and Brent Waters. Encoding functions with constant online rate or how to compress garbled circuits keys. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part II*, volume 8043 of *LNCS*, pages 166–184. Springer, Berlin, Heidelberg, August 2013.

[AMR25] Damiano Abram, Giulio Malavolta, and Lawrence Roy. Key-homomorphic computations for RAM: Fully succinct randomised encodings and more. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part III*, volume 16002 of *LNCS*, pages 236–268. Springer, Cham, August 2025.

[AMR26] Damiano Abram, Giulio Malavolta, and Lawrence Roy. How to authenticate a non-deterministic computation. Cryptology ePrint Archive, Paper 2026/741, 2026.

[AT83] Selim G. Akl and Peter D. Taylor. Cryptographic solution to a problem of access control in a hierarchy. *ACM Trans. Comput. Syst.*, 1(3):239–248, 1983.

[BCCT13] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. Recursive composition and bootstrapping for SNARKS and proof-carrying data. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *45th ACM STOC*, pages 111–120. ACM Press, June 2013.

[BCF+25] Jan Bormet, Arka Rai Choudhuri, Sebastian Faust, Sanjam Garg, Hussien Othman, Guru-Vamsi Policharla, Ziyan Qu, and Mingyuan Wang. BEAST-MEV: Batched

threshold encryption with silent setup for MEV prevention. Cryptology ePrint Archive, Report 2025/1419, 2025.

[BdM94] Josh Cohen Benaloh and Michael de Mare. One-way accumulators: A decentralized alternative to digital sinatures (extended abstract). In Tor Helleseth, editor, *EUROCRYPT'93*, volume 765 of *LNCS*, pages 274–285. Springer, Berlin, Heidelberg, May 1994.

[BDN18] Dan Boneh, Manu Drijvers, and Gregory Neven. Compact multi-signatures for smaller blockchains. In Thomas Peyrin and Steven Galbraith, editors, *ASIACRYPT 2018, Part II*, volume 11273 of *LNCS*, pages 435–464. Springer, Cham, December 2018.

[Béz79] Etienne Bézout. *Théorie générale des équations algébriques*. Ph.-D. Pierres, 1779.

[BFOQ25] Jan Bormet, Sebastian Faust, Hussien Othman, and Ziyan Qu. BEAT-MEV: Epochless approach to batched threshold encryption for MEV prevention. In Lujo Bauer and Giancarlo Pellegrino, editors, *USENIX Security 2025*, pages 3457–3476. USENIX Association, August 2025.

[BGG+18] Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter M. R. Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part I*, volume 10991 of *LNCS*, pages 565–596. Springer, Cham, August 2018.

[BGLS03] Dan Boneh, Craig Gentry, Ben Lynn, and Hovav Shacham. Aggregate and verifiably encrypted signatures from bilinear maps. In Eli Biham, editor, *EUROCRYPT 2003*, volume 2656 of *LNCS*, pages 416–432. Springer, Berlin, Heidelberg, May 2003.

[BGOY07] Alexandra Boldyreva, Craig Gentry, Adam O'Neill, and Dae Hyun Yum. Ordered multisignatures and identity-based sequential aggregate signatures, with applications to secure routing. In Peng Ning, Sabrina De Capitani di Vimercati, and Paul F. Syverson, editors, *ACM CCS 2007*, pages 276–285. ACM Press, October 2007.

[BGR12] Kyle Brogle, Sharon Goldberg, and Leonid Reyzin. Sequential aggregate signatures with lazy verification from trapdoor permutations - (extended abstract). In Xiaoyun Wang and Kazue Sako, editors, *ASIACRYPT 2012*, volume 7658 of *LNCS*, pages 644–662. Springer, Berlin, Heidelberg, December 2012.

[BGW05] Dan Boneh, Craig Gentry, and Brent Waters. Collusion resistant broadcast encryption with short ciphertexts and private keys. In Victor Shoup, editor, *CRYPTO 2005*, volume 3621 of *LNCS*, pages 258–275. Springer, Berlin, Heidelberg, August 2005.

[BJ10] Ali Bagherzandi and Stanislaw Jarecki. Identity-based aggregate and multi-signature schemes based on RSA. In Phong Q. Nguyen and David Pointcheval, editors, *PKC 2010*, volume 6056 of *LNCS*, pages 480–498. Springer, Berlin, Heidelberg, May 2010.

[BK20] Dan Boneh and Sam Kim. One-time and interactive aggregate signatures from lattices, 2020.

[BLT25] Dan Boneh, Evan Laufer, and Ertem Nusret Tas. Batch decryption without epochs and its application to encrypted mempools. Cryptology ePrint Archive, Report 2025/1254, 2025.

[BNN07] Mihir Bellare, Chanathip Namprempre, and Gregory Neven. Unrestricted aggregate signatures. In Lars Arge, Christian Cachin, Tomasz Jurdzinski, and Andrzej Tarlecki, editors, *ICALP 2007*, volume 4596 of *LNCS*, pages 411–422. Springer, Berlin, Heidelberg, July 2007.

[BNRT26] Dan Boneh, Rohit Nema, Arnab Roy, and Ertem Nusret Tas. Efficient batch threshold encryption using partial fraction techniques. In *Advances in Cryptology – CRYPTO 2026*, pages 36–67, 2026.

[BO22] Joseph Bebel and Dev Ojha. Ferveo: Threshold decryption for mempool privacy in BFT networks. Cryptology ePrint Archive, Report 2022/898, 2022.

[Bol03] Alexandra Boldyreva. Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme. In Yvo Desmedt, editor, *PKC 2003*, volume 2567 of *LNCS*, pages 31–46. Springer, Berlin, Heidelberg, January 2003.

[BP97] Niko Bari and Birgit Pfitzmann. Collision-free accumulators and fail-stop signature schemes without trees. In Walter Fumy, editor, *EUROCRYPT'97*, volume 1233 of *LNCS*, pages 480–494. Springer, Berlin, Heidelberg, May 1997.

[BS03] Dan Boneh and Alice Silverberg. Applications of multilinear forms to cryptography. *Contemporary Mathematics*, 324(1), 2003.

[BT23] Katharina Boudgoust and Akira Takahashi. Sequential half-aggregation of lattice-based signatures. In Gene Tsudik, Mauro Conti, Kaitai Liang, and Georgios Smaragdakis, editors, *ESORICS 2023, Part I*, volume 14344 of *LNCS*, pages 270–289. Springer, Cham, September 2023.

[BV22] Zvika Brakerski and Vinod Vaikuntanathan. Lattice-inspired broadcast encryption and succinct ciphertext-policy ABE. In Mark Braverman, editor, *ITCS 2022*, volume 215, pages 28:1–28:20. LIPIcs, January / February 2022.

[BW06] Dan Boneh and Brent Waters. A fully collusion resistant broadcast, trace, and revoke system. In Ari Juels, Rebecca N. Wright, and Sabrina De Capitani di Vimercati, editors, *ACM CCS 2006*, pages 211–220. ACM Press, October / November 2006.

[BW13] Dan Boneh and Brent Waters. Constrained pseudorandom functions and their applications. In Kazue Sako and Palash Sarkar, editors, *ASIACRYPT 2013, Part II*, volume 8270 of *LNCS*, pages 280–300. Springer, Berlin, Heidelberg, December 2013.

[BWZ14] Dan Boneh, Brent Waters, and Mark Zhandry. Low overhead broadcast encryption from multilinear maps. In Juan A. Garay and Rosario Gennaro, editors, *CRYPTO 2014, Part I*, volume 8616 of *LNCS*, pages 206–223. Springer, Berlin, Heidelberg, August 2014.

[BZ14] Dan Boneh and Mark Zhandry. Multiparty key exchange, efficient traitor tracing, and more from indistinguishability obfuscation. In Juan A. Garay and Rosario Gennaro, editors, *CRYPTO 2014, Part I*, volume 8616 of *LNCS*, pages 480–499. Springer, Berlin, Heidelberg, August 2014.

[CGJ+23] Arka Rai Choudhuri, Sanjam Garg, Abhishek Jain, Zhengzhong Jin, and Jiaheng Zhang. Correlation intractability and SNARGs from sub-exponential DDH. In Helena

Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part IV*, volume 14084 of *LNCS*, pages 635–668. Springer, Cham, August 2023.

[CGPP24] Arka Rai Choudhuri, Sanjam Garg, Julien Piet, and Guru-Vamsi Policharla. Mempool privacy via batched threshold encryption: Attacks and defenses. In Davide Balzarotti and Wenyuan Xu, editors, *USENIX Security 2024*. USENIX Association, August 2024.

[CGPW25] Arka Rai Choudhuri, Sanjam Garg, Guru-Vamsi Policharla, and Mingyuan Wang. Practical mempool privacy via one-time setup batched threshold encryption. In Lujo Bauer and Giancarlo Pellegrino, editors, *USENIX Security 2025*, pages 3477–3495. USENIX Association, August 2025.

[CGW15] Jie Chen, Romain Gay, and Hoeteck Wee. Improved dual system ABE in prime-order groups via predicate encodings. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 595–624. Springer, Berlin, Heidelberg, April 2015.

[CHW25] Jeffrey Champion, Yao-Ching Hsieh, and David J. Wu. Registered ABE and adaptively-secure broadcast encryption from succinct LWE. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part III*, volume 16002 of *LNCS*, pages 3–34. Springer, Cham, August 2025.

[CJJ22] Arka Rai Choudhuri, Abhishek Jain, and Zhengzhong Jin. SNARGs for $\mathcal{P}$ from LWE. In *62nd FOCS*, pages 68–79. IEEE Computer Society Press, February 2022.

[CT90] Gerald C. Chick and Stafford E. Tavares. Flexible access control with master keys. In Gilles Brassard, editor, *CRYPTO'89*, volume 435 of *LNCS*, pages 316–322. Springer, New York, August 1990.

[CW24] Jeffrey Champion and David J. Wu. Distributed broadcast encryption from lattices. In Elette Boyle and Mohammad Mahmoody, editors, *TCC 2024, Part III*, volume 15366 of *LNCS*, pages 156–189. Springer, Cham, December 2024.

[DDO+01] Alfredo De Santis, Giovanni Di Crescenzo, Rafail Ostrovsky, Giuseppe Persiano, and Amit Sahai. Robust non-interactive zero knowledge. In Joe Kilian, editor, *CRYPTO 2001*, volume 2139 of *LNCS*, pages 566–598. Springer, Berlin, Heidelberg, August 2001.

[DF90] Yvo Desmedt and Yair Frankel. Threshold cryptosystems. In Gilles Brassard, editor, *CRYPTO'89*, volume 435 of *LNCS*, pages 307–315. Springer, New York, August 1990.

[DGK+20] Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In *2020 IEEE Symposium on Security and Privacy*, pages 910–927. IEEE Computer Society Press, May 2020.

[DGKV22] Lalita Devadas, Rishab Goyal, Yael Kalai, and Vinod Vaikuntanathan. Rate-1 non-interactive arguments for batch-NP and applications. In *63rd FOCS*, pages 1057–1068. IEEE Computer Society Press, October / November 2022.

[DGM24] Jesko Dujmovic, Rachit Garg, and Giulio Malavolta. Time-lock puzzles with efficient batch solving. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part II*, volume 14652 of *LNCS*, pages 311–341. Springer, Cham, May 2024.

[DHSS20] Yarkın Doröz, Jeffrey Hoffstein, Joseph H. Silverman, and Berk Sunar. MMSAT: A scheme for multimessage multiuser signature aggregation. Cryptology ePrint Archive, Report 2020/520, 2020.

[EFF85] Paul Erdős, Peter Frankl, and Zoltán Füredi. Families of finite sets in which no set is covered by the union of r others. *Israel J. Math*, 51(1-2):79–89, 1985.

[EMP16] Rachid El Bansarkhani, Mohamed Saied Emam Mohamed, and Albrecht Petzoldt. MQSAS - A multivariate sequential aggregate signature scheme. In Matt Bishop and Anderson C. A. Nascimento, editors, *ISC 2016*, volume 9866 of *LNCS*, pages 426–439. Springer, Cham, September 2016.

[FHPS13] Eduarda S. V. Freire, Dennis Hofheinz, Kenneth G. Paterson, and Christoph Striecks. Programmable hash functions in the multilinear setting. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part I*, volume 8042 of *LNCS*, pages 513–530. Springer, Berlin, Heidelberg, August 2013.

[Fia90] Amos Fiat. Batch RSA. In Gilles Brassard, editor, *CRYPTO'89*, volume 435 of *LNCS*, pages 175–185. Springer, New York, August 1990.

[FLS90] Uriel Feige, Dror Lapidot, and Adi Shamir. Multiple non-interactive zero knowledge proofs based on a single random string (extended abstract). In *31st FOCS*, pages 308–317. IEEE Computer Society Press, October 1990.

[FLS12] Marc Fischlin, Anja Lehmann, and Dominique Schröder. History-free sequential aggregate signatures. In Ivan Visconti and Roberto De Prisco, editors, *SCN 12*, volume 7485 of *LNCS*, pages 113–130. Springer, Berlin, Heidelberg, September 2012.

[FN94] Amos Fiat and Moni Naor. Broadcast encryption. In Douglas R. Stinson, editor, *CRYPTO'93*, volume 773 of *LNCS*, pages 480–491. Springer, Berlin, Heidelberg, August 1994.

[FPS01] Pierre-Alain Fouque, Guillaume Poupard, and Jacques Stern. Sharing decryption in the context of voting or lotteries. In Yair Frankel, editor, *FC 2000*, volume 1962 of *LNCS*, pages 90–104. Springer, Berlin, Heidelberg, February 2001.

[FPTX25] Rex Fernando, Guru-Vamsi Policharla, Andrei Tonkikh, and Zhuolun Xiang. TrX: Encrypted mempools in high performance BFT protocols. Cryptology ePrint Archive, Report 2025/2032, 2025.

[FSZ22] Nils Fleischhacker, Mark Simkin, and Zhenfei Zhang. Squirrel: Efficient synchronized multi-signatures from lattices. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 1109–1123. ACM Press, November 2022.

[FWW23] Cody Freitag, Brent Waters, and David J. Wu. How to use (plain) witness encryption: Registered ABE, flexible broadcast, and more. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part IV*, volume 14084 of *LNCS*, pages 498–531. Springer, Cham, August 2023.

[GKSW10] Sanjam Garg, Abishek Kumarasubramanian, Amit Sahai, and Brent Waters. Building efficient fully collusion-resilient traitor tracing and revocation schemes. In Ehab Al-Shaer, Angelos D. Keromytis, and Vitaly Shmatikov, editors, *ACM CCS 2010*, pages 121–130. ACM Press, October 2010.

[GKW18] Romain Gay, Lucas Kowalczyk, and Hoeteck Wee. Tight adaptively secure broadcast encryption with short ciphertexts and keys. In Dario Catalano and Roberto De Prisco, editors, *SCN 18*, volume 11035 of *LNCS*, pages 123–139. Springer, Cham, September 2018.

[GL89] Oded Goldreich and Leonid A. Levin. A hard-core predicate for all one-way functions. In *21st ACM STOC*, pages 25–32. ACM Press, May 1989.

[GMR88] Shafi Goldwasser, Silvio Micali, and Ronald L. Rivest. A digital signature scheme secure against adaptive chosen-message attacks. *SIAM Journal on Computing*, 17(2):281–308, 1988.

[GR06] Craig Gentry and Zulfikar Ramzan. Identity-based aggregate signatures. In Moti Yung, Yevgeniy Dodis, Aggelos Kiayias, and Tal Malkin, editors, *PKC 2006*, volume 3958 of *LNCS*, pages 257–273. Springer, Berlin, Heidelberg, April 2006.

[GV22] Rishab Goyal and Vinod Vaikuntanathan. Locally verifiable signature and key aggregation. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part II*, volume 13508 of *LNCS*, pages 761–791. Springer, Cham, August 2022.

[GVW19] Rishab Goyal, Satyanarayana Vusirikala, and Brent Waters. Collusion resistant broadcast and trace from positional witness encryption. In Dongdai Lin and Kazue Sako, editors, *PKC 2019, Part II*, volume 11443 of *LNCS*, pages 3–33. Springer, Cham, April 2019.

[GW09] Craig Gentry and Brent Waters. Adaptive security in broadcast encryption systems (with short ciphertexts). In Antoine Joux, editor, *EUROCRYPT 2009*, volume 5479 of *LNCS*, pages 171–188. Springer, Berlin, Heidelberg, April 2009.

[GWWW26] Junqing Gong, Brent Waters, Hoeteck Wee, and David J. Wu. Threshold batched identity-based encryption from pairings in the plain model. In *EUROCRYPT (5)*, volume 16545 of *Lecture Notes in Computer Science*, pages 548–578. Springer, 2026.

[GY26] Rishab Goyal and Saikumar Yadugiri. Equivocal broadcast encryption: Adaptively-secure optimal distributed broadcast encryption from lattices. In *Advances in Cryptology – CRYPTO 2026*, pages 414–445, 2026.

[Hal35] Peter Hall. On representatives of subsets. *Journal of The London Mathematical Society*, 1935.

[HKW15] Susan Hohenberger, Venkata Koppula, and Brent Waters. Universal signature aggregators. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 3–34. Springer, Berlin, Heidelberg, April 2015.

[HLR07] Chun-Yuan Hsiao, Chi-Jen Lu, and Leonid Reyzin. Conditional computational entropy, or toward separating pseudoentropy from compressibility. In Moni Naor, editor, *EUROCRYPT 2007*, volume 4515 of *LNCS*, pages 169–186. Springer, Berlin, Heidelberg, May 2007.

[HSW13] Susan Hohenberger, Amit Sahai, and Brent Waters. Full domain hash from (leveled) multilinear maps and identity-based aggregate signatures. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part I*, volume 8042 of *LNCS*, pages 494–512. Springer, Berlin, Heidelberg, August 2013.

[HW09] Susan Hohenberger and Brent Waters. Short and stateless signatures from the RSA assumption. In Shai Halevi, editor, *CRYPTO 2009*, volume 5677 of *LNCS*, pages 654–670. Springer, Berlin, Heidelberg, August 2009.

[HW18] Susan Hohenberger and Brent Waters. Synchronized aggregate signatures from the RSA assumption. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 197–229. Springer, Cham, April / May 2018.

[HWW25a] Susan Hohenberger, Brent Waters, and David J. Wu. Pairing-based aggregate signatures without random oracles. In Goichiro Hanaoka and Bo-Yin Yang, editors, *ASIACRYPT 2025, Part VI*, volume 16250 of *LNCS*, pages 271–303. Springer, Singapore, December 2025.

[HWW25b] Yao-Ching Hsieh, Brent Waters, and David J. Wu. A generic approach to adaptively-secure broadcast encryption in the plain model. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part III*, volume 15603 of *LNCS*, pages 336–365. Springer, Cham, May 2025.

[Ita83] Kazuharu Itakura. A public-key cryptosystem suitable for digital multisignature. *NEC research and development*, 71:1–8, 1983.

[KLJD25] Alireza Kavousi, Duc Viet Le, Philipp Jovanovic, and George Danezis. Blindperm: Efficient MEV mitigation with an encrypted mempool and permutation. In Andrei Arusoaie, Emanuel Onica, Michael Spear, and Sara Tucci Piergiovanni, editors, *29th International Conference on Principles of Distributed Systems, OPODIS 2025, Iaşi, Romania, December 3-5, 2025*, volume 361 of *LIPIcs*, pages 36:1–36:21. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025.

[KMW23] Dimitris Kolonelos, Giulio Malavolta, and Hoeteck Wee. Distributed broadcast encryption from bilinear groups. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part V*, volume 14442 of *LNCS*, pages 407–441. Springer, Singapore, December 2023.

[KRS99] Ravi Kumar, Sridhar Rajagopalan, and Amit Sahai. Coding constructions for blacklisting problems without computational assumptions. In Michael J. Wiener, editor, *CRYPTO'99*, volume 1666 of *LNCS*, pages 609–623. Springer, Berlin, Heidelberg, August 1999.

[LLY13a] Kwangsu Lee, Dong Hoon Lee, and Moti Yung. Sequential aggregate signatures made shorter. In Michael J. Jacobson, Jr., Michael E. Locasto, Payman Mohassel, and Reihaneh Safavi-Naini, editors, *ACNS 2013*, volume 7954 of *LNCS*, pages 202–217. Springer, Berlin, Heidelberg, June 2013.

[LLY13b] Kwangsu Lee, Dong Hoon Lee, and Moti Yung. Sequential aggregate signatures with short public keys: Design, analysis and implementation studies. In Kaoru Kurosawa and Goichiro Hanaoka, editors, *PKC 2013*, volume 7778 of *LNCS*, pages 423–442. Springer, Berlin, Heidelberg, February / March 2013.

[LMRS04] Anna Lysyanskaya, Silvio Micali, Leonid Reyzin, and Hovav Shacham. Sequential aggregate signatures from trapdoor permutations. In Christian Cachin and Jan Camenisch, editors, *EUROCRYPT 2004*, volume 3027 of *LNCS*, pages 74–90. Springer, Berlin, Heidelberg, May 2004.

[LOS+06] Steve Lu, Rafail Ostrovsky, Amit Sahai, Hovav Shacham, and Brent Waters. Sequential aggregate signatures and multisignatures without random oracles. In Serge Vaudenay, editor, *EUROCRYPT 2006*, volume 4004 of *LNCS*, pages 465–485. Springer, Berlin, Heidelberg, May / June 2006.

[MT07] Di Ma and Gene Tsudik. Extended abstract: Forward-secure sequential aggregate authentication. In *2007 IEEE Symposium on Security and Privacy*, pages 86–91. IEEE Computer Society Press, May 2007.

[Nev08] Gregory Neven. Efficient sequential aggregate signed data. In Nigel P. Smart, editor, *EUROCRYPT 2008*, volume 4965 of *LNCS*, pages 52–69. Springer, Berlin, Heidelberg, April 2008.

[NRS26] Jonas Nick, Tim Ruffing, and Yannick Seurin. Dahlias: Discrete logarithm-based interactive aggregate signatures. In *EUROCRYPT (1)*, volume 16541 of *Lecture Notes in Computer Science*, pages 215–245. Springer, 2026.

[Oka88] Tatsuaki Okamoto. A digital multisignature scheme using bijective public-key cryptosystems. *ACM Transactions on Computer Systems (TOCS)*, 6(4):432–441, 1988.

[Pai99] Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In Jacques Stern, editor, *EUROCRYPT'99*, volume 1592 of *LNCS*, pages 223–238. Springer, Berlin, Heidelberg, May 1999.

[Pol26] Guru-Vamsi Policharla. A simple batched threshold encryption scheme. Cryptology ePrint Archive, Paper 2026/760, 2026.

[RS09] Markus Rückert and Dominique Schröder. Aggregate and verifiably encrypted signatures from multilinear maps without random oracles. In Jong Hyuk Park, Hsiao-Hwa Chen, Mohammed Atiquzzaman, Changhoon Lee, Tai-Hoon Kim, and Sang-Soo Yeo, editors, *Advances in Information Security and Assurance, Third International Conference and Workshops, ISA 2009, Seoul, Korea, June 25-27, 2009. Proceedings*, volume 5576 of *Lecture Notes in Computer Science*, pages 750–759. Springer, 2009.

[RSA78] Ronald L Rivest, Adi Shamir, and Leonard Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978.

[Sak25] Yusuke Sakai. Aggregate signatures tightly secure under adaptive corruptions. In Goichiro Hanaoka and Bo-Yin Yang, editors, *ASIACRYPT 2025, Part VI*, volume 16250 of *LNCS*, pages 304–336. Springer, Singapore, December 2025.

[SAS24] Sora Suegami, Shinsaku Ashizawa, and Kyohei Shibano. Constant-cost batched partial decryption in threshold encryption. Cryptology ePrint Archive, Paper 2024/762, 2024.

[Sha83] Adi Shamir. On the generation of cryptographically strong pseudorandom sequences. *ACM Trans. Comput. Syst.*, 1(1):38–44, February 1983.

[Sho00] Victor Shoup. Practical threshold signatures. In Bart Preneel, editor, *EUROCRYPT 2000*, volume 1807 of *LNCS*, pages 207–220. Springer, Berlin, Heidelberg, May 2000.

[STY01] Tomas Sander, Amnon Ta-Shma, and Moti Yung. Blind, auditable membership proofs. In Yair Frankel, editor, *FC 2000*, volume 1962 of *LNCS*, pages 53–71. Springer, Berlin, Heidelberg, February 2001.

[Wat09] Brent Waters. Dual system encryption: Realizing fully secure IBE and HIBE under simple assumptions. In Shai Halevi, editor, *CRYPTO 2009*, volume 5677 of *LNCS*, pages 619–636. Springer, Berlin, Heidelberg, August 2009.

[Wee21] Hoeteck Wee. Broadcast encryption with size $N^{1/3}$ and more from k-lin. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part IV*, volume 12828 of *LNCS*, pages 155–178, Virtual Event, August 2021. Springer, Cham.

[Wee22] Hoeteck Wee. Optimal broadcast encryption and CP-ABE from evasive lattice assumptions. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part II*, volume 13276 of *LNCS*, pages 217–241. Springer, Cham, May / June 2022.

[Wee24] Hoeteck Wee. Circuit ABE with $\mathsf{poly}(\mathsf{depth}, \lambda)$-sized ciphertexts and keys from lattices. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part III*, volume 14922 of *LNCS*, pages 178–209. Springer, Cham, August 2024.

[Wee25] Hoeteck Wee. Almost optimal KP and CP-ABE for circuits from succinct LWE. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part III*, volume 15603 of *LNCS*, pages 34–62. Springer, Cham, May 2025.

[WW22] Brent Waters and David J. Wu. Batch arguments for NP and more from standard bilinear group assumptions. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part II*, volume 13508 of *LNCS*, pages 433–463. Springer, Cham, August 2022.

A Deferred Details of Section 5

A.1 Security Definitions of Batched Threshold Encryption

Definition 23 (IND-CCA Security)**.** *Let λ be the security parameter, $\mathcal{A}$ be a PPT adversary, and β be a challenge bit. The IND-CCA security experiment $\mathsf{Exp}^{\beta}_{\mathsf{CCA}}(1^{\lambda}, \mathcal{A})$ is defined as follows.*

- *The adversary $\mathcal{A}$ submits the maximal batch size 1^B, the index space size ℓ, the number of decryption parties 1^n, and the decryption threshold t. The challenger runs $\mathsf{pp} \leftarrow \mathsf{Setup}(1^{\lambda}, 1^B, \ell)$ and $(\mathsf{pk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n) \leftarrow \mathsf{KeyGen}(\mathsf{pp}, 1^n, t)$, and gives pp and pk to $\mathcal{A}$.*
- *The adversary $\mathcal{A}$ can specify a set of corrupted parties $\mathcal{I} \subseteq [n]$ with $|\mathcal{I}| < t$, and receives the secret keys $\{\mathsf{sk}_i\}_{i \in \mathcal{I}}$.*
- *The adversary $\mathcal{A}$ can adaptively make the following queries.*
  - *Challenge query: $\mathcal{A}$ submits a pair of messages (m_0, m_1) and an index $j^* \in [\ell]$, and receives the challenge ciphertext $\mathsf{ct}^* \leftarrow \mathsf{Enc}(\mathsf{pp}, \mathsf{pk}, j^*, m_\beta)$. The adversary can only make one challenge query.*
  - *Decryption query: $\mathcal{A}$ submits a batch of ciphertexts $\{j, \mathsf{ct}_j\}_{j \in S}$ with distinct indices and $|S| \leq B$, and a decryption party $i \in [n] \setminus \mathcal{I}$. The challenger responds with $\perp$ if the challenge*

query has been made and ct^* *is in the batch. Otherwise, the challenger responds with* $d_i = \mathsf{PartialDec}(\mathsf{pp}, \mathsf{sk}_i, \{j, \mathsf{ct}_j\}_{j \in S})$.

- *Finally, the adversary* $\mathcal{A}$ *outputs a guess* $b \in \{0, 1\}$*, which is the output of the experiment.*

We say a BTE scheme is IND-CCA secure if for all PPT adversaries $\mathcal{A}$*, there exists a negligible function* $\mathsf{negl}(\cdot)$ *such that*

$$\left|\Pr\left[\mathsf{Exp}^0_{\mathsf{CCA}}(1^\lambda, \mathcal{A}) = 1\right] - \Pr\left[\mathsf{Exp}^1_{\mathsf{CCA}}(1^\lambda, \mathcal{A}) = 1\right]\right| \leq \mathsf{negl}(\lambda).$$

Definition 24 (Rogue Ciphertext Security)**.** *Let* λ *be the security parameter, and* $\mathcal{A}$ *be a PPT adversary. The rogue ciphertext security experiment* $\mathsf{Exp}_{\mathsf{RC}}(1^\lambda, \mathcal{A})$ *is defined as follows.*

- *The adversary* $\mathcal{A}$ *submits the maximal batch size* 1^B*, the index space size* ℓ*, the number of decryption parties* 1^n*, and the decryption threshold* t*. The challenger runs* $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, 1^B, \ell)$ *and* $(\mathsf{pk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n) \leftarrow \mathsf{KeyGen}(\mathsf{pp}, 1^n, t)$*, and gives* pp*,* pk*, and* $\mathsf{sk}_1, \ldots, \mathsf{sk}_n$ *to* $\mathcal{A}$*.*
- *The adversary submits a challenge index* α *and a message* m*, and receives an honest challenge ciphertext* $\mathsf{ct}_\alpha \leftarrow \mathsf{Enc}(\mathsf{pp}, \mathsf{pk}, \alpha, m)$*.*
- *The adversary* $\mathcal{A}$ *submits a batch of ciphertexts* $\{j, \mathsf{ct}_j\}_{j \in S}$ *with distinct indices, along with a decryption set* $D \subseteq [n]$*. The adversary wins if all the following conditions hold:*
  - *The batch size* $|S| \leq B$*, and the decryption set size* $|D| \geq t$*.*
  - *The challenge ciphertext* $(\alpha, \mathsf{ct}_\alpha)$ *is in the batch.*
  - *Write* d_i *for party* i*'s partial decryption of the submitted batch, and write* $\mathbf{m}'$ *for the output of* $\mathsf{Combine}$ *on these partial decryptions and the submitted batch. The batch is accepted and its decryption output on index* α *is incorrect, i.e.,*

$$\mathbf{m}' \neq \perp \quad \textit{and} \quad \mathbf{m}'[\alpha] \neq m.$$

- *The experiment outputs* 1 *if the adversary wins, and* 0 *otherwise.*

We say a BTE scheme is rogue ciphertext secure if for all PPT adversaries $\mathcal{A}$*, there exists a negligible function* $\mathsf{negl}(\cdot)$ *such that*

$$\Pr\left[\mathsf{Exp}_{\mathsf{RC}}(1^\lambda, \mathcal{A}) = 1\right] \leq \mathsf{negl}(\lambda).$$

A.2 Deferred Security Proof for Construction 3

Lemma 12. *Construction 3 is rogue ciphertext secure (Definition 24).*

Proof. The rogue ciphertext security follows from the simulation-extractability of the NIZKPoK, which guarantees that every ciphertext accepted by the NIZK checks inside $\mathsf{PartialDec}$ is a valid encryption of some message. Formally, consider a hybrid experiment $\mathcal{H}_1$, which makes the following modification to the rogue ciphertext security experiment $\mathsf{Exp}_{\mathsf{RC}}$ defined in Definition 24:

- In the setup algorithm, the challenger runs $(\mathsf{crs}, \mathsf{td}) \leftarrow \mathsf{NIZK.TSetup}(1^\lambda)$ instead of $\mathsf{crs} \leftarrow \mathsf{NIZK.Setup}(1^\lambda)$. When generating the honest challenge ciphertext $\mathsf{ct}_\alpha = (\mathsf{ct}_{\mathsf{KBE},\alpha}, \mathsf{ct}_{\mathsf{HTE},\alpha}, \pi_\alpha)$, it computes $\mathsf{expp}_{\alpha,\mathsf{KBE}} \leftarrow \mathsf{KBE.PreEnc}(\mathsf{pp}_{\mathsf{KBE}}, \alpha)$, sets

$$x_\alpha := (\mathsf{pp}_{\mathsf{KBE}}, \mathsf{expp}_{\alpha,\mathsf{KBE}}, \mathsf{pp}_{\mathsf{HTE}}, \mathsf{pk}_{\mathsf{HTE}}, \alpha, \mathsf{ct}_{\mathsf{KBE},\alpha}, \mathsf{ct}_{\mathsf{HTE},\alpha}),$$

  and replaces the real proof with $\pi_\alpha \leftarrow \mathsf{NIZK.Sim}(\mathsf{crs}, \mathsf{td}, x_\alpha)$. The challenger retains x_α and the honest witness $w_\alpha = (m, h_\alpha, r_{\mathsf{KBE},\alpha}, r_{\mathsf{HTE},\alpha})$.

By the zero-knowledge property of the NIZKPoK, the switch in the crs mode is indistinguishable to the adversary. We thus have that for all PPT adversaries $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that

$$\left|\Pr\left[\mathsf{Exp}_{\mathsf{RC}}(1^\lambda, \mathcal{A}) = 1\right] - \Pr\left[\mathcal{H}_1(1^\lambda, \mathcal{A}) = 1\right]\right| \leq \mathsf{negl}(\lambda).$$

Next, consider a hybrid experiment $\mathcal{H}_2$, which makes the following modifications to $\mathcal{H}_1$:

- For the batch submitted by the adversary, after the NIZK checks inside PartialDec accept and before the experiment checks the final decryption condition, the challenger computes $\mathsf{expp}_{j,\mathsf{KBE}} \leftarrow \mathsf{KBE.PreEnc}(\mathsf{pp}_{\mathsf{KBE}}, j)$ and runs the extraction algorithm of the NIZKPoK to get

$$x_j := (\mathsf{pp}_{\mathsf{KBE}}, \mathsf{expp}_{j,\mathsf{KBE}}, \mathsf{pp}_{\mathsf{HTE}}, \mathsf{pk}_{\mathsf{HTE}}, j, \mathsf{ct}_{\mathsf{KBE},j}, \mathsf{ct}_{\mathsf{HTE},j}),$$
$$w_j = (m_j, h_j, r_{\mathsf{KBE},j}, r_{\mathsf{HTE},j}) \leftarrow \mathsf{NIZK.Ext}(\mathsf{crs}, \mathsf{td}, x_j, \pi_j).$$

  for each ciphertext $\mathsf{ct}_j = (\mathsf{ct}_{\mathsf{KBE},j}, \mathsf{ct}_{\mathsf{HTE},j}, \pi_j)$ with $j \in S \setminus \{\alpha\}$. If $\mathcal{R}(x_j, w_j) = 0$ for any $j \in S \setminus \{\alpha\}$, the experiment aborts and outputs 0. Otherwise, it proceeds to check the remaining winning conditions of the rogue ciphertext security experiment.

It is easy to see that an adversary winning $\mathcal{H}_1$ but not $\mathcal{H}_2$ is an attacker that breaks the simulation-extractability of the NIZKPoK. Thus, for all PPT adversaries $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that

$$\left|\Pr\left[\mathcal{H}_1(1^\lambda, \mathcal{A}) = 1\right] - \Pr\left[\mathcal{H}_2(1^\lambda, \mathcal{A}) = 1\right]\right| \leq \mathsf{negl}(\lambda).$$

Finally, we note that in $\mathcal{H}_2$, if the submitted batch is accepted and all extracted witnesses are valid, each ciphertext ct_j is a valid encryption of some message m_j. Therefore, by the correctness of the BTE scheme, one must have

$$\mathsf{Combine}(\mathsf{pp}, \mathsf{pk}, \{\mathsf{PartialDec}(\mathsf{pp}, \mathsf{sk}_i, \{j, \mathsf{ct}_j\}_{j\in S})\}_{i\in D}, \{j, \mathsf{ct}_j\}_{j\in S})[\alpha] = m,$$

i.e., no adversary can win the $\mathcal{H}_2$ experiment. Combining the above three hybrid experiments, we conclude that for all PPT adversaries $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that

$$\Pr\left[\mathsf{Exp}_{\mathsf{RC}}(1^\lambda, \mathcal{A}) = 1\right] \leq \mathsf{negl}(\lambda),$$

which proves the rogue ciphertext security of Construction 3. □

Lemma 13. *Construction 3 is IND-CCA secure (Definition 23).*

Proof. The proof follows from the following sequence of hybrid experiments.

- $\mathcal{H}_0^\beta$: This is the original IND-CCA security experiment $\mathsf{Exp}_{\mathsf{CCA}}^\beta$ defined in Definition 23.
- $\mathcal{H}_1^\beta$: In this hybrid, the challenger runs $(\mathsf{crs}, \mathsf{td}) \leftarrow \mathsf{NIZK.TSetup}(1^\lambda)$ instead of $\mathsf{crs} \leftarrow \mathsf{NIZK.Setup}(1^\lambda)$ in the setup algorithm. Furthermore, in the challenge query, it computes $\mathsf{expp}_{j^*,\mathsf{KBE}} \leftarrow \mathsf{KBE.PreEnc}(\mathsf{pp}_{\mathsf{KBE}}, j^*)$ and generates the challenge ciphertext $\mathsf{ct}^* = (\mathsf{ct}^*_{\mathsf{KBE}}, \mathsf{ct}^*_{\mathsf{HTE}}, \pi^*)$ with a simulated proof

$$\pi^* \leftarrow \mathsf{NIZK.Sim}(\mathsf{crs}, \mathsf{td}, (\mathsf{pp}_{\mathsf{KBE}}, \mathsf{expp}_{j^*,\mathsf{KBE}}, \mathsf{pp}_{\mathsf{HTE}}, \mathsf{pk}_{\mathsf{HTE}}, j^*, \mathsf{ct}^*_{\mathsf{KBE}}, \mathsf{ct}^*_{\mathsf{HTE}}))$$

  instead of a real proof.

- $\mathcal{H}_2^\beta$: In this hybrid, the challenger performs additional extraction during decryption queries whose NIZK checks inside PartialDec accept. Specifically, after these internal checks accept and before the partial decryptions are computed, the challenger computes $\mathsf{expp}_{j,\mathsf{KBE}} \leftarrow \mathsf{KBE.PreEnc}(\mathsf{pp}_{\mathsf{KBE}}, j)$ and runs the extraction algorithm of the NIZKPoK to get

$$x_j := (\mathsf{pp}_{\mathsf{KBE}}, \mathsf{expp}_{j,\mathsf{KBE}}, \mathsf{pp}_{\mathsf{HTE}}, \mathsf{pk}_{\mathsf{HTE}}, j, \mathsf{ct}_{\mathsf{KBE},j}, \mathsf{ct}_{\mathsf{HTE},j}),$$
$$w_j = (m_j, h_j, r_{\mathsf{KBE},j}, r_{\mathsf{HTE},j}) \leftarrow \mathsf{NIZK.Ext}(\mathsf{crs}, \mathsf{td}, x_j, \pi_j).$$

  for each ciphertext $\mathsf{ct}_j = (\mathsf{ct}_{\mathsf{KBE},j}, \mathsf{ct}_{\mathsf{HTE},j}, \pi_j)$ in the batch. If $\mathcal{R}(x_j, w_j) = 0$ for any $j \in S$, the experiment aborts and outputs 0. Otherwise, it proceeds to compute the partial decryptions, which, by construction, are of the form

$$d_i = \mathsf{HTE.PartialDec}(\mathsf{pp}_{\mathsf{HTE}}, \mathsf{sk}_i, \mathsf{ct}_{\mathsf{HTE},f}),$$

  where $\mathsf{ct}_{\mathsf{HTE},f} = \mathsf{HTE.Eval}(\mathsf{pp}_{\mathsf{HTE}}, \mathsf{pk}_{\mathsf{HTE}}, f_S, \{\mathsf{ct}_{\mathsf{HTE},j}\}_{j\in S})$ is the batched HTE ciphertext for the function

$$f_S(\{h_j\}_{j\in S}) := \mathsf{KBE.BatchKey}(\mathsf{pp}_{\mathsf{KBE}}, \{j, h_j\}_{j\in S}).$$

- $\mathcal{H}_3^\beta$: In this hybrid, the challenger no longer computes the partial decryptions for decryption queries. Instead, it outputs simulated partial decryptions (Definition 22) via

$$\{d_i\}_{i\in[n]\setminus\mathcal{I}} \leftarrow \mathsf{HTE.Sim}(\mathsf{pp}_{\mathsf{HTE}}, \mathsf{pk}_{\mathsf{HTE}}, \mathcal{I}, \{\mathsf{sk}_i\}_{i\in\mathcal{I}}, \mathsf{bk}_S, r_{\mathsf{HTE},S})$$

  where

$$\mathsf{bk}_S = \mathsf{KBE.BatchKey}(\mathsf{pp}_{\mathsf{KBE}}, \{j, h_j\}_{j\in S})$$

  is the plaintext of $\mathsf{ct}_{\mathsf{HTE},f}$, and $r_{\mathsf{HTE},S}$ is its corresponding randomness, derived from the extracted HTE randomness $\{r_{\mathsf{HTE},j}\}_{j\in S}$.

  Note that in this hybrid, the challenger no longer uses the secret keys of the honest parties to compute partial decryptions.

- $\mathcal{H}_4^\beta$: In this hybrid, the challenger generates the challenge ciphertext $\mathsf{ct}^* = (\mathsf{ct}^*_{\mathsf{KBE}}, \mathsf{ct}^*_{\mathsf{HTE}}, \pi^*)$ with a simulated proof π^* and a dummy HTE ciphertext $\mathsf{ct}^*_{\mathsf{HTE}} \leftarrow \mathsf{HTE.Enc}(\mathsf{pp}_{\mathsf{HTE}}, \mathsf{pk}_{\mathsf{HTE}}, 1)$ instead of encrypting the KBE decryption key h^*.

The switch from $\mathcal{H}_0^\beta$ to $\mathcal{H}_1^\beta$ is indistinguishable to the adversary by the zero-knowledge property of the NIZKPoK. The switch from $\mathcal{H}_1^\beta$ to $\mathcal{H}_2^\beta$ follows from the simulation-extractability of the NIZKPoK, as any adversary that can distinguish the two hybrids must provide a proof π_j for an invalid ciphertext that is accepted by the NIZK check inside PartialDec. The only proof simulated by the challenger is π^*, and the IND-CCA experiment rejects every decryption query containing ct^*; hence every proof extracted in $\mathcal{H}_2^\beta$ is distinct from the simulated statement-proof pair. The switch from $\mathcal{H}_2^\beta$ to $\mathcal{H}_3^\beta$ follows from the simulatability of HTE. The switch from $\mathcal{H}_3^\beta$ to $\mathcal{H}_4^\beta$ follows from the IND-CPA security of HTE. Since in $\mathcal{H}_3^\beta$ the challenger no longer needs HTE decryption keys to compute the partial decryptions, the view of the adversary can be simulated using only the view of the IND-CPA security experiment of HTE. Therefore, the real challenge ciphertext $\mathsf{ct}^*_{\mathsf{HTE}}$ can be replaced with a dummy ciphertext. Finally, in $\mathcal{H}_4^\beta$, the KBE decryption key h^* associated with the challenge ciphertext is never used. Therefore, by the security of KBE, the challenge KBE components generated as $(\mathsf{ct}^*_{\mathsf{KBE},b}, h^*_b) \leftarrow \mathsf{KBE.OnlineEnc}(\mathsf{pp}_{\mathsf{KBE}}, \mathsf{expp}_{j^*,\mathsf{KBE}}, m_b, j^*)$ for $b \in \{0,1\}$ are indistinguishable to the adversary.

Combining the above hybrid experiments, we conclude that for all PPT adversaries $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that

$$\left|\Pr\left[\mathsf{Exp}^{0}_{\mathsf{CCA}}(1^{\lambda},\mathcal{A})=1\right]-\Pr\left[\mathsf{Exp}^{1}_{\mathsf{CCA}}(1^{\lambda},\mathcal{A})=1\right]\right|\leq \mathsf{negl}(\lambda),$$

which proves the IND-CCA security of Construction 3. □